The Evolution of AI Office: From Passive Assistants to Goal-Driven Agents

Yihao Liu[*,§1,2], **Yongsiqi Tu**[*,§1,3], **Haoxuan Ma**[*,§1,4], **Wen Wang**[*,§1], **Yang Hao**[1], **Chang Liu**[1], **Li Wang**[1], **Huadong Li**[1], **Dexin Wang**[1], **Zhiyuan Shi**[1], **Jianhe Lin**[1], **Yunhai Wang**[3], **Mengyu Zhou**[†1], **Xiaoxi Jiang**[1], **Guanjun Jiang**[1]

[1]Qwen Large Model Application Team, Alibaba, [2]Peking University, [3]Renmin University of China, [4]Nanjing University

[*]Work done during an internship at Alibaba. [†]Corresponding author. [§]Core contributor.

Large language models are transforming how knowledge workers create, edit, and collaborate on documents, spreadsheets, and slide decks—the central artifacts of professional work. Yet the research landscape remains fragmented: spreadsheet agents, slide generators, and document assistants have advanced in separate communities with their own datasets, metrics, and methods, despite confronting one underlying problem—translating a user's intent into a valid, editable file that can be shared, audited, and built upon. This survey unifies these efforts under a common framing we term *AI Office*, whose premise is simple: what matters is not the model's conversational response but the *committed deliverable*—the file the agent actually leaves behind. Documents, spreadsheets, and slides, we argue, share one iterative loop of observation, editing, and verification, differing principally in what constitutes "correctness" for each. Tracing the evolution from narrow in-application features, through copilots that orchestrate workflows under human supervision, to goal-driven agents that own the entire deliverable, we show that intelligence is migrating out of individual applications and into a model-driven *harness* that coordinates skills, memory, and verification across files. We structure the field along three axes: how agents *operate* on artifacts through code, APIs, and the GUI; how deliverables are *evaluated* along multiple quality dimensions, from structural validity to cross-file consistency; and how *training* can ground model improvement in artifact outcomes rather than surface metrics. We further identify a persistent gap between what benchmarks measure and what users demand—execution, formatting, and iterative refinement all under-represented—and conclude with the open problems separating today's copilots from dependable office agents: cross-artifact consistency, learnable quality standards, and the governance that responsible delegation requires.

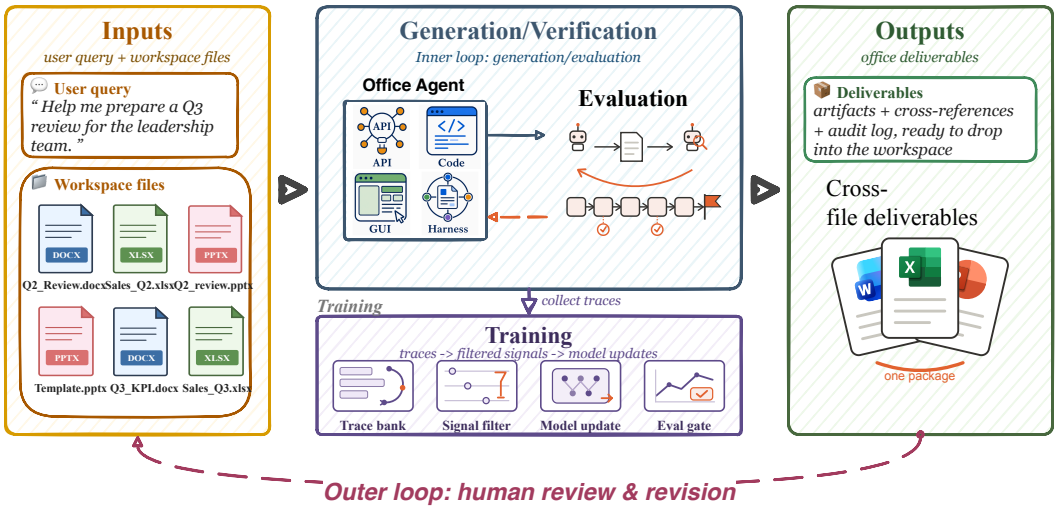


Figure 1 | **Overview of AI Office.** An *office agent* turns a user goal and workspace files *(left)* into validated single- or cross-file *deliverables*—artifacts with cross-references and an audit log *(right)*; we formalize this agent–deliverable view in §2. An *inner* generation–verification loop *(center)* repeats until an evaluation gate passes; the deliverable then re-enters the workspace as an *outer* loop where the user reviews and re-tasks the agent *(bottom arrow)*. Execution traces feed a separate *training* loop *(bottom)*. These elements preview the survey's operation (§3), evaluation (§4), and training (§5) axes, the deployed systems and runtime trends of §6, and the open challenges toward goal-driven agents discussed in §7. Supplementary crosswalks, product landscapes, and detailed per-axis tables are provided in the Appendices.

Corresponding author(s): Mengyu Zhou (zhoumengyu.zmy@alibaba-inc.com)

Contents

1 Introduction **4**

2 AI Office Formulation: From Passive Assistance to Goal-Driven Agency **6**

2.1 The Autonomy Spectrum 6
2.2 Definition and Scope 12
2.3 Office Artifacts as Agentic Environments 13
2.4 From Application-Local Logic to Model-Harness Orchestration 14
2.5 Where We Stand and How the Survey Proceeds 16

3 AI Office Operation: From Features to Orchestration **17**

3.1 Observing Office Artifacts 17
3.2 Three Operation Surfaces 18
3.3 Per-Artifact Operation Pipelines 19
3.4 Cross-Surface Routing and Composition 20
3.5 Harness Engineering 21
3.6 Takeaways 22

4 AI Office Evaluation: From Local Checks to Certification **22**

4.1 Modeling Office Tasks as Workspace Transitions 23
4.2 Judging Deliverables, Not Isolated Outputs 24
4.3 Alignment and Protocols 26
4.4 Benchmark–Deployment Distribution Gap 27
4.5 Takeaways 30

5 AI Office Training: From Demonstrations to Rewards **30**

5.1 Learning Office Actions 32
5.2 On-Policy Distillation for Office Trajectories 32
5.3 Turning Runtime Traces into Reusable Capability 33
5.4 Optimizing Deliverable Outcomes 34
5.5 Team Training and Organizational Adaptation 35
5.6 Takeaways 36

6 Systems, Products, and Deployment Trends **36**

6.1 Deployment Signals and Office Demand 36
6.2 Representative Systems across the Autonomy Spectrum 37
6.3 Skill Libraries and Runtime Architectures 40
6.4 Loop Engineering: Operating the Office Runtime as a Closed Loop 40
6.5 Deployment Evidence for the Shifting Architecture 42

6.6 Takeaways 43

7 Toward Goal-Driven AI Office **43**

A Crosswalk of Representative Work **67**

B Deployed Office Systems in the Product Landscape **69**

C Representative Work by Operation Surface **71**

D Representative Work by Evaluation Target **73**

E Online Office Query Intent Analysis **75**

E.1 Data Processing and Label Normalization 75

E.2 Task-Family Profiles 76

E.3 Summary of Observed Demand 76

F Representative Work by Training Paradigm **82**

1. Introduction

(M)LLMs are evolving from conversational assistants into agents grounded in real software environments (Xie et al., 2024; Drouin et al., 2024; Li et al., 2025). Among these environments, office artifacts such as spreadsheets, slides, and text documents pose particularly demanding challenges. They are the recurring outputs of *knowledge work*,[1] which now accounts for a large share of U.S. labor across fields from engineering and finance to law and public administration (OpenAI, 2026b). Knowledge work is also where language models are projected to have the greatest reach: occupational-exposure studies estimate that around 80% of U.S. workers could have at least 10% of their tasks affected by LLMs, with knowledge-intensive roles among the most exposed (Eloundou et al., 2023), and large-scale measurements of real assistant usage find AI applied most heavily to writing, analytical, and software tasks (Handa et al., 2025). Yet their internal complexity makes them harder for agents to manipulate reliably than free-form text.

This complexity takes a different form in each artifact type. Spreadsheets contain cross-cell dependencies and executable formulas; slides bind semantic content to visual hierarchies and spatial layouts; documents combine intricate formatting, multimodal elements, and long-range structural constraints. Operating on such files requires translating natural-language intent into structure-aware edits over the artifact itself, well beyond generating a textual response.

This structural complexity reshapes how success is defined. In AI Office, the unit of evaluation is the *committed deliverable*: a valid, editable, and trustworthy artifact whose internal structure survives downstream editing, sharing, and auditing intact. Certifying such a file demands verification criteria that couple content correctness with structural validity, presentational quality, and provenance, rather than relying on any single signal such as textual answer matching (Mathew et al., 2021; Tito et al., 2023), interface-level task completion (Zhou et al., 2023), or code execution validity (Jimenez et al., 2024b). We formalize these requirements in §2.

Research on spreadsheet, slide, and document agents has so far evolved in largely separate communities, each with its own representations, action spaces, and evaluation conventions (Li et al., 2023a; Chen et al., 2025; Zheng et al., 2025; Ge et al., 2025; Wang et al., 2024b). Beneath this diversity, however, lies a shared workflow: observing an artifact's current state, planning edits, committing file-level changes, and verifying the result against acceptance criteria. As these artifacts collectively support knowledge work, they pose demanding cases in general-purpose agent benchmarks (Wang et al., 2024d), yet the field still lacks a unified language to define, operate, and evaluate them.

This survey bridges that gap by introducing a unified framework for *AI Office*. Office work spans many digital surfaces, including email, calendars, databases, BI dashboards, and project trackers, but these do not all pose the same challenge for agents. We focus on three core artifact types: documents, spreadsheets, and slide decks. They are persistent, structured, editable files whose internal complexity (executable formulas, spatial layouts, long-range cross-references, provenance constraints) makes the committed-deliverable criterion especially demanding. Adjacent objects such as email, calendar events, and code enter the discussion when they contribute to a cross-artifact deliverable package, but are not primary subjects; we exclude general web browsing, pure conversational QA, isolated tool calls, artifact-agnostic GUI tasks, and specialized applications such as CAD or BI dashboards. We define *office agents* as (M)LLM systems that iteratively revise these artifacts and are evaluated on the committed file state rather than on free-form responses (§2). The committed deliverable, rather than any single prompt–response exchange, is the central object of analysis throughout this work.

To organize the growing literature, we divide the evolution from passive assistance to goal-driven agency into three stages along an autonomy spectrum (Figure 2). At one end, *AI in Office* provides bounded assistance: the user supplies a local intent, and the agent executes a few reversible edits, such as writing a formula or reformatting a slide title. In the middle, the *Office Copilot* orchestrates reasoning, retrieval, and synthesis across multiple artifacts within a single request, while the user reviews and approves each proposed change. At the other end, *AI as Office*—a conceptual endpoint on the autonomy spectrum, not a claim about current unsupervised capability—accepts a high-level goal and takes responsibility for a long, cross-artifact workflow, planning, validating, and recovering from errors over many steps. Most real systems sit between these poles, acting locally for small edits and taking on more of the workflow when given a broader goal.

The evolution from passive assistance to goal-driven agency hinges on three interdependent questions:

[1]The term, coined by Peter Drucker in 1959, denotes occupations whose primary inputs and outputs are symbols rather than crops or manufactured goods.

what methods translate intent into artifact edits (*operation*, §3), what criteria make a deliverable valid and acceptable (*evaluation*, §4), and what training paradigms ground model improvement in deliverable outcomes (*training*, §5). Figure 2 situates representative systems by artifact family and autonomy stage, while the operation, evaluation, and training dimensions are developed in the sections that follow.

Relation to prior surveys. A growing body of surveys addresses individual facets of this landscape. Tabular reasoning surveys cover table understanding, question answering, and fact verification, but treat the spreadsheet primarily as a static data source, not an editable artifact with executable semantics (Lu et al., 2025). Document understanding surveys catalog advances in layout-aware reading and multimodal extraction (Ding et al., 2026), yet focus on comprehension and not on how agents produce or revise documents. Slide-focused work reviews generation pipelines and design quality (Zheng et al., 2025; Ge et al., 2025), but remains confined to a single artifact type. Each line of work contributes valuable technical depth within its scope; what is missing is a unified perspective that treats all three artifact types as instances of one shared problem: producing committed deliverables that must satisfy coupled verification criteria. To our knowledge, no prior survey adopts this deliverable-centric framing across documents, spreadsheets, and slides under a committed-deliverable lens while crosswalking representative systems, benchmarks, and training methods against it (Table 4).

Methodology. This is a framework-driven survey, not a systematic literature review with exhaustive recall. We searched ACL Anthology, arXiv, OpenReview, and recent proceedings of major ML/NLP/HCI/vision venues using query families that combined artifact terms (*spreadsheet*, *Excel*, *slide*, *PowerPoint*, *document*, *PDF*, *office*) with agent terms (*copilot*, *GUI agent*, *computer use*, *tool use*, *benchmark*, *reinforcement learning*, *verifier*, *skill*, *harness*). We included works when they (i) target a persistent office artifact or an adjacent agent environment that directly informs office deliverables, and (ii) contribute a benchmark, system, method, evaluation protocol, or training signal tied to a final workspace state. We excluded pure chat QA, generic web browsing, isolated tool-calling papers without artifact state, and specialized professional software whose artifacts do not enter the document–spreadsheet–slide lifecycle. For the arXiv version, we updated the evidence base through June 2026, adding several post-ARR benchmarks and deployment reports that appeared after the initial May-cycle draft. Peer-reviewed papers, arXiv preprints, benchmarks, and technical reports form the evidence base for technical claims. Product documentation, telemetry reports, and startup case studies are used only as demand or deployment signals; they motivate the problem distribution and runtime direction but are not treated as controlled capability evidence or product comparisons. Product names and trademarks are used only to identify public systems and protocols; their inclusion does not imply endorsement, ranking, or controlled capability comparison by the authors or their affiliations. All trademarks mentioned in this paper are the property of their respective owners. Each included work was coded along four axes that organize the paper: *artifact*, *operation surface* (§3), *evaluation target* (§4), and *training signal* (§5). Borderline cases were discussed until the scope decision was consistent with the committed-deliverable criterion; we do not report formal inter-annotator agreement. The resulting crosswalk is provided in Appendix A, Table 4.

Contributions and paper type. The paper makes three contributions. First, it defines *AI Office* as a deliverable-centric agent problem: the unit of analysis is not a response, action trace, or GUI completion flag, but the committed state of editable office artifacts. Second, it synthesizes the literature into a taxonomy and operational framework that links artifacts, autonomy, operation, evaluation, and training (Figure 1): agents observe and edit persistent artifacts; evaluation certifies structural validity, correctness, presentation, editability, provenance, and package consistency; training should therefore learn from traces and rewards grounded in those same artifact states. Third, it adds a demand-aware view of benchmark design by comparing surveyed benchmark coverage with aggregate online-query intent distributions, showing where current evaluations overrepresent understanding tasks and underrepresent concrete execution, formatting, iterative refinement, and delivery-oriented work. The contribution is therefore conceptual and methodological: we do not introduce a new benchmark or model, and we do not claim that product deployments prove technical capability. Instead, the survey provides a vocabulary, evidence map, and set of design constraints for evaluating and training goal-driven office agents.

Paper organization. We begin with preliminaries that present the taxonomy of AI Office research, formalize office agents as iterative revisers of persistent artifacts judged on committed file state, and advance the central claim that heterogeneous office artifacts instantiate one agent–environment template while differing in how their deliverables are verified (§2). Operation (§3) traces the shift from invoking single features to orchestrating multi-surface workflows; evaluation (§4) turns deliverable quality into certifiable criteria and exposes a demand-aware benchmark gap; and training (§5) shows how SFT, OPD, skills, and RL can be grounded

in artifact states and verifiers. We then use deployed systems and products only to contextualize runtime trends across the autonomy spectrum (§6), and conclude by identifying what remains before dependable goal-driven office agents can be realized (§7).

2. AI Office Formulation: From Passive Assistance to Goal-Driven Agency

Office systems have progressed from bounded features that assist individual edits to agents accountable for entire deliverables. This section traces that progression and formalizes the structure it reveals. We trace the autonomy spectrum through three stages (§2.1), formalize the iterative generation and verification loop shared by the later stages (§2.2), and show that spreadsheets, slides, and documents instantiate the same loop while differing chiefly in their verification criterion (§2.3).

2.1. The Autonomy Spectrum

How much of a task does the system own, and what remains the user's responsibility? We divide the answer into three stages. *AI in Office* (Stage I) embeds bounded features inside individual applications; the *Office Copilot* (Stage II) orchestrates multi-artifact workflows under user supervision; *AI as Office* (Stage III), a conceptual endpoint on the autonomy spectrum, accepts a high-level goal and autonomously decomposes, executes, verifies, and delivers the result. Crucially, these stages classify *modes of interaction and levels of delegation*, not products. A single deployed product typically spans several of them—defaulting to a bounded edit for a small request and escalating to a supervised or delegated workflow for a broader goal—and tends to migrate upward over time as its underlying models and harness mature. Throughout, we name products and features only to illustrate the characteristic behavior of a stage at the time of writing, not to assign a fixed label: the same offering (for example, a "Copilot") may expose a Stage I feature in one interaction and drive a Stage III workflow in another. Figures 3–5 illustrate this progression by holding the office setting fixed while shifting the locus of control.

Figure 2 positions representative products and research systems along two dimensions: artifact type (horizontal) and autonomy level (vertical). Two coupled gradients run from bottom to top: each higher tier is both more general in design and more capable in the scenarios it handles. At the base, AI in Office systems are partitioned by artifact type *and* operation surface (code, API, GUI), each targeting a specific artifact through a specific interface and yielding narrow, task-specialized tools such as SpreadsheetCoder for formula generation (Chen et al., 2021b), LayoutGPT for slide layout (Feng et al., 2023), and CoEdIT for text editing (Raheja et al., 2023). The middle tier broadens: copilot-style assistants—such as those in Microsoft 365, Google Workspace, Beautiful.ai, and Notion—together with research prototypes for data analysis, slide generation, and long-form writing, compose Stage I features into multi-step workflows that operate within and increasingly across artifact types. At the top, AI as Office systems adopt a general-purpose design: runtime harnesses such as Claude Code (Anthropic, 2025b) and Codex (OpenAI, 2025c) inherit strong cross-domain capabilities from their underlying models, while artifact-specific skill libraries (Anthropic, 2025e; MiniMax, 2026) extend those models with office-domain procedures so the harness can plan, execute, and verify deliverables across all artifact types. The progression is therefore not a parallel replacement but an upward extension: each higher tier expands the range of supported scenarios, from single-operation features to supervised multi-step workflows to fully autonomous, cross-artifact delegation. This upward extension is itself made possible by foundation-model scaling: as models grow (Kaplan et al., 2020; Brown et al., 2020) and emergent capabilities arrive at sufficient size (Wei et al., 2022; OpenAI et al., 2023; Bubeck et al., 2023), the general abilities of higher-tier systems can match or surpass earlier solutions purpose-built for a single domain (Bommasani et al., 2022; Nori et al., 2023). The pattern is already visible in office tasks: GPT-4 with appropriate spreadsheet encoding outperforms dedicated table-detection models without task-specific fine-tuning (Dong et al., 2024); general-purpose LLMs trained with reinforcement learning surpass pipeline approaches on realistic multi-step spreadsheet manipulation (Chi et al., 2026); and leading multimodal models match or exceed specialized architectures on document understanding (OpenAI et al., 2023; Team, 2023).

AI in Office: bounded assistance (Stage I). Figure 3 illustrates three representative examples. On the left, a document editor flags a grammatical error ("analyze" → "analyzed") and offers Accept/Dismiss buttons; the user decides on a single word-level correction. In the middle, Excel's Flash Fill infers a first-name/last-name extraction pattern from one user-provided example and propagates it down the column, while a Macro Recorder captures manual steps for later replay. On the right, PowerPoint Designer suggests alternative

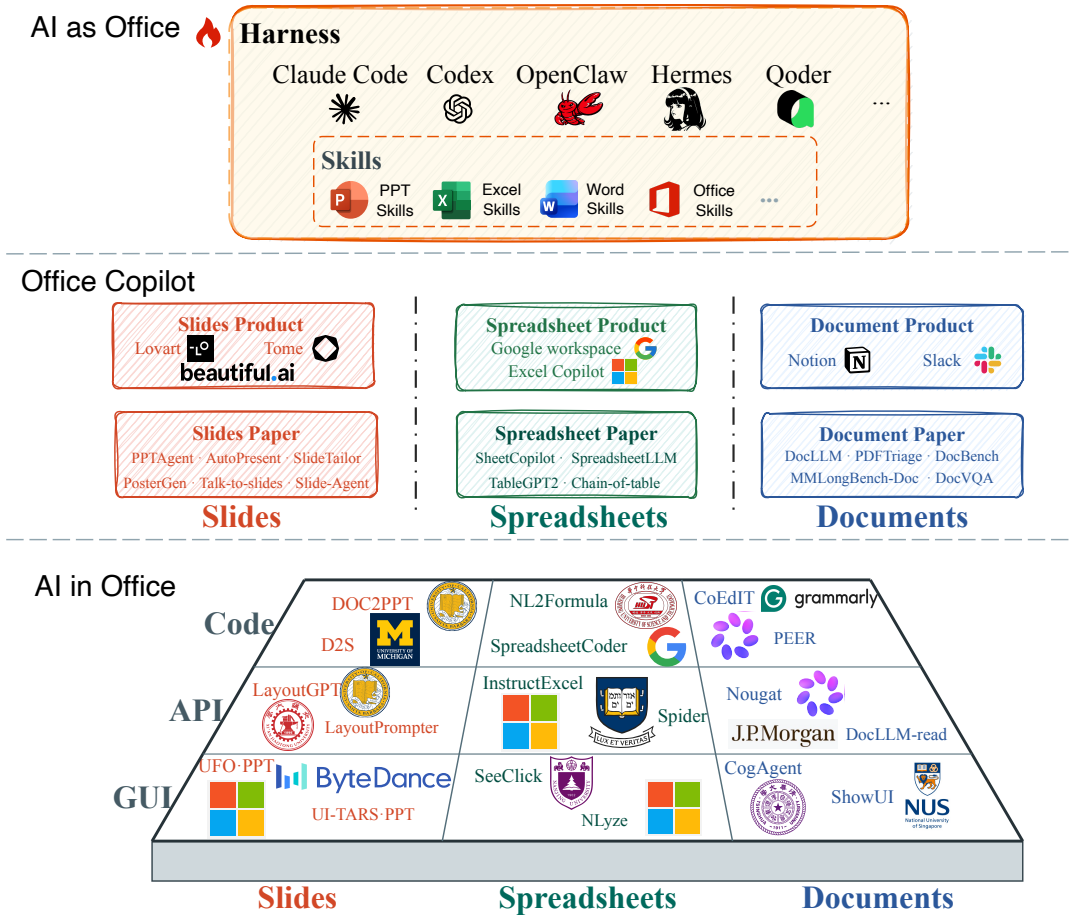


Figure 2 | **Landscape of AI Office.** We organize the literature along two dimensions: the target persistent artifacts (horizontal) and the agent autonomy spectrum (vertical). Systems evolve upward from bounded AI in Office on single artifacts to autonomous AI as Office frameworks capable of cross-artifact delegation. The map is representative, not exhaustive; a complete roster appears in Table 5 (Appendix B).

layout options for an existing slide; the user browses thumbnails and selects one or keeps the current layout. In each panel, AI is confined to one artifact and one operation: the user triggers the feature, inspects the result, and decides whether to accept.

A defining characteristic of this stage is *fragmentation*: each application scenario has spawned its own specialized technique, and no single system covers even one artifact type end to end. As the bottom layer of Figure 2 shows, the result is a large number of narrow tools, one per artifact-and-interface combination. For spreadsheets alone, Flash Fill learns data-transformation patterns from examples (Gulwani, 2011); SpreadsheetCoder and NL2Formula generate individual formulas from natural-language descriptions (Chen et al., 2021b; Zhao et al., 2024); NLyze lets users express data manipulations in natural language (Gulwani & Marron, 2014); Auto-Tables synthesizes multi-step transformations to relationalize messy tables (Li et al., 2023c); FormaT5 generates conditional-formatting rules from natural-language specifications (Singh et al., 2023); and Text2Chart31 produces charts from textual descriptions (Pesaran Zadeh et al., 2024), with each addressing only one facet of spreadsheet work. The same pattern holds for documents, where grammar and style correction (Microsoft Editor, Grammarly) (Microsoft, 2025a; Grammarly, 2024), layout-aware document understanding (LayoutLMv2, LayoutLMv3, DocVQA) (Xu et al., 2021; Huang et al., 2022; Mathew et al., 2021), instruction-tuned rewriting (CoEdIT, mEdIT, RewriteLM) (Raheja et al., 2023, 2024; Shu et al., 2024), and collaborative revision planning (PEER) (Schick et al., 2022) each target a single editing or reading operation in isolation. For slides, layout generation (LayoutGPT, LayoutPrompter) (Feng et al., 2023; Lin et al., 2023), controllable template synthesis (Desigen) (Weng et al., 2024), and document-to-slide conversion (DOC2PPT, D2S) (Fu et al., 2022; Sun et al., 2021) each handle one step of the creation process, while single-slide design

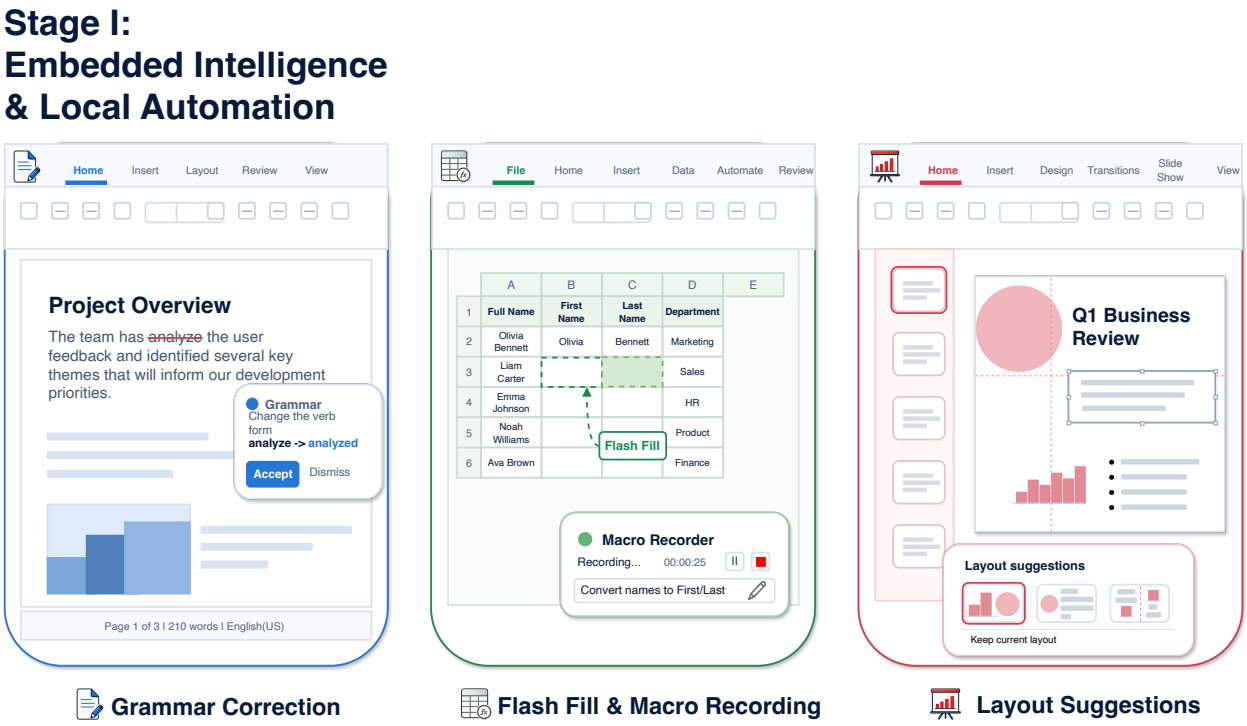


Figure 3 | **Stage I: AI in Office.** AI appears as bounded features inside individual applications. The user remains the planner and operator: they choose the context, trigger a local function (correction, formula suggestion, formatting, summarization), and manually decide whether to accept. Edits are narrow, reversible, and confined to a single application, and the user alone judges whether to accept them—there is no systematic verification step.

tools such as PowerPoint Designer (Microsoft, 2025c), Beautiful.ai (Beautiful.ai, 2023), and Gamma (Gamma, 2023) offer layout recommendations within the editing loop. This fragmentation mirrors what OpenAI's knowledge-work report calls *piecemeal digitization*: successive waves of office software, from email and collaborative documents through spreadsheets, dashboards, ticketing systems, CRMs, knowledge bases, chat tools, and SaaS applications, each solved a local problem but added another disconnected tool, so the office became digital without becoming integrated (OpenAI, 2026b). As a result, assembling a multi-artifact deliverable requires the user to manually chain dozens of specialized features across separate applications; the system optimizes *local convenience* and does not manage an end-to-end workflow, and cross-artifact consistency remains entirely the user's responsibility. The same report identifies three frictions that fragmentation imposes on daily knowledge work: *search* (finding the right file, dataset, or expert across sprawling systems), *coordination* (moving information through teams, tools, and formats), and *approval and verification* (getting work accepted and ensuring it holds up in practice) (OpenAI, 2026b). These map directly onto the dimensions this survey examines: operation (§3) addresses search and coordination friction; evaluation (§4) addresses verification friction; and training (§5) improves the agent's ability to reduce all three.

Office Copilot: orchestrated assistance (Stage II). The second stage shifts control to a conversational copilot that accepts natural-language goals and interprets higher-level intent that Stage I features cannot parse. As Figure 4 illustrates, a user can express a compound request such as "draft a market analysis, use the sales data to create a table and chart, then create three slides for the management update," and the copilot identifies the relevant data sources, selects appropriate output formats, and prepares coordinated drafts across a document, a spreadsheet, and a slide deck, with the user reviewing, correcting, or approving key outputs as the workflow proceeds. The copilot therefore covers substantially broader requirements than bounded features, but parts of the workflow remain constrained by model reliability, context understanding, and cross-artifact consistency. Users may still need to decompose a business need into supervisable subgoals and intervene for judgment, correction, and approval at key points, yet they no longer specify low-level implementation details such as exact formulas, layout operations, or wording changes.

On the product side, the first wave of office copilots (2023–2024) exemplified this stage, their supervised, review-each-step form reflecting the model reliability and context limits of the period: Microsoft 365 Copilot drafts documents and generates linked charts across Word, Excel, and PowerPoint (Microsoft, 2023); Excel Copilot performs formula generation, data analysis, and chart creation within the spreadsheet surface (Microsoft, 2025b); Google Workspace's Gemini integration performs context-aware drafting across Docs, Sheets, and Slides (Google Workspace, 2025a; Google, 2025); Notion AI chains retrieval with structured editing in a wiki-style workspace (Notion Labs, 2023); and Beautiful.ai and Tome generate complete slide decks from natural-language prompts, automating layout and visual design (Beautiful.ai, 2023; Tome App AI, 2023). On the research side, slide-focused prototypes span the full generation pipeline: PPTAgent extracts reference-based templates and generates decks through a multi-agent pipeline (Zheng et al., 2025); AutoPresent generates slides from free-form user instructions (Ge et al., 2025); SlideTailor personalizes scientific-paper-to-slide generation (Zeng et al., 2026); PosterGen produces research posters via multi-agent collaboration (Zhang et al., 2025b); Talk-to-Slides enables language-driven slide editing (Jung et al., 2025); and Slide-Agent orchestrates deck generation with verifiable layout rewards (Z.AI, 2026). For spreadsheets and data-centric work, SheetCopilot orchestrates multi-step spreadsheet operations from natural-language instructions (Li et al., 2023a); Data-Copilot turns a natural-language analysis goal into a workflow over large data resources, covering task decomposition, data discovery, transformation, and result synthesis (Zhang et al., 2023); SpreadsheetLLM addresses the encoding challenge of large, sparse workbooks for LLM consumption (Dong et al., 2024); and TableGPT2 integrates tabular data understanding as a general-purpose multimodal capability (Su et al., 2024). For documents and writing, Stage II copilots handle compound requests over a document context: they organize reading, evidence retrieval, revision, and drafting within a single supervised interaction instead of exposing standalone document-understanding functions. DocLLM and mPLUG-DocOwl illustrate multimodal capabilities for reading rendered documents and PDFs (Wang et al., 2024b; Ye et al., 2023a; Hu et al., 2025a); editing-oriented LLMs such as CoEdIT, mEdIT, RewriteLM, and PEER show how natural-language instructions can drive localized rewrites and revision planning (Raheja et al., 2023, 2024; Shu et al., 2024; Schick et al., 2022); and STORM extends the pattern to long-form article production by coordinating source discovery, question-driven information seeking, outline construction, and grounded drafting (Shao et al., 2024a). The user still owns final acceptance: Stage II copilots can draft, revise, analyze, and assemble intermediate artifacts, but their cross-application control and package-level consistency remain weaker than full delegation. Product evolution reflects this boundary: Microsoft has extended Copilot from assistant-style drafting toward actions and multi-agent orchestration (Microsoft, 2023; Microsoft 365 Blog, 2024, 2025b), Anthropic has exposed richer computer-use and reusable-skill capabilities around Claude (Anthropic, 2024a, 2025c,d,e), and OpenAI has moved from desktop access and agent tools toward ChatGPT Agent and computer-using agents (OpenAI, 2025b,e,a,d). The boundary is fluid, not fixed: early copilots emphasized simple drafting, retrieval, and in-application assistance (Microsoft, 2023; Google Workspace, 2025a; Microsoft, 2025b), while recent computer-use systems and environment-based RL improve multi-step action, recovery, and delegation (Anthropic, 2024a; OpenAI, 2025d; Lai et al., 2025; Bai et al., 2024a). Stage II therefore denotes a regime that is increasingly capable but still dependent on human judgment for task decomposition, error correction, and final deliverable acceptance.

AI as Office: autonomous delegation (Stage III). We use *AI as Office* as a conceptual endpoint on the autonomy spectrum, not as a claim that current systems can be trusted with unsupervised office work; in current deployments, human acceptance, policy constraints, and artifact-level verification remain necessary. At the far end of the spectrum, the user specifies a high-level business goal and delegates the path from intent to deliverable package. As Figure 5 shows, the agent maintains a plan over multiple steps, tracks the state of each artifact and the dependencies among files, and uses retry loops, quality checks, and structured handoffs to iterate toward a finished package across documents, spreadsheets, and slides. Planning, self-verification, recovery from failed intermediate results, and cross-file consistency are now the agent's responsibilities, marking the shift from supervised assistance to delegated execution.

What distinguishes Stage III from Stage II is *accountability for the committed workspace*: the agent needs to plan over a long horizon, preserve provenance, handle errors, and keep the spreadsheet, document, and deck mutually consistent before surfacing a deliverable for final human acceptance. We are in the transition into this stage now. Recent foundation models have approached a practical capability range for office delegation: scaling and emergent capabilities make heterogeneous artifact understanding possible (Kaplan et al., 2020; Wei et al., 2022; OpenAI et al., 2023; Team, 2023; Bubeck et al., 2023), and recent reasoning, multimodal, and computer-use systems make grounded action over multiple steps realistic (Google DeepMind, 2025; Anthropic, 2025a; OpenAI, 2025f,d; Lai et al., 2025; Zhang et al., 2026). A promising architecture once this range is

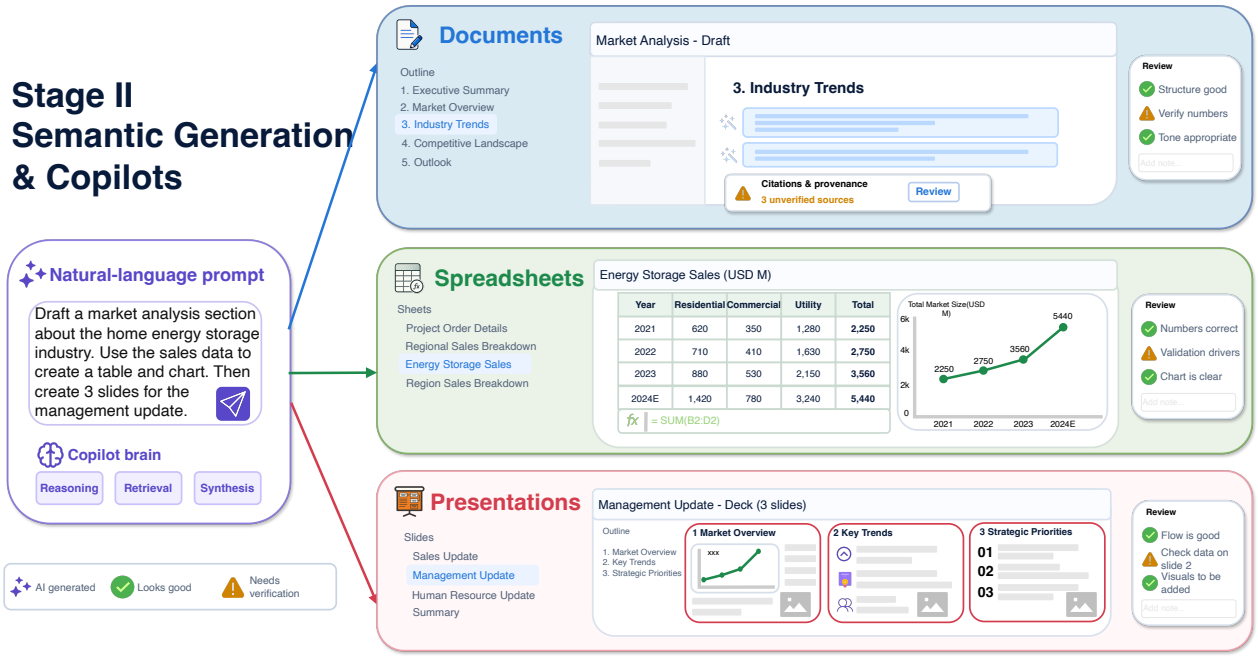


Figure 4 | **Stage II: Office Copilot.** A conversational copilot accepts a natural-language goal, retrieves relevant office context, and prepares edits across documents, spreadsheets, or slides. The user supervises key outputs and intervenes for judgment, correction, or approval. The unit of work shifts from single feature invocation to supervised workflow.

reached is a harness centered on deliverables: a runtime that maintains workspace state, loads reusable skills, coordinates procedures for each artifact type, and carries intermediate evidence across actions.

Recent systems make this direction visible. Anthropic's computer-use work, Claude Code, and reusable Skills, together with OpenAI's agent tools, computer-using agent, ChatGPT Agent, Codex CLI, and enterprise orchestration mechanisms, all point toward persistent runtimes that combine general models with reusable artifact skills and carry state over a longer task (Anthropic, 2024a, 2025b,d,e; OpenAI, 2025e,d,a,c; Microsoft 365 Blog, 2025b; Microsoft Learn, 2026a).

The capability stack also changes. A Stage III system needs three ingredients that a single model call cannot supply: task memory that persists beyond the current prompt; reusable skills for documents, spreadsheets, slides, and PDFs; and verification loops that catch broken formulas, stale citations, inconsistent numbers, and layout failures before the user sees the package. Skill libraries and agent runtimes meet the first two needs by packaging procedural knowledge, validators, and repair routines that a general model can reuse across deliverables (Anthropic, 2025d,e; OpenAI, 2025c; Li et al., 2026a; Xu & Yan, 2026). Domain-specific agent work in spreadsheets and slides addresses the third, using feedback, rewards, or task-specific procedures to improve artifact-level execution and verification (Chi et al., 2026; Z.AI, 2026; Zeng et al., 2026). At the same time, enterprise systems are adopting delegation structures in which specialized workers handle individual subtasks while an orchestrator maintains the package-level goal (Microsoft 365 Blog, 2025b).

Current systems should therefore be read as partial steps toward Stage III, not as complete solutions. They demonstrate planning, workspace state management, execution over office artifacts, and recovery across multiple steps, but reliable office delegation still requires stronger guarantees on provenance, organizational policy, cross-file consistency, and failure recovery. Recent evidence that LLMs can corrupt documents under delegated editing reinforces this point: current models support useful delegation, not unsupervised trust (Laban et al., 2026). The user's role changes accordingly: instead of supervising every draft, the user specifies intent, constraints, and acceptance criteria, then audits the completed package or intervenes when the agent escalates an ambiguity. Early deployment data already shows this shift in practice: roughly half of Codex users now run multiple tasks simultaneously, and the emerging pattern is one in which the user orchestrates workstreams instead of executing a single task at a time (OpenAI, 2026b). In practice, the three stages are not mutually

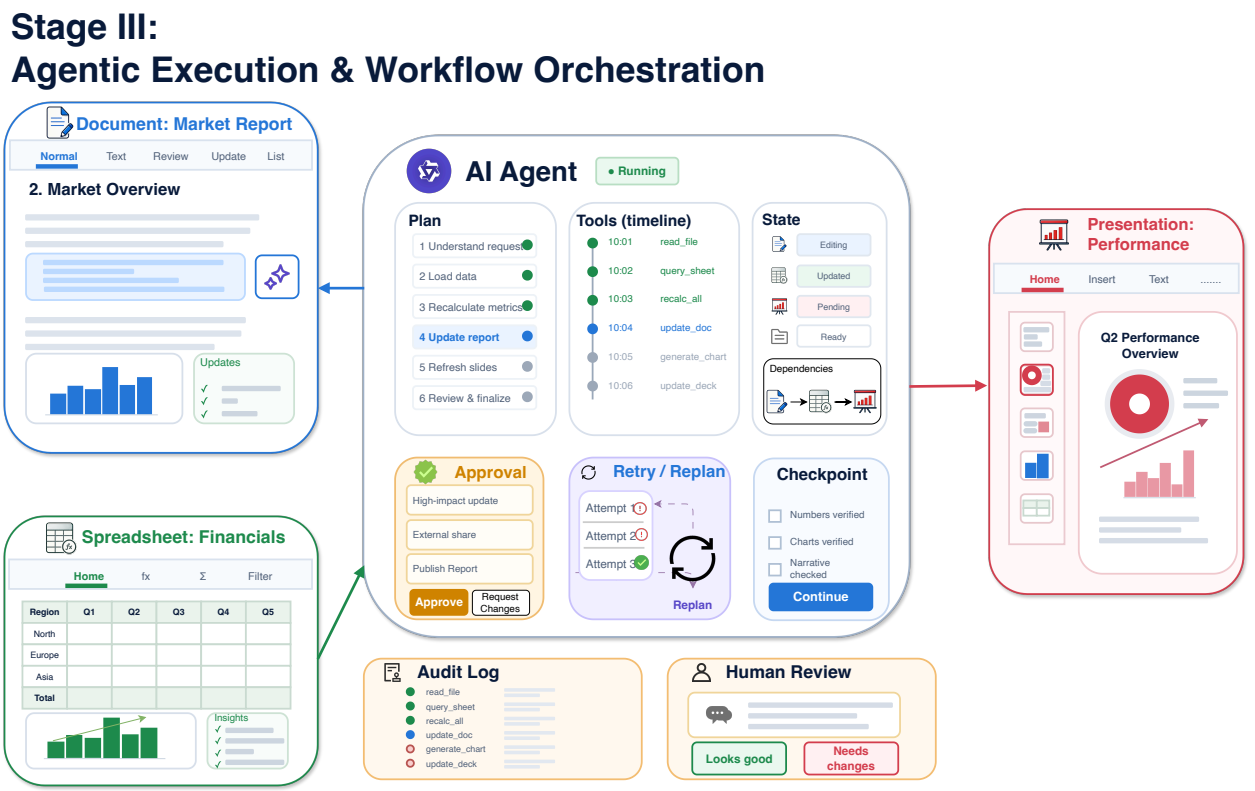


Figure 5 | **Stage III: AI as Office.** The user specifies a business goal; the agent decomposes it into subtasks, gathers or updates source artifacts, executes edits, checks intermediate results, and iterates until the deliverable package is ready for final human acceptance. The agent is accountable for the committed workspace: cross-file consistency, provenance, and final deliverable quality.

exclusive: most deployed systems blend them, defaulting to bounded edits for routine requests but escalating to autonomous delegation when the user specifies a broader goal.

A running example: the quarterly review. Tracing a single task across all three stages makes the shifting division of labor concrete. Consider a sales manager who must deliver a quarterly review package to leadership. The job follows a natural logic: *aggregate* the raw Q3 data into regional summaries and trend charts in a spreadsheet; *interpret* the numbers in a written analysis document that attributes every claim to its source; and *present* the key findings in a slide deck for the board meeting. The three artifacts are tightly coupled: every number in the slides must match the spreadsheet, and every claim in the document must cite the correct figure. What changes across stages is not the task but who owns the decisions, how errors are caught, and where human attention is spent.

In Stage I, the manager owns every step. She writes the aggregation formulas herself, choosing which function to use and which data range to include; a formula autocomplete may suggest arguments, but the analytical choices are hers. She builds trend charts and formats them to the corporate palette; the tool may recommend a chart type, but she decides whether it fits. She drafts the analysis document and runs a grammar checker, which catches surface errors in her prose but has no access to the spreadsheet and cannot judge whether her claims match the data. In the slide editor, a layout suggester proposes spacing options, but she selects the template, arranges the narrative, and manually copies key numbers from the spreadsheet. Each feature helps within its own application; none communicates with another. When she later discovers that a regional total is wrong, the cost of this isolation surfaces: she must fix the formula, re-export the chart, and re-paste it into the document and the slides one file at a time. If the document says “12% growth” while the spreadsheet shows 11.7%, only she will notice.

In Stage II, the manager still decomposes the task herself, but she describes each sub-task in natural language instead of executing it by hand: “Summarize Q3 sales by region, flag underperformers, create a trend

chart, and draft three slides for the quarterly review." The copilot selects the aggregation method, generates the chart, drafts the analysis with section headings, and proposes a three-slide deck. Her job shifts from building to reviewing each artifact against her own quality criteria: Is the aggregation logic correct? Does the document tone suit a board audience? Is the chart readable and the narrative arc coherent? When she spots that Asia-Pacific is missing from the trend chart, a single follow-up instruction is enough. She no longer needs to know which formula or chart type to use; she evaluates outputs, not methods. Cross-file consistency, however, remains her responsibility: the copilot produced all three artifacts from the same prompt, but the document quotes "12.3% growth" while the slide chart shows 11.8%, and only she catches the mismatch. The instruction must be specific enough for the copilot to act on, and quality assurance still rests with her.

In Stage III, the manager no longer decomposes the task at all. She specifies only a business outcome, one that would be too vague for a Stage II copilot to act on: "Prepare the board-ready quarterly review package." The agent infers the aggregate-interpret-present logic on its own, deciding which data to retrieve, which aggregations to compute, what document structure to use, and which template to apply. What makes the result more robust than Stage II is self-verification: after each major step the agent checks its own work, re-executing the formula chain, rendering the slides to catch layout problems, and comparing every number across all three files so that cross-file consistency is enforced before the manager sees the package. When it detects that European Q3 revenue is missing July data, it flags the gap and adds a footnote rather than silently presenting a wrong total. When a chart overflows its slide placeholder, it resizes and regenerates the slide without being asked. The manager receives a package that has already passed the quality checks she would have performed manually in Stage II. She reads the executive summary, and her only intervention is strategic: "Emphasize Asia-Pacific growth more prominently." The agent revises the document narrative, updates the chart emphasis in the slides, re-verifies consistency, and resubmits. Her role is no longer quality assurance; it is setting direction and accepting the final deliverable.

Across this progression, the technical complexity of the task does not shrink; it is absorbed by the agent's planning, execution, and verification loop. The user's required expertise shifts accordingly: from tool proficiency (Stage I, knowing which formula to write), through workflow supervision (Stage II, decomposing the task and checking each output), to goal articulation and strategic judgment (Stage III, specifying what the deliverable should achieve and steering its emphasis). Each stage raises the ceiling of what a single user can accomplish: a manager who could not write a pivot-table formula can still commission a board-ready package, because the technical work has moved behind the agent's interface. Early deployment data already shows this ceiling-raising effect: the boundary between software work and knowledge work is blurring as agents enable people to reach beyond their formal role (OpenAI, 2026b).

Delegation, however, introduces its own risk. The further the user stands from the technical details, the harder it becomes to catch errors the agent's own verification misses. In Stage I the manager would have noticed a wrong formula because she wrote it; in Stage II she might still catch a suspicious number during review; in Stage III she trusts a package that has already passed automated checks, and a subtle error in the underlying data or an over-confident self-verification may go undetected. The autonomy spectrum is therefore also a trust spectrum: higher delegation demands not only a more capable agent but more reliable verification, since the user's ability to serve as a fallback diminishes at exactly the pace the agent's responsibility grows. This is a well-documented pattern in automation research: as operators delegate more to automated systems, their situational awareness degrades and they become less able to detect or recover from the system's errors (Bainbridge, 1983; Parasuraman & Riley, 1997); recent evidence of silent document corruption under delegated LLM editing shows the pattern already reaches AI Office (Laban et al., 2026). The gap between delegation and oversight is not fixed: as foundation models become more capable and verification mechanisms mature, the class of errors that slip through will narrow. The structural lesson remains: verification needs to advance in lockstep with capability, because each new level of delegation raises user expectations while further reducing the user's inclination and ability to double-check. The goal is not to keep the user in the loop for every detail, but to make the agent's own quality assurance trustworthy enough that the user's shift from operator to judge is safe.

2.2. Definition and Scope

The iterative loop visible in Stage II and especially Stage III suggests a common formal structure across artifact types. As Figure 1 illustrates, the core mechanism is an iterative *generation and verification loop*: given a user goal and one or more source artifacts, the agent observes the current artifact state, plans and commits edits,

and submits the result to verification. If verification fails, the agent re-enters the loop with updated feedback; if it succeeds, the result is promoted to a *deliverable*, or to a *cross-file deliverable package* when multiple artifacts must remain mutually consistent. We now formalize this process.

At each step t, the agent observes the current artifact d_t under goal g and selects an action that induces three coupled state updates.

Observation. The agent perceives the artifact through a (possibly lossy) observation function:

$$o_t = \Omega(d_t). \tag{1}$$

Action selection. Conditioned on the goal, internal working state m_t, open task-specification state q_t, and the observation, the agent chooses an action:

$$a_t = \mathcal{A}(g,\ m_t,\ q_t,\ o_t). \tag{2}$$

State transitions. The action induces three coupled updates:

$$d_{t+1} = T_D(d_t,\ a_t), \tag{3}$$
$$m_{t+1} = T_M(m_t,\ a_t,\ o_{t+1}), \tag{4}$$
$$q_{t+1} = T_Q(q_t,\ g,\ d_{t+1}). \tag{5}$$

Equation equation 3 updates the *persistent artifact* d_t, which changes only when an action commits an edit. Equation equation 4 updates the *working state* m_t, which carries the agent's plan, scratchpad, and memory; it evolves at every step regardless of whether d_t changes. Equation equation 5 updates the *task-specification state* q_t, which tracks the open sub-goals of g and narrows as the agent makes progress.

What separates AI Office from adjacent settings is what changes during the loop and what is finally judged. Document QA systems produce text without updating d_t (Mathew et al., 2021; Tito et al., 2023; Bai et al., 2024b; Saad-Falcon et al., 2023). OS and web agents commit actions to generic interfaces but do not maintain the schema-rich internal state that office artifacts demand (Xie et al., 2024; Bonatti et al., 2024; Zhou et al., 2023; Deng et al., 2023). Single-step tool-use models are scored per invocation rather than on the cumulative artifact state d_T (Schick et al., 2023; Qin et al., 2024). In each case the missing element is the same: the persistent, structured file an office agent is expected to leave behind in a valid and usable form.

2.3. Office Artifacts as Agentic Environments

Although spreadsheets, slides, and documents appear heterogeneous, they share the agentic environment template of §2.2 and differ chiefly in their verification criterion $\mathcal{V}$ (Liu et al., 2024; Wang et al., 2024d). For each type, the artifact's internal structure determines what the agent can observe, what actions it can take, and what $\mathcal{V}$ must ultimately check.

Spreadsheets: executable verification. A spreadsheet is simultaneously a data store and a computation engine: cells hold formulas that reference other cells, ranges carry dependencies, and pivot tables aggregate data through defined transformations. The first challenge is perceiving this structure, because table boundaries, headers, merged cells, and cross-sheet references are rarely encoded explicitly in the file format, and real-world workbooks are large, sparse, and heterogeneous, mixing data regions, formula blocks, and free-text annotations on the same sheet (Dong et al., 2024; Zhang et al., 2024b). Once this latent structure is recovered, the available actions span several granularities: writing individual formulas from natural-language descriptions (Chen et al., 2021b; Zhao et al., 2024), executing atomic cell and range operations through application APIs (Li et al., 2023a; Payan et al., 2023), composing multi-step workflows that interleave computation and data transformation (Chen et al., 2025; Ma et al., 2024b), and scripting complex logic in VBA or Python (Chi et al., 2026). Because every edit produces a new executable state, $\mathcal{V}$ is the most directly checkable of the three artifact types: evaluation can re-execute the formula chain and compare output cells against expected values (Li et al., 2023a; Ma et al., 2024b). The check is less straightforward than it appears, however: a workbook may produce the correct final number while embedding its computation in hard-coded constants rather than live formulas, making the file brittle and unauditable (Chen et al., 2025; Ma et al., 2024b); conversely, structurally correct formulas may

silently propagate a single-cell error across an entire dependency chain. Verification must therefore assess both output correctness and implementation quality, a distinction that separates spreadsheet agents from the adjacent literature on table reasoning and text-to-SQL, which shares the challenge of tabular structure understanding (Yu et al., 2018; Li et al., 2023b; Wang et al., 2024e; Herzig et al., 2020; Wu et al., 2025) but treats the table as a static data source, not an editable, executable artifact.

Slides: aesthetic verification. Slides fuse semantics and visuals: a deck is not a sequence of statements but a spatial, temporal, and aesthetic composition in which content, layout, typography, color, and narrative flow must cohere (Fu et al., 2022; Sun et al., 2021; Zheng et al., 2025). The agent observes this composition as a structured object model of placeholders, text boxes, charts, and images arranged in a spatial hierarchy, where the rendered appearance depends on template inheritance, master-slide definitions, and absolute positioning (Zheng et al., 2025; Ge et al., 2025). The action space mirrors this hierarchy: at the element level, the agent creates, moves, resizes, and styles individual objects; at the slide level, it selects layouts and arranges visual compositions; at the deck level, it orchestrates narrative structure and cross-slide consistency (Jung et al., 2025; Zeng et al., 2026). End-to-end generation chains these levels into a multi-stage pipeline that runs from source analysis through template selection, content population, and layout adjustment (Fu et al., 2022; Sun et al., 2021; Zheng et al., 2025; Ge et al., 2025; Zhang et al., 2025b). $\mathcal{V}$ needs to operate at matching granularities: element-level checks (text overflow, image resolution), slide-level checks (layout balance, font consistency), and deck-level checks (narrative coherence, topic progression) (Zheng et al., 2025; Pan et al., 2026; Guo et al., 2024). Some of these dimensions can be made deterministic: AeSlides translates layout aesthetics such as aspect-ratio compliance, whitespace distribution, and element collision into verifiable rules (Pan et al., 2026). Many remain perceptual, however: a factually correct deck can still fail because the narrative arc breaks, fonts drift between slides, or the overall design lacks coherence. The gap between local and global evaluation is particularly telling: PPTC shows that a model handling each edit instruction correctly can still produce an incoherent deck, because per-slide scoring misses cross-slide consistency (Guo et al., 2024). The perceptual, multi-granular character of slides makes their verification intrinsically harder to automate than spreadsheets'.

Documents: provenance verification. Text documents demand long-range consistency across pages, sections, and modalities, making them the most structurally complex of the three artifact types. Observation requires bridging a gap between physical and logical structure: the file stores paragraphs, embedded tables, figures, headers, and footnotes in an order that may diverge substantially from the rendered narrative a reader follows (Xu et al., 2021; Huang et al., 2022; Wang et al., 2024b). Layout-aware understanding is therefore a prerequisite, learning joint representations of text position, visual features, and semantic content to close this gap (Xu et al., 2021; Huang et al., 2022), with recent generative models extending it to both comprehending and producing document content with spatial fidelity (Wang et al., 2024b; Hu et al., 2025a, 2024). Multi-page reasoning compounds the challenge: evidence relevant to a single question may be scattered across distant pages, cross-references must resolve, and the hierarchical structure must be tracked throughout (Mathew et al., 2021; Tito et al., 2023; Ma et al., 2024a; Saad-Falcon et al., 2023). The action space takes the form of diffable typed edits (insertions, deletions, reformattings, citation additions) that preserve revision history and structural constraints (Raheja et al., 2023; Schick et al., 2022). Unlike spreadsheet formulas or slide layouts, the central verification challenge for documents is provenance: office documents serve as records of analysis or decision, so every claim should trace back to evidence and every cited figure should match its underlying data. $\mathcal{V}$ therefore needs to check source attribution and citation validity alongside fluency, factuality, and structural formatting (Zou et al., 2025; Ma et al., 2024a). Document verification is the most encompassing of the three: it requires simultaneous attention to content accuracy, structural integrity, source grounding, and cross-section consistency, and its failure modes can span many pages and only surface during careful review.

2.4. From Application-Local Logic to Model-Harness Orchestration

The three stages also trace a shift in where the logic of work resides. Traditional office software asks users to translate goals into application-specific operations: choose the right formula, pick the right chart type, apply the right template. As models absorb more of this translation, the software layer that once encoded it thins out and the intelligence layer that replaces it thickens. Figure 6 contrasts the two architectures.

A useful historical parallel is electrification. Robert Solow observed that "you can see the computer age everywhere but in the productivity statistics" (Solow, 1987), and Erik Brynjolfsson named this puzzle the *productivity paradox*, arguing that information technology produces large gains only when organizations

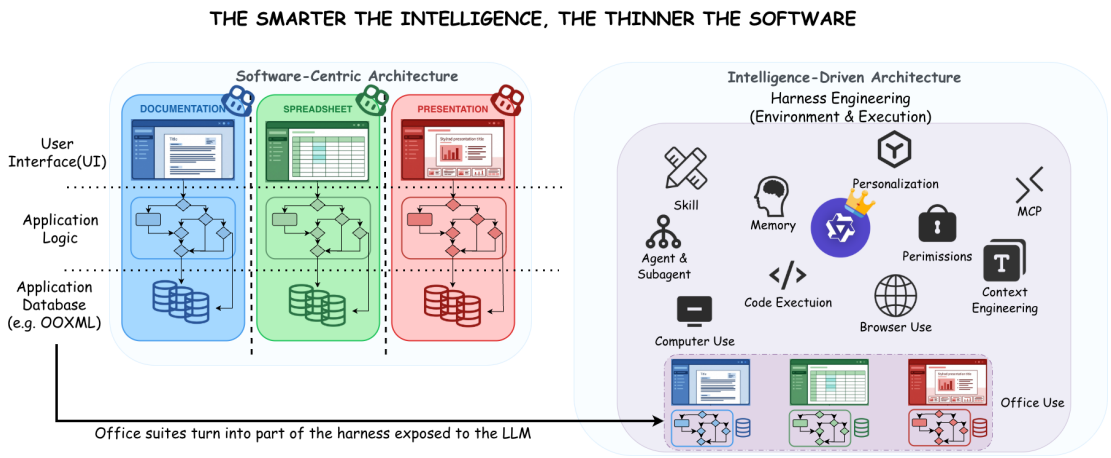


Figure 6 | **From application-local logic to model-harness orchestration.** Left: traditional office suites isolate work in product-specific UIs, each with its own application logic and data format. Right: a model-driven harness mediates user intent across the same artifact types through skills, memory, personalization, permissions, and callable capabilities, making the application layer a thinner execution surface.

redesign their processes, skills, management structures, and workflows around what the technology has made cheap (Brynjolfsson, 1993; OpenAI, 2026b). Electricity followed the same trajectory: large gains arrived only after factories stopped treating electric power as a substitute for steam engines and rebuilt their floorplans around distributed electric motors, a transformation that took decades. Previous generations of workplace software lowered the cost of *producing* intermediate artifacts without reducing the attention required to *consume* them: email made correspondence cheap, then multiplied correspondence; document editors made drafting cheap, then multiplied drafts and review cycles (OpenAI, 2026b). The result is a paradox of abundance: modern workers produce documents, messages, dashboards, models, and presentations faster than ever, yet spend a large share of their time looking for context, reconciling conflicting versions, waiting for responses, and moving information across systems (OpenAI, 2026b). A McKinsey Global Institute study quantifies the imbalance: the average knowledge worker spends roughly 28 percent of the workweek managing email and nearly 20 percent looking for internal information or tracking down colleagues who can help with specific tasks (Bughin et al., 2012). The bottleneck is no longer producing artifacts but finding, coordinating, verifying, and maintaining them. Large organizations historically met this bottleneck with human infrastructure: administrative support teams, cross-functional groups, layered review chains, and coordination overhead, all built around the heavy cost of producing and moving artifacts (OpenAI, 2026b).

AI Office, as formalized in this survey, addresses the same coordination challenge that this organizational infrastructure evolved to handle. The progression from Stage I to Stage III maps directly onto the three frictions identified in §2.1: the harness addresses *search friction* by retrieving relevant files, datasets, and precedents across systems on behalf of the user; it addresses *coordination friction* by carrying context across artifact types so the user no longer reformulates intent at each application boundary; and it addresses *verification friction* by executing formulas, rendering layouts, and checking cross-file consistency before the deliverable reaches a human reviewer. In the terms of this subsection, the intelligence layer absorbs work that the software layer never automated and that previously fell to the user or to organizational overhead.

Vertical unbundling. The left side of Figure 6 shows the traditional architecture. Each office application bundles three layers: a user interface of menus, dialogs, and canvases; an application logic layer that encodes workflows, formatting rules, and feature-specific heuristics; and a data layer that persists the artifact in a format such as OOXML. In Stage I, AI enters this stack as a narrow feature inside one layer of one application, such as a formula autocomplete or a grammar checker.

As models grow more capable, this bundling loosens. The application logic layer is the first to thin out: the heuristics that once decided how formulas are suggested, charts are chosen, layouts are arranged, and text is corrected are increasingly replaced by the model's own reasoning. What remains below is a durable substrate of data models, permissions, and execution endpoints; what rises above is a model-driven orchestration layer that interprets user intent and coordinates actions across that substrate (Anthropic, 2025d,e, 2024b). The

total amount of software does not shrink, but its role changes: applications evolve from self-contained work environments into providers of structured capabilities that the intelligence layer can call.

Horizontal softening. At the same time, the boundaries between applications loosen. In the traditional architecture, each application defines its own interface, data model, and workflow assumptions; the quarterly-review manager in §2.1 had to move context from spreadsheet to document to slides by hand, reformulating her intent at each transition. As the orchestration layer grows stronger, it carries context across these boundaries. In Stage II a copilot can coordinate actions across several artifact types within a single prompt; in Stage III the agent treats documents, spreadsheets, and slides as nodes in a continuous workspace, not isolated destinations.

The right side of Figure 6 shows the resulting architecture: a harness mediates between the user and the underlying artifact types through skills, memory, code execution, computer use, and protocol-level interoperability such as MCP and A2A (Anthropic, 2024b; Google Developers Blog, 2025). The user specifies outcomes at whatever level of abstraction suits the task, and the model decides which artifacts to create, which tools to invoke, and how to keep them consistent. Much of the friction that the quarterly-review manager experienced in Stage I (the manual re-export, re-paste, and cross-check across files) disappears, not because the files have merged but because the intelligence layer manages the connections between them.

The model-harness co-evolution cycle. How far will this shift go? The answer is shaped by a dynamic visible in recent office systems: models absorb what harnesses once provided, and the harness rebuilds at a higher level.

Consider slides. A year ago, ensuring that a generated deck had no text overflow, consistent fonts, and balanced whitespace required hand-crafted layout heuristics in the harness. Today, the GLM slide agent replaces those heuristics with a model trained via verifiable layout rewards, producing better aesthetic quality than the rules it displaced (Z.AI, 2026; Pan et al., 2026). The same pattern holds for spreadsheets: Spreadsheet-RL trains agents end-to-end with reinforcement learning on real workbooks, surpassing pipeline approaches that relied on manually engineered formula-selection logic (Chi et al., 2026). In both cases, capability that once lived in the harness migrated into the model through training.

The harness did not become unnecessary; it shifted upward. Once the model can generate formulas and lay out slides on its own, the harness's job changes: it manages which files to coordinate, whether numbers in the spreadsheet match claims in the document, whether the deck conforms to organizational templates, and whether the completed package should be surfaced to the user or sent back for revision (Microsoft Learn, 2026a). This higher-level role, governing multi-artifact consistency, organizational policy, and trust rather than single-artifact generation, is what separates a Stage III runtime from a Stage I feature.

The result is an iterative cycle: harness improvements produce better task trajectories; those trajectories become training signal through imitation or reinforcement learning; the next-generation model absorbs the improvement; and the harness is rebuilt on a higher foundation. A further twist is now visible: the model can generate harness code itself. Dynamic Workflows in Claude Code allow the model to produce a deterministic orchestration script that schedules sub-agents, encodes adversarial verification, and persists intermediate state, effectively authoring a task-specific harness on the fly rather than executing within a pre-designed one (Anthropic, 2026b). This blurs the boundary between model and harness: the model both operates within and dynamically creates the orchestration layer, and the runtime's role narrows to providing execution guarantees (concurrency, budgets, permissions, resumability) rather than encoding the plan. The systems in this survey span the full spectrum, from Stage I systems that are mostly software with thin model involvement to Stage III systems that are mostly intelligence with thin runtime scaffolding. As the cycle continues, the harness's role will shift from compensating for model weakness toward governing model autonomy and securing the trust guarantees discussed in §2.1.

2.5. Where We Stand and How the Survey Proceeds

We sit in the transition between Stage II and Stage III. As §2.1 argued, recent foundation models have reached a practical capability range for office delegation, and recent advances in reasoning, multimodal perception, and computer use make grounded, multi-step action over real files increasingly realistic (Anthropic, 2025a; OpenAI, 2025f,d; Lai et al., 2025). Commercial copilots already orchestrate drafts across documents, spreadsheets, and slides (Microsoft, 2023; Google Workspace, 2025a), and research prototypes demonstrate end-to-end slide-deck generation (Zheng et al., 2025; Ge et al., 2025) and multi-step spreadsheet manipulation (Chi et al., 2026).

The full Stage III vision (an agent accountable for a complete, audit-ready deliverable package) remains open: individual artifacts can be impressive while the numbers, claims, and formatting across them still disagree.

The generation and verification loop formalized in §2.2 (Figure 1) provides the structure for examining what is needed to close this gap. Whether the loop produces trustworthy deliverables depends on three interdependent dimensions, each addressed by a subsequent section:

- **Operation** (§3): how does the agent act? This section traces the shift from invoking single features to orchestrating code, APIs, and GUI through a runtime harness.
- **Evaluation** (§4): how do we know the result is good enough? This section traces the shift from local checks on individual outputs to certification of entire deliverable packages.
- **Training** (§5): how does the agent improve? This section traces the shift from imitating curated demonstrations to optimizing artifact-grounded rewards.

The three dimensions are tightly coupled. Operation surfaces constrain what the agent can observe and do; evaluation criteria define what counts as success; training signals determine whether the agent improves toward that standard. Advancing one without the others produces familiar failure modes: an agent that can edit files but cannot verify its own work, an evaluator with no path back into training, or a well-trained model with no harness to govern it at deployment. We conclude (§7) by identifying the open problems across all three dimensions.

3. AI Office Operation: From Features to Orchestration

How do agents translate user intent into edits on office artifacts? The answer has shifted from invoking a single feature inside one application to orchestrating code, APIs, and GUI through a runtime harness that manages entire workspaces.

An *operation paradigm* defines how the policy $\mathcal{A}$ selects actions a_t and how the artifact transition T_D commits them to d_t. Before the agent can act, however, it needs to perceive: the observation function $\Omega(d_t)$ determines what the agent sees, and its fidelity upper-bounds what any action can achieve. We therefore organize the discussion around observation (§3.1), the three action surfaces (§3.2), per-artifact pipelines (§3.3), cross-surface routing (§3.4), and harness engineering (§3.5). Appendix Table 6 expands the citations in this section into capability-level descriptions.

3.1. Observing Office Artifacts

Action quality cannot exceed observation quality. If the agent cannot correctly read the file's structure—distinguishing header rows from data, resolving template inheritance, or tracking cross-references—every downstream operation inherits that confusion. The observation function $\Omega(d_t)$ is therefore the first bottleneck in the operation loop.

Structured formats make programmatic parsing the default. The dominant editable formats (DOCX, PPTX, XLSX) are all OOXML packages: ZIP archives of well-schemed XML parts. This is the key enabler—any agent with code-execution access can unzip the file and recover content, styling, and relationships directly, without relying on rendered screenshots. Mature libraries wrap this access in Python (`openpyxl`, `python-pptx`, `python-docx` (Li et al., 2023a; Chen et al., 2025; Zheng et al., 2025; Ge et al., 2025)), JavaScript (`SheetJS`, `PptxGenJS`, `docx`), and C#/.NET (`Open XML SDK`, `ClosedXML`, `EPPlus`). When these libraries introduce lossy round-tripping, agents fall back further to direct XML manipulation—unpack, edit elements as text, repack (Anthropic, 2025e; MiniMax, 2026). The common thread: the observation path for editable artifacts is overwhelmingly text-based and programmatic (Anthropic, 2025b; OpenAI, 2025c).

Three boundaries where parsing alone fails. (1) *Non-editable formats.* PDFs encode rendered pages, not editable object models. Pipelines like MinerU (Wang et al., 2024a) and Docling (Livathinos et al., 2025) reverse-engineer structure, but the conversion remains lossy. Layout-aware models (LayoutLMv3 (Huang et al., 2022), DocLLM (Wang et al., 2024b)) and general multimodal models (Google DeepMind, 2025; OpenAI et al., 2023; Anthropic, 2025a; Bai et al., 2025) provide learned perception as a complement. (2) *Scale.* Large spreadsheets exceed the context window even after extraction. Compression (e.g., SheetCompressor

at 25× (Dong et al., 2024)) or multi-format routing to sub-agents (Ren et al., 2026) is required. (3) *Visual correctness.* A file can be structurally valid in XML yet visually broken—overflowing placeholders, clashing colors, wrong fonts. Only rendering to pixels reveals these problems (Zheng et al., 2026; Pan et al., 2026).

These three boundaries define a layered observation stack: programmatic parsing is the primary path for editable artifacts; specialized pipelines handle non-editable formats; and multimodal vision serves as the verification layer that catches what structure alone cannot express.

3.2. Three Operation Surfaces

Once the agent has observed the artifact, it must act on it. The action takes one of three forms—three fundamentally different *interaction modes* between agent and office artifact, each with a distinct tradeoff between freedom, safety, and coverage.

Code: direct file manipulation. The agent has a terminal or sandbox, writes arbitrary programs, and executes them against the file. A spreadsheet agent might write Python with `openpyxl` to restructure a workbook; a slide agent might call `PptxGenJS` to generate a deck; a document agent might use the Open XML SDK to patch styling, or simply edit raw XML in a shell script. The defining characteristic: the agent *authors the implementation itself*, unconstrained by any pre-defined operation set (Chen et al., 2021a; Gao et al., 2023; Wang et al., 2024c; Jimenez et al., 2024b).

In the office domain this manifests as: writing formulas, VBA, or scripts for spreadsheets (Li et al., 2023a; Chen et al., 2025; Zhao et al., 2024; Chen et al., 2021b); scripting object-model operations for slides (Zheng et al., 2025; Ge et al., 2025; Fu et al., 2022); composing SQL or code-interleaved reasoning for tables (Yu et al., 2018; Wang et al., 2024e; Li et al., 2023b); and applying typed diffs for documents (Raheja et al., 2023; Schick et al., 2022). The tradeoff: maximum precision and composability, but the agent must know *how*—which library, which XML element, which edge cases to avoid—and errors are silent until something downstream breaks.

Protocol: invoking typed services. The agent does not write the implementation; it *discovers and invokes* pre-defined operations through a standardized interface. Each call is a structured a_t whose schema the platform enforces—invalid actions are rejected before execution, and every invocation carries authentication, permissions, and an audit trail.

Two protocol layers have converged to make this practical for office work. *Tool protocols* let an agent dynamically discover what operations are available without per-integration wiring. MCP (Model Context Protocol) standardizes this: a single office agent can enumerate tools from a spreadsheet server, a document server, and a calendar server through the same interface, then call them with schema-validated arguments (Anthropic, 2024b; Li et al., 2026c). Production function-calling APIs implement the same pattern at the provider level (Schick et al., 2023; Qin et al., 2024; OpenAI, 2026a). *Agent-to-agent protocols* extend this one level further: A2A (Agent-to-Agent) lets specialized office agents publish their own capabilities as callable services—a formatting agent can expose "reformat this table to corporate style" as a typed operation that other agents invoke, turning multi-agent collaboration into protocol-mediated composition rather than free-form conversation (Google Developers Blog, 2025).

Within these protocols, office-specific services fall into two families. *Connector services* provide governed access to organizational data—Microsoft Graph lets Copilot search email, calendar, and files within administrative boundaries (Microsoft, 2023); Google Workspace APIs let Gemini read and write across Docs, Sheets, and Slides (Google Workspace, 2025a). *Authoring services* expose higher-level verbs ("summarize this page," "generate a chart from this range") that encapsulate multi-step logic behind a single call (Notion Labs, 2023; Z.AI, 2026). The tradeoff: protocol-based actions are discoverable, governed, and safe (the agent can only do what someone has defined as a tool), but they sacrifice the open-ended flexibility of code.

GUI: visual interaction. The agent operates office software through the same rendered interface a person uses—screenshots or accessibility trees as observation, mouse clicks and keystrokes as actions (Xie et al., 2024; Drouin et al., 2024; Cheng et al., 2024; Qin et al., 2025; Zhang et al., 2024a; Anthropic, 2024a; Hu et al., 2025b; Nguyen et al., 2025).

GUI is the fallback when neither code nor protocol access covers the required operation: adjusting a slide master template through a multi-step dialog, configuring conditional formatting rules, or verifying that a

chart renders correctly with the chosen color palette. It can reach *any* feature the application exposes to a human, but at a cost: actions are slow (each step waits for a render), error-prone (small UI elements are hard to target), and hard to reverse (undoing requires traversing the application's own undo stack) (Hong et al., 2024; Lin et al., 2025; Lu et al., 2024). Recent advances narrow these gaps by unifying GUI with programmatic access—UFO2's desktop AgentOS (Zhang et al., 2025a), OmegaUse's cross-environment training (Zhang et al., 2026), and production computer-use systems (Anthropic, 2024a; OpenAI, 2025d)—so that agents can attempt the GUI path but fall back to code when precision matters.

Surfaces compose in practice. Real office tasks rarely stay on one surface. An agent might discover available tools via MCP (protocol), write a Python script to restructure a workbook (code), then open the application and visually verify the chart renders correctly (GUI). The routing decision—which surface to use for each step—depends on what's available, what precision is needed, and whether governance constraints require an auditable protocol path. §3.4 details how deployed systems make this decision.

3.3. Per-Artifact Operation Pipelines

The three surfaces combine into end-to-end pipelines that differ by artifact type. Just as §2.3 showed that spreadsheets, slides, and documents share an agentic environment template but differ in verification, we now show that they share a plan-execute-validate loop but differ in the structure and granularity of operations.

Spreadsheet operation pipeline. Spreadsheet operations span a wide granularity range: writing a single formula from a natural-language description (Chen et al., 2021b; Zhao et al., 2024), executing atomic cell and range operations via application APIs (Li et al., 2023a; Payan et al., 2023), composing multi-step workflows that interleave computation, data transformation, and chart creation (Chen et al., 2025; Ma et al., 2024b), and scripting complex logic in VBA or Python (Chi et al., 2026).

An end-to-end spreadsheet pipeline typically runs four stages: (1) goal parsing and structure recovery, identifying relevant tables, headers, and data ranges; (2) operation planning, decomposing the goal into a sequence of formula insertions, range operations, or scripts; (3) execution, committing each planned action to the workbook; and (4) validation, re-executing formulas or inspecting output cells to verify correctness. SheetCopilot formalizes this as a sequence of 283 atomic actions drawn from spreadsheet software functionality (Li et al., 2023a); SheetAgent adds a Planner-Informer-Retriever architecture that decomposes complex tasks and retrieves relevant context before acting (Chen et al., 2025); SheetMind introduces a three-agent system (Manager, Action, Reflection) in which the Reflection agent inspects execution results and triggers replanning on failure (Zhu et al., 2025); and Spreadsheet-RL reports gains from training spreadsheet agents end-to-end with reinforcement learning on realistic workbooks rather than relying on manually engineered pipelines (Chi et al., 2026). Recent benchmarks quantify the gap between atomic operations and realistic workflows. Finch embeds spreadsheet tasks in 172 multi-step finance workflows: even GPT-5.1 Pro, spending 16.8 minutes per workflow, passes only 38.4% (Dong et al., 2026). MBABench evaluates agents building complete spreadsheets from scratch (Yen et al., 2026), and BlueFin tests financial reasoning with 131 tasks on which leading models score below 50% (Kundurthy et al., 2026).

Slide operation pipeline. Slide generation chains multiple operation levels: source analysis, template or reference selection, outline generation, content population, layout adjustment, and visual refinement, with cross-slide consistency maintained throughout (Fu et al., 2022; Sun et al., 2021; Zheng et al., 2025; Ge et al., 2025).

PPTAgent introduces a reference-based pipeline that extracts structural patterns from an existing deck (layouts, element types, visual styles), generates a content outline from a source document, populates slides through object-model operations, and adjusts layout to prevent overflow (Zheng et al., 2025). AutoPresent takes a complementary approach: generating slides from free-form instructions without reference decks, using an 8B model that emits structured JSON specifications per element (Ge et al., 2025). SlideTailor personalizes the pipeline for academic papers, adapting content selection and visual style to individual presentation preferences (Zeng et al., 2026). PosterGen extends the multi-agent pattern to research posters, where layout, content, and aesthetic agents collaborate within a shared canvas (Zhang et al., 2025b). DeepPresenter introduces environment-grounded reflection: rather than generating all slides in a single pass, the agent renders intermediate slides, inspects them visually, and revises iteratively, turning slide generation from open-loop production into a closed-loop refinement process (Zheng et al., 2026). This aligns slide operation with the

iterative generation-and-verification loop formalized in §2.2: the agent's working state m_t now carries visual feedback from rendered previews, not just textual plans.

The central challenge in slide operations is maintaining coherence across the deck: narrative flow, visual consistency (fonts, colors, spacing), and content-layout co-optimization all require deck-level reasoning that per-slide generation misses (Guo et al., 2024; Pan et al., 2026). PPTArena formalizes this setting as a benchmark for agentic PowerPoint editing with structure-aware evaluation (Ofengenden et al., 2025).

Document operation pipeline. Document operations are distinguished by their length, provenance requirements, and structural complexity. An end-to-end document pipeline proceeds through: (1) source gathering and evidence collection; (2) outline planning with hierarchical section structure; (3) section-by-section drafting with citation integration; (4) cross-reference resolution and internal consistency checking; and (5) formatting and revision (Shao et al., 2024a; Schick et al., 2022).

STORM illustrates a multi-agent approach to source-grounded writing: a question-driven information-seeking agent discovers relevant sources, a writing agent drafts sections with inline citations, and an editing agent revises for coherence and factual accuracy (Shao et al., 2024a). PEER decomposes editing into a plan-edit-explain-rewrite loop in which each revision is accompanied by an explicit rationale that supports auditability (Schick et al., 2022). Long-document operations also introduce challenges that shorter tasks do not: maintaining consistent terminology across 10+ pages, managing figure and table numbering as content shifts, keeping bibliography references valid as sections are added or removed, and ensuring cross-section claims remain consistent after local edits. Unlike spreadsheet formulas, which can be re-executed, and slide layouts, which can be rendered, document consistency depends on provenance chains that span the entire artifact and demand semantic rather than syntactic verification. Documents are therefore the artifact type most dependent on the harness's working memory m_t to track what has been claimed, cited, and cross-referenced.

3.4. Cross-Surface Routing and Composition

Real office tasks rarely confine themselves to a single surface. A slide generation task might use code (python-pptx) to create and populate slides, GUI to verify the rendered result, and an API to retrieve source data from a shared drive. A spreadsheet task might use an API to pull data from an enterprise system, code to write formulas and scripts, and GUI to configure conditional formatting through a dialog that lacks API exposure. Surface selection and composition is therefore a design decision with direct consequences for both capability and reliability.

When to use which surface. The choice depends on four factors: (1) *API availability*, whether the required operation is exposed programmatically; (2) *operation complexity*, where simple lookups favor APIs, multi-step logic favors code, and visual verification favors GUI; (3) *reversibility*, since code and API calls produce inspectable, replayable traces while GUI actions must traverse the application's undo history; and (4) *verification requirements*, where operations whose correctness only shows in the rendered view (layout, color, overflow) require GUI-level observation even if the edit itself was made through code. A common strategy is *code-first with GUI verification*: the agent performs edits through the programmable substrate for precision and auditability, then switches to GUI-level observation to verify the rendered result.

Hybrid and unified action spaces. Recent systems formalize this composition. UltraCUA unifies primitive GUI operations (click, type, scroll) with high-level tool execution (API calls, code snippets) in a single action space, letting the agent select the most appropriate surface per step (Yang et al., 2025). CoAct introduces a multi-agent architecture in which an orchestrator delegates each subtask to either a GUI Operator or a Programmer agent, achieving stronger performance than either surface alone (Song et al., 2025). AutoRPA goes further: it observes a GUI agent solving a task through pixel-level interactions, then *distills* the solution into RPA code that replays the same workflow programmatically, combining the generality of GUI with the efficiency and reliability of code (Chen et al., 2026). UFO2 provides a desktop AgentOS layer that exposes both native application APIs and GUI interaction through a unified interface, letting the agent mix surfaces within a single task without explicit routing decisions (Zhang et al., 2025a).

Cross-artifact surface composition. When a task spans multiple artifact types, for example a quarterly review that bundles a spreadsheet, a document, and a slide deck, the agent needs to route across surfaces and carry state between artifacts. The spreadsheet analysis might use Python code to compute aggregations; the

document draft might use an authoring API to insert formatted sections; the slide deck might use python-pptx for layout and GUI rendering for visual verification; and an email might use a connector API to send the finished package. At each artifact transition, the agent transfers relevant context (computed values, chart images, section summaries) from the working state m_t into the new artifact's operation surface. This cross-artifact state transfer is where most current systems are weakest. Each surface maintains its own context; ensuring that a number computed in the spreadsheet matches the number cited in the document and displayed in the slide requires explicit cross-file checks that few general harnesses provide natively (Wang et al., 2024d, 2025).

3.5. Harness Engineering

The harness as the runtime boundary. Once office work spans several artifacts and operation surfaces, the policy $\mathcal{A}$ can no longer be understood as a single model call. The operational unit becomes a *harness*: a persistent runtime that maintains workspace state, loads artifact-specific procedures, routes actions to code, APIs, or GUI, and validates each committed artifact state before continuing. Where Stage I features embed narrow intelligence inside one application, the harness lifts the locus of control above applications, coordinating the loop over g, m_t, q_t, o_t, and d_t introduced in §2.2. It is therefore the engineering layer where observation, action selection, verification, and recovery compose into a reliable office workflow.

The harness + skill pattern. The dominant pattern pairs a general runtime with loadable *Skills*: scoped folders of instructions, scripts, and assets activated on demand (Anthropic, 2025d). General runtimes such as Claude Code and Codex illustrate this separation between orchestration and executable procedures, while vendor skill libraries show how office-specific routines can be packaged as reusable assets instead of embedding them only in model weights (Anthropic, 2025b; OpenAI, 2025c; Anthropic, 2025e; MiniMax, 2026; Cursor, 2025; OpenCode Contributors, 2025). Portability is therefore an architectural direction, not a solved compatibility guarantee: a document, spreadsheet, or slide skill still depends on the runtime's available tools, permissions, and validation hooks.

Concretely, a harness provides capabilities that no single model call supplies: *planning*, which decomposes g into sub-goals tracked in q_t; *tool dispatch*, which routes each a_t to the appropriate surface (code execution, API call, or GUI interaction); *workspace memory*, which maintains m_t across turns so the agent can refer to prior observations, intermediate results, and its own plan; *validation*, which checks d_t after each edit and triggers replanning when $\mathcal{V}$ signals a failure; and *recovery*, which backtracks or retries when an action produces an unexpected state. A skill, in turn, packages artifact-specific knowledge: a PPTX skill might bundle python-pptx instructions, layout templates, and validation scripts that check for text overflow; an XLSX skill might bundle formula libraries and data-cleaning pipelines. Office capability accumulates as a portable layer: the harness manages m_t and q_t, while each skill localizes the artifact-specific tools that implement T_D, so new deliverables compose existing skills without rebuilding the runtime.

From sequential execution to parallel orchestration. As tasks grow from single edits into deliverable packages, harnesses must move from sequential execution to parallel orchestration. Several subgoals are often independently executable (inspecting a workbook, drafting a report section, rendering slide previews, collecting source documents), yet their results must later reconcile into one accepted package. This imposes two requirements on the runtime: it must maintain independent working states m_t for concurrent streams, and it must prevent cross-stream interference such as two workers editing the same file or propagating incompatible numbers into different artifacts. Parallelism in office work is therefore not just a latency optimization but a coordination problem over artifacts whose outputs feed one another.

The harness capability stack. Beyond the basic planning-dispatch-validate loop, effective office operation requires several harness capabilities that are still maturing.

Error recovery and replanning. Office tasks fail in artifact-specific ways: formulas produce `#REF!` errors when ranges shift, slide elements overflow their placeholders when content is longer than expected, document cross-references break when sections are reordered, and API calls fail when permissions are insufficient. A capable harness detects these failures, diagnose their root cause, and select a recovery strategy: retry with modified parameters, revert to a checkpoint and replan, or escalate to the user. Recent work on self-healing agent orchestrators formalizes this as a mapping from failure signals to recovery actions under explicit retry budgets (Babu & Agrawal, 2026), and DeepPresenter demonstrates environment-grounded replanning for slides, where the agent inspects rendered output, identifies layout defects, and revises iteratively (Zheng et al., 2026).

Skill evolution and curation. As agents accumulate experience, their skill libraries should grow and improve. SkillOS trains a skill curator via reinforcement learning, pairing a frozen agent executor with a trainable module that decides which skills to load, compose, and retire (Ouyang et al., 2026); Trace2Skill distills successful trajectory segments into reusable skills that transfer across model scales and out-of-distribution settings (Ni et al., 2026); SkillsBench evaluates whether models can discover, select, and apply skills from a library (Li et al., 2026a). For office work, skill evolution lets a harness that successfully generates a quarterly report extract the pipeline as a reusable skill for future quarters.

Multi-artifact workspace management. When the deliverable is a package rather than a single file, the harness must track dependencies across artifacts: a chart in the slide deck depends on data in the spreadsheet; a claim in the document depends on a computation in the workbook; an email summary depends on the conclusions in the report. This cross-file dependency graph determines the order in which operations must be executed and the consistency checks that must be performed after each edit. CORPGEN formalizes this as multi-horizon task environments with 45+ interleaved tasks spanning 500–1500+ steps, where coherent cross-task execution is the unit of evaluation (Jaye et al., 2026). Current general harnesses manage individual file states but do not maintain an explicit cross-file dependency graph; building this capability is a prerequisite for reliable package-level deliverables.

Toward an office-specialized harness. The harness–skill separation stays load-bearing even as models improve: post-training absorbs reasoning into the base model (Team et al., 2026b,a; Lai et al., 2025), but governance and package-level consistency still require a runtime above it (Anthropic, 2025b; OpenAI, 2025a; Wang et al., 2024d, 2025).

As single-artifact work becomes cheaper, the focus shifts upward: the interesting request is no longer "make a chart" but "turn this quarter's numbers into a board update," a deliverable spanning workbook, deck, document, and email (Microsoft 365 Blog, 2025b). Current general runtimes share planning, dispatch, and validation loops, but they rarely provide explicit cross-file consistency checks, artifact-aware rendering verification, or organizational-policy enforcement. They operate as powerful general tools, not as purpose-built office runtimes. An office-specialized harness would instead operate on rendered outputs, not text alone (Chen et al., 2025; Zheng et al., 2025; Pan et al., 2026): opening the workbook to verify that formulas recompute, rendering the deck to check for layout defects, tracing document claims to their sources, and comparing cross-file references for consistency before surfacing the deliverable. Building such a harness is among the open problems we identify in §7.

3.6. Takeaways

Office AI operation has converged on a shared architecture—observe the artifact, choose an action surface, execute, then verify—but the details diverge sharply by artifact type and the stack is still incomplete.

- **How well you can *see* the file caps everything else.** Programmatic parsing (OOXML, DOM) remains the primary observation channel; vision is a fallback verifier, not the main input (§3.1).
- **Code, APIs, and GUI are complementary, not competing.** The strongest systems route each step to the surface that offers the best precision–reversibility trade-off for that step (§3.2).
- **All pipelines plan–execute–validate, but "validate" means different things.** Spreadsheets check formula chains; slides check rendered layout; documents check provenance (§3.3).
- **The harness pattern works but isn't finished.** Artifact-aware validators, cross-file dependency tracking, and org-specific adaptation slots are still missing from general-purpose harnesses (§3.5).

4. AI Office Evaluation: From Local Checks to Certification

Given that agents can now edit artifacts through code, APIs, and GUI, how do we know the result is good enough? As agents attempt more ambitious deliverables spanning multiple surfaces, longer horizons, and cross-artifact packages, the gap between what can be verified automatically and what requires human judgment widens rapidly. Evaluation is the dimension whose difficulty grows fastest, moving from local checks a user can eyeball to certification of entire deliverable packages. Software-engineering deployments offer a useful analogue for what happens once generation becomes cheap. At Meta, agentic AI drove substantial growth

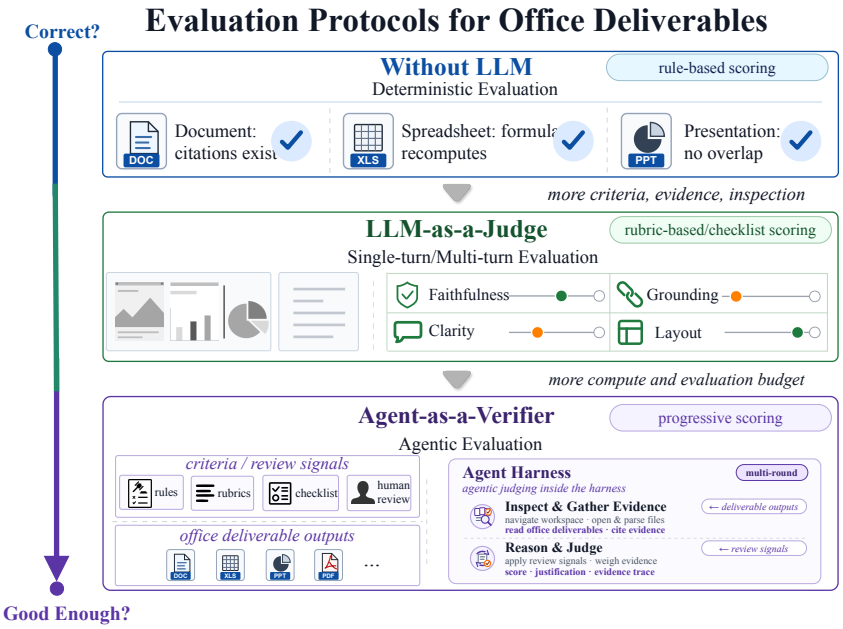


Figure 7 | **Evolution of evaluation protocols for office artifacts.** The progression from rule-based scoring to Agent-as-a-Verifier increasingly aligns with human acceptance.

in code output, but the resulting reviewer-bandwidth pressure motivated RADAR, a risk-aware automated review pipeline evaluated on more than 535K diffs (Adams et al., 2026). A longitudinal study of professional developers reports a parallel shift from code creation toward supervisory engineering: most participants spent less time writing code, while verification, correction, flow interruption, and cognitive load became more salient (Vella & Blincoe, 2026). The same pattern will likely appear in AI Office: as agents make documents, spreadsheets, and decks cheaper to produce, the scarce resource becomes certifying whether the committed deliverable is correct, editable, and acceptable. The knowledge worker's role correspondingly moves from manually producing each artifact to *supervisory office work*: checking numbers, tracing claims, reconciling inconsistencies, and deciding whether the package can be accepted.

Evaluation defines what AI Office systems are trained to become. The deliverable-centric criterion established in §1 and formalized in §2 requires benchmarks to reward the files and workspace state left behind, not the plausibility of the reply. Recent benchmarks reflect this shift through artifact-structure checks on spreadsheets and slides and long-horizon workflows in OfficeBench and OdysseyBench (Ma et al., 2024b; Zheng et al., 2025; Ge et al., 2025; Wang et al., 2024d, 2025). The common evaluation unit now looks like real work: initial artifact d_0, user goal g, tool trajectory $(a_0, \ldots, a_{T-1})$, rendered output, and final artifact state d_T. Appendix Table 7 expands the citations in this section into role-level descriptions, clarifying which cited works contribute benchmarks, evaluation protocols, or bias analyses.

Benchmarks should also be read against demand. Real office requests are typically concrete and delivery-oriented: reformat this deck, follow this template, convert this table, export this package, or reconcile these files. These tasks are often treated as routine, but they decide whether an artifact can be used in a meeting, forwarded to a client, or accepted by an organization. Current evaluations cover the easiest-to-formalize parts (formula correctness, table QA, source-grounded extraction, single-application task completion) more thoroughly than formatting, template adherence, delivery and export behavior, cross-document synthesis, and aesthetic fit. The gap matters because an office benchmark that overweights mechanically checkable tasks overestimates deployability precisely where user demand depends on subjective review, local convention, and final-package acceptance.

4.1. Modeling Office Tasks as Workspace Transitions

The basic evaluation unit should be a workspace transition. Building on the iterative revision of §2.2, an evaluation instance must specify the initial artifacts d_0, the user goal g, the available operations, and the expected

properties of the final d_T; the assistant's reply is secondary to whether d_T is correct. This is where AI Office parts ways with generic computer-use evaluation (Xie et al., 2024; Drouin et al., 2024), which scores realistic interface operation but not the in-software validity of what the agent leaves behind. A workspace-transition view also imposes structure on trajectories: each $d_{t+1} = T_D(d_t, a_t)$ must be attributable to the action that caused it, intermediate states must be inspectable, and d_T must be reproducible from d_0 and the action sequence.

A per-artifact look at what a workspace-transition instance contains. For a spreadsheet task, d_0 might be a workbook with raw sales data, g is "create a quarterly pivot table with year-over-year growth rates," and evaluation checks that the resulting formulas are live instead of hard-coded values, that column headers match the specification, and that the pivot recomputes correctly when an input cell changes (Ma et al., 2024b; Li et al., 2023a). For slides, d_0 could be a set of research notes, g is "create a 10-slide conference presentation," and $\mathcal{V}$ checks content fidelity to the source, layout quality (no overflow, consistent fonts), and narrative flow across slides (Zheng et al., 2025; Ge et al., 2025). For documents, d_0 is a collection of source files, g is "draft a summary report with citations," and $\mathcal{V}$ must verify factual accuracy against the sources, check that every cited claim is traceable, and ensure structural formatting compliance (Zou et al., 2025; Ma et al., 2024a). What unifies these instances is that the evaluation target is the committed file, not the agent's natural-language explanation of what it did.

Most benchmarks still evaluate actions, not artifact states. Despite the workspace-transition ideal, many benchmarks score the agent's action sequence (e.g., "did the agent click the right menu item?") or check a single output cell instead of validating the full artifact state d_T. OSWorld and WindowsAgentArena measure whether the agent completed the correct sequence of GUI interactions but do not open the resulting file to verify its internal structure (Xie et al., 2024; Bonatti et al., 2024). SpreadsheetBench moves closer by checking output cells against expected values (Ma et al., 2024b), but it does not verify whether intermediate formulas are reusable or whether the workbook structure supports future edits. Its successor SpreadsheetBench 2 pushes toward exactly this workspace-transition ideal: its 321 tasks target end-to-end business workflows such as financial modeling, debugging, and visualization over multi-sheet workbooks, scoring an answer correct only when all required cells are modified and all unchanged cells are left intact, and judging generated charts against rubric assertions, not a single output value (Ma et al., 2026). OfficeBench, OdysseyBench, and TheAgentCompany go further by checking cross-application state and embedding tasks in enterprise contexts (Wang et al., 2024d, 2025; Xu et al., 2024), but even these benchmarks test only a subset of the quality dimensions a deliverable must satisfy (§4.2). The progression from action-level scoring to full workspace-transition evaluation remains incomplete, and closing this gap is a prerequisite for training signals that optimize deliverable quality instead of surface behavior.

Artifact fidelity is a prerequisite, not an embellishment. The verification criteria $\mathcal{V}$ of §2.3 (executable for spreadsheets, aesthetic for slides, provenance-bearing for documents) are not optional at scoring time. Stripping them rewards outputs that look acceptable as text but fail in the file: a deck whose claims a reader cannot trace, or a workbook whose "answer cell" no longer recomputes once an input changes. Evaluation that ignores fidelity selects for plausible chat behavior; evaluation that enforces $\mathcal{V}(d_T, g)$ selects for behavior that survives contact with the artifact's runtime.

4.2. Judging Deliverables, Not Isolated Outputs

Correctness is necessary but insufficient. Artifacts that pass local checks often fail as deliverables. SpreadsheetBench verifies output cells against expected values, but as noted in §2.3, a workbook whose output is correct may still embed its computation in pasted constants instead of live formulas, passing the benchmark while failing any downstream audit (Ma et al., 2024b; Chen et al., 2025). Slide benchmarks can score individual slides for content fidelity, but PPTC shows that a model handling each edit instruction correctly can still produce an incoherent deck when those edits accumulate, because single-slide evaluation misses deck-level narrative and visual consistency (Zheng et al., 2025; Pan et al., 2026; Guo et al., 2024). Document benchmarks test source-grounded answers, but a report that answers every factual query correctly may still hallucinate a citation, lose structural consistency across sections, or fail to render embedded tables in the final PDF (Zou et al., 2025; Ma et al., 2024a). In each case, a single correctness check passes while the deliverable as a whole would be rejected by a human reviewer.

Evaluation spans multiple quality dimensions. Six dimensions recur across artifact types, ordered from most to least automatable. *Structural validity*: the file is well-formed, parseable, and renderable (formulas execute, the XML schema validates, the PDF renders without errors). *Content correctness*: the artifact answers

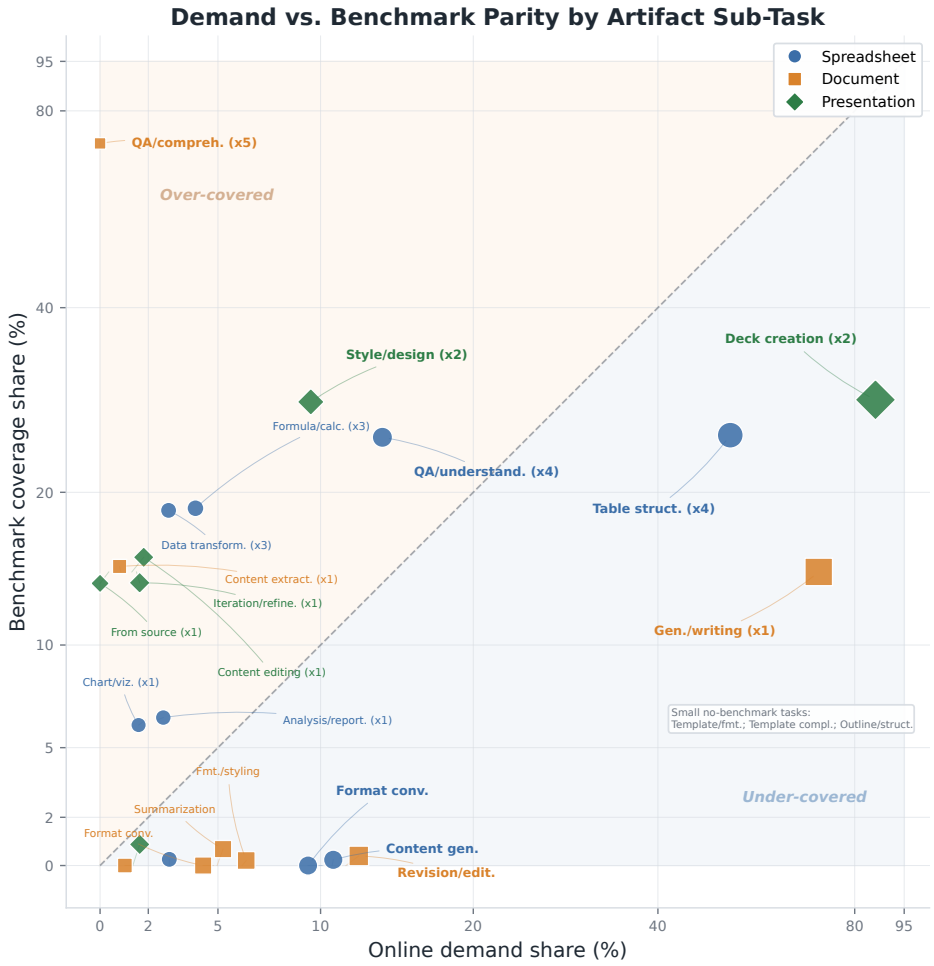


Figure 8 | **Demand–benchmark gap by artifact sub-task.** User demand (x; n = 11,155 conversations) is compared with benchmark coverage (y) for spreadsheet, document, and presentation tasks on a symmetric-log scale. The dashed diagonal indicates parity: lower-right points are under-covered, while upper-left points are over-covered. QA and comprehension are over-represented, whereas high-demand production tasks—such as deck creation, table structuring, content generation, revision, and formatting—remain under-covered.

the user goal g accurately (numbers are right, claims are true, requested elements are present). *Presentational quality*: the artifact looks professional (layout is clean, typography is consistent, charts are readable, color palettes are accessible). *Usability and editability*: the artifact is not a dead end (formulas remain live, styles are applied via templates rather than manual overrides, and a downstream user can revise without rebuilding). *Provenance and auditability*: every claim is traceable to a source, every number can be walked back to the raw data, and the artifact supports the organization's review and approval workflows. *Package consistency*: when the deliverable spans multiple files, values, references, and claims need to agree across the spreadsheet, document, slides, and any accompanying message (detailed below). No single benchmark covers all six: SpreadsheetBench focuses on structural validity and content correctness (Ma et al., 2024b), PPTEval extends to presentational quality with multi-aspect rubrics (Zheng et al., 2025), DocBench probes provenance through source-grounded questions (Zou et al., 2025), and OfficeBench and OdysseyBench begin to test package consistency across applications (Wang et al., 2024d, 2025). The open challenge is to compose these dimensions into a unified scoring framework that reflects the full deliverable lifecycle.

Verification cost determines the deployable frontier. These dimensions differ not only in what they measure but in how costly they are for a user or evaluator to confirm. Some requests are *trivially verifiable* (changing a title font); some are *mechanically verifiable* (checking that a spreadsheet formula recomputes); a third class is *semantically verifiable* (tracing a report claim to the correct source and business definition); the

most difficult class is *subjectively verifiable* (judging whether a deck is persuasive, professional, and aligned with an organization's style). This taxonomy explains why spreadsheet tasks often appear more mature than document or presentation tasks: execution gives spreadsheets a cheap verifier, while documents and slides require provenance checks, rendered inspection, and human judgment. Capability therefore cannot be separated from verification cost: a system is deployable only when the cost of confirming d_T stays below the value created by delegating the task.

Presentation quality is integral, not cosmetic. Presentation generation makes this vivid: the GLM slide agent and the GLM-5 system frame slide quality as structured aesthetic objectives (Z.AI, 2026; Team et al., 2026a); AeSlides turns visual traits (aspect ratio, whitespace distribution, element collision, spatial balance) into verifiable layout checks that can serve as both evaluation criteria and training rewards (Pan et al., 2026); and the single-turn vs. session-level gap identified by PPTC (Guo et al., 2024) confirms that visual coherence across a deck cannot be decomposed into per-slide scores. Readability and visual layout are intrinsic to task success, not post-hoc decorations, and evaluation that ignores them systematically overestimates deliverable quality.

Many deliverables are packages, not single files. A quarterly review may bundle a workbook, charts, a written summary, slides, and an email brief; scoring each in isolation misses a common failure mode in which every file looks plausible while the package is internally inconsistent. Concretely, a revenue figure in the slide deck may not match the corresponding cell in the spreadsheet because the agent regenerated the chart instead of linking it; a document may cite "see attached spreadsheet" when the attachment contains a different date range; an email summary may report a conclusion that contradicts the narrative in the document it forwards. These cross-artifact inconsistencies are invisible to single-file evaluation and require package-level checks that compare values, references, and claims across files. OfficeBench and OdysseyBench take this direction by coordinating several applications and states within a single task (Wang et al., 2024d, 2025), and TheAgentCompany extends it by embedding such tasks in a simulated company so that long-horizon knowledge work, not isolated tool calls, becomes the unit of evaluation (Xu et al., 2024). Package-level consistency remains the least developed dimension in current benchmarks and one of the most consequential for real-world deployment.

4.3. Alignment and Protocols

Human alignment in AI Office is artifact-level. Users do more than judge whether a response sounds helpful: they decide whether an artifact is acceptable for a meeting, a client, or a workflow, and their preferences surface in revisions, comments, approvals, and repeated edits, not in chat ratings. The same artifact is also judged differently across organizations, whose conventions for number formatting, evidence standards, chart styles, and approval workflows vary. Copilot Tuning treats such organizational examples and rubrics as inputs to context, tool, and model tuning (Microsoft Learn, 2026b); local standards need to attach to artifacts and workflows, and prompting alone rarely carries them. This also means the user's role is broader than final approval. For low-risk edits the user is a reviewer; for ambiguous office work the user is often a decider who selects among goals, audiences, evidence standards, and visual styles, or a collaborator who supplies missing constraints mid-run. Evaluation protocols should therefore measure whether a system can elicit the right clarification, expose meaningful alternatives, and preserve editability for partial takeover, not only whether it can autonomously finish a task.

Rule-based checks are the foundation. Deterministic checks are reliable for explicit constraints: formula execution verifies that spreadsheet outputs recompute correctly, XML/OOXML schema validation confirms that the file is well-formed, diff inspection catches unintended edits, and rendering checks detect element overflow or collision in slides (Li et al., 2023a; Ma et al., 2024b; Pan et al., 2026). Rule-based evaluation can also check beyond final values: for spreadsheets, whether the computation path uses live formulas instead of hard-coded constants; for documents, structural compliance (heading levels, citation format, table numbering); for slides, layout constraints (font sizes, margin widths, color contrast ratios). The strength of rules is their determinism and interpretability: a failed rule points to a specific, fixable defect. Their limitation is coverage: they can verify what has been anticipated but cannot judge whether a deck is persuasive, a chart is misleading, or a report reads naturally.

Rubric-based evaluation captures quality dimensions that rules miss. When evaluation criteria involve judgment (Is the narrative coherent? Is the chart type appropriate for the data? Does the executive summary capture the key findings?), human evaluators apply rubrics with ordered scales and anchor descriptions. PPTEval operationalizes the multi-granularity evaluation discussed in §2.3 by defining rubrics at each level (element,

slide, and deck) with ordered scales and anchor descriptions that make subjective quality reproducible across raters (Zheng et al., 2025). The rubric approach makes subjective assessment structured and reproducible, but it is expensive to scale and subject to inter-rater variability.

LLM-as-a-Judge scales rubric evaluation, with known limitations. G-Eval uses chain-of-thought prompting to guide LLMs through evaluation rubrics, achieving higher correlation with human judgments than traditional metrics on NLG tasks (Liu et al., 2023). MT-Bench and Chatbot Arena validate that LLM judges can approximate human preferences in pairwise comparisons (Zheng et al., 2023). Prometheus shows that fine-tuning a model specifically for rubric-based evaluation produces more consistent and fine-grained scores (Kim et al., 2024). For office artifacts, LLM judges can score content fidelity, rubric adherence, and textual quality, but they have specific limits: they cannot execute formulas to verify spreadsheet correctness, they may miss subtle layout issues visible only in the rendered output, and they carry known biases of position (favoring earlier content), verbosity (favoring longer outputs), and scoring instability (different scores on repeated runs) (Chen et al., 2024). These biases matter for office evaluation because they systematically favor verbose but shallow deliverables over concise but correct ones.

Agentic evaluation is a promising direction, with caveats. Agent-as-a-Judge scores the task-solving process via tools and intermediate-state inspection, not the final response alone (Zhuge et al., 2024). In AI Office, this mirrors how human reviewers actually work: they open the workbook and check that formulas are live, render the deck and scan for layout issues, trace document claims back to their sources, and verify that cross-file references are consistent. An agentic evaluator can compose rule-based checks (execute the formulas, validate the schema), rendered-output inspection (screenshot the slides, check for overflow), and rubric-conditioned judgments (assess narrative quality) into a single evaluation pass. The advantage is that the evaluator operates on the same artifact surfaces the agent used to produce the deliverable, so it can catch failures a text-only judge would miss. Agentic judges need to be calibrated against human acceptance, however, and audited for the same failure modes they are designed to detect: hallucinated assessments, tool-inspection errors (failing to open the right file), and inherited biases from the underlying LLM (Chen et al., 2024).

A practical evaluation stack composes these protocols. No single protocol covers the full spectrum of office deliverable quality, and any protocol used in isolation has visible limits: a rule verifies one cell but not whether its value transferred to the slides; a rubric scores clarity but not source grounding; a single-pass LLM judge may skip intermediate artifacts and trajectories. A practical evaluation stack therefore layers them: deterministic checks establish structural validity and content correctness as prerequisites; rubric-conditioned LLM judges score presentational quality and communicative effectiveness at scale; agentic judges handle cross-artifact consistency and provenance checks that require opening and inspecting multiple files; and human reviewers are reserved for calibration, high-stakes cases, and dimensions where automated methods remain unreliable. This composition parallels the evolution traced in Figure 7, from simple rule-based scoring to the Agent-as-a-Verifier paradigm that combines all protocol types under agentic orchestration.

4.4. Benchmark–Deployment Distribution Gap

Current benchmarks have built indispensable infrastructure. SpreadsheetBench, SheetCopilot, and SheetAgent established execution-based evaluation for spreadsheet manipulation (Ma et al., 2024b; Li et al., 2023a; Chen et al., 2025); PPTEval and AeSlides operationalized multi-granularity and layout-level quality for slides (Zheng et al., 2025; Pan et al., 2026); DocBench and MMLongBench-Doc brought source-grounded evaluation to documents (Zou et al., 2025; Ma et al., 2024a); and OfficeBench and OdysseyBench pioneered cross-application workflow evaluation (Wang et al., 2024d, 2025). These contributions define the evaluation paradigms (workspace transition as the basic unit, multi-dimensional quality criteria, artifact-state scoring) on which future work can build. What they have not yet addressed is whether the *distribution* of tasks they evaluate matches observed demand signals from deployed office-assistant settings.

Online query evidence suggests a structural mismatch. To triangulate benchmark coverage against demand, we analyze aggregate intent labels from online AI Office query exports (details in Appendix E). The statistics are aggregate and de-identified; we do not analyze raw user prompts, uploaded files, or user-level behavior in this paper. The analysis covers 11,155 successfully classified conversations after removing invalid or missing labels: 5,656 Excel, 3,054 Doc/PDF, and 2,445 PPT conversations. Because the exports are classified logs rather than a public random sample, and because their collection window is unavailable, we use them as descriptive demand evidence rather than as a population estimate of all office work or a market-wide estimate

Deliverable dimension	What evaluation inspects	Verifier / protocol	Training signal	Office example
Structural validity	File opens, schema is valid, formulas execute, rendered objects do not overflow	Rules, parsers, execution, rendering checks	Rejection sampling, RLVR, regression tests	XLSX formulas recompute; PPTX text boxes fit
Content correctness	Numbers, claims, requested elements, and transformations match the goal and source data	Unit checks, source comparison, task rubrics, agentic inspection	Curated SFT, verifier rewards, repair labels	Table totals match source workbook; report claims match uploaded PDFs
Presentation quality	Layout balance, typography, chart readability, visual hierarchy, and deck-level coherence	Visual rules, rubric judges, human calibration	Visual rewards, preference data, OPD critiques	Slide deck is readable and follows template constraints
Editability and maintainability	Output remains live, structured, and easy to revise rather than flattened or manually patched	Diff inspection, object-model checks, style/template checks	Demonstration filtering, skill updates, verifier-guided SFT	Formulas remain formulas; headings use styles rather than manual spacing
Provenance and auditability	Claims, numbers, and edits can be traced to sources, raw data, or prior versions	Citation checks, lineage comparison, agentic file inspection	Trace-level rewards, preference pairs, organizational tuning	Executive summary cites the correct sheet, chart, and document section
Package consistency	Values, references, and narrative agree across spreadsheets, documents, slides, and messages	Cross-file validators, agent-as-verifier, human review for high-stakes cases	Multi-agent RL, package-level rewards, orchestrator feedback	Revenue figure in spreadsheet, memo, and deck is identical

Table 1 | **Deliverable-centric evaluation as a training interface.** Each quality dimension can be operationalized as an inspection target, a verifier, and a training signal. This mapping connects the evaluation stack in §4 to the SFT, OPD, skill-learning, and RL pipeline in §5.

of AI Office usage. Within that scope, the distribution is strongly concentrated on concrete artifact production. For Excel, 80.4% of requests are direct-execution operations (table structuring, data transformation, formula writing); for Doc/PDF, 88.2% target content generation, revision, formatting, or conversion; for PPT, 82.7% are deliverable-generation requests, with 83.0% of actions being creation from scratch or from source files. By contrast, only 2.8% of online requests (weighted across families) target standalone QA or understanding, yet QA and comprehension benchmarks (DocVQA, ChartQA, TableBench, Spider, BIRD, and variants) constitute roughly a quarter of the benchmarks surveyed in §4 and Appendix D. Read at the finer grain of artifact sub-tasks, the imbalance is starker still: the available benchmark suite is heavily skewed toward question answering, while the production work that is prominent in the analyzed exports—deck creation, table structuring, content generation, multi-turn revision, and formatting or template compliance—is sparsely covered, and several such sub-tasks have no dedicated public benchmark at all. A practical consequence is that open-source datasets offer limited reference for many real-world business scenarios: models tuned and ranked on the current public distribution can look strong on leaderboards while remaining under-prepared for the office work represented in these demand signals. Table 2 and Figure 8 quantify this imbalance across task categories and artifact sub-tasks. Appendix E defines the coarse aggregation used in Table 2; Figure 8 further breaks those categories into artifact-family sub-tasks to show where the mismatch appears within each family.

Public industry telemetry independently corroborates this demand profile. Because these exports are a single internal sample, we triangulate them against two large public studies of how people use current-generation assistants. OpenAI's analysis of Codex usage reports that knowledge-artifact production is the most common knowledge-work category, with 72% of knowledge workers producing artifacts such as documents and multimedia each week, and that data analysis is the fastest-growing category at roughly +110% week-over-

Table 2 | Task-category gap between online user demand and benchmark coverage. Online shares are weighted by conversation count across the three artifact families (Excel, Doc/PDF, PPT). Categories are coarse cross-family aggregates whose label mapping is defined in Appendix E; shares are therefore approximate ("~" marks an approximate value; "—" a category with no dedicated online label). Benchmark counts are drawn from the works surveyed in this section and Appendix D and are unweighted by benchmark size or difficulty.

Task Category	Online (%)	Benchmarks	Gap
Direct execution	56.2	4	Under-covered
Artifact generation	25.6	4	Moderate
Iterative refinement	10.4	1	Severe
QA / understanding	2.8	6	Over-covered
Formatting / templates	~5	0	Blind spot
Cross-artifact workflow	—	2	Under-explored

week, ahead of research (+37%) and knowledge-artifact creation (+36%), with PDF- and spreadsheet-related work growing by more than 50% (OpenAI, 2026b). This is consistent with the artifact-production-dominated, analysis-heavy distribution we observe online and suggests that standalone QA is a small slice of the sampled demand. Anthropic's survey of roughly 81,000 users adds the verification dimension from the user side: the single most frequent concern is *unreliability* (26.7%; hallucination, inaccuracy, and the burden of checking outputs), followed by loss of *autonomy and agency* (21.9%) and inadequate *governance* (14.7%) (Anthropic, 2026c). These concerns are consistent with the high verification-risk share in our own exports (30.5% of Excel and 25.6% of Doc/PDF requests) and reinforce the central evaluation argument: plausible artifacts are insufficient unless users can afford to verify their correctness, which is what the certification-oriented protocol stack of §4.3 aims to support.

A task-attribute view points to the same bottleneck from the supply side. The query logs above characterize *what* users ask for; a complementary internal catalog of 318 office-agent tasks characterizes *how hard each is to delegate*, scoring every task on value, automatability, and risk (1–5 scales that are expert judgments, not measured quantities, so we read them as ordinal structure). The catalog concentrates on valuable, automatable, multi-step work: 78% of tasks are high-value (≥ 4), 61% are highly automatable (≥ 4), 53% score high on both, 79% require genuine multi-step autonomy beyond a single-turn answer, and three quarters are best served by hierarchical planning (15% are amenable to parallel multi-agent execution). Risk, by contrast, is dispersed: 26% of tasks carry elevated risk (≥ 3.5) and 16% are high-risk (≥ 4), concentrated in financial, legal, health, and irreversible-action work. The practical reading of this internal catalog is that many candidate office-agent tasks are high-value, automatable, and long-horizon, while a substantial risk-bearing minority sets the verification and guardrail requirements, so benchmark and reward coverage of the harder-to-verify dimensions (§4.2) matters disproportionately to real deployability.

Three specific gaps deserve attention. First, *formatting, styling, and template compliance* account for roughly 5–10% of online requests at the operation-semantics level (Doc/PDF format/style 8.3%, PPT formatting/template 4.3%, and a substantial share of Excel structure-management requests), yet to our knowledge no current benchmark in the surveyed set targets these operations directly. This is not a minor omission: as argued in §4.2, presentational quality is integral to deliverable acceptance, and systems trained without formatting-aware rewards may underperform on tasks that appear frequently in the analyzed exports. Second, *iterative multi-turn refinement* accounts for 10.4% of primary intents (weighted) and 42.9% of PPT sessions involve multi-turn interaction, but only PPTC (Guo et al., 2024) evaluates session-level editing, and only for slides. Real office work is inherently iterative: users draft, review, revise, and re-submit, and benchmarks that test only single-turn generation miss the failure modes that emerge over multiple revision cycles. Third, the *verification risk* data shows that 30.5% of Excel and 25.6% of Doc/PDF requests carry high or critical verification risk (Appendix E), yet current benchmarks concentrate on mechanically checkable correctness (formula execution, cell-value comparison) and leave the semantically and subjectively verifiable tasks, which are the most difficult to evaluate and often the most consequential to get wrong, without systematic coverage.

The coverage gaps are partly explained by genuine difficulty. Not every under-covered category reflects oversight; some reflect open problems. Formatting and template compliance are under-benchmarked partly because the task is not yet well-defined: "follow the company template" presupposes organizational context that no public benchmark can ship, and the line between correct and acceptable formatting is blurry. Iterative

refinement is hard to benchmark because success depends on the full session trajectory, not a single turn, and constructing realistic multi-turn editing scenarios at scale requires either expensive human authoring or faithful simulation of revision behavior. At a deeper level, several high-demand capabilities such as aesthetic judgment, layout balance, and visual professionalism draw on design expertise adjacent to but distinct from core AI Office competence; benchmarking them requires evaluation criteria imported from graphic design and visual communication, which the NLP and agent communities have only recently begun to formalize (Pan et al., 2026; Zheng et al., 2025). Recognizing these difficulties does not diminish the gap but reframes it: closing the mismatch requires not just more benchmarks but new evaluation methodologies for tasks that resist mechanical verification.

Post-ARR end-to-end benchmarks sharpen this gap. Several recent benchmarks move evaluation closer to the deliverable-centric ideal. Mind the Gap evaluates 200 standardized practical Office-operation tasks across Word, Excel, and PowerPoint with 7,118 machine-gradable rubric criteria, showing that current systems still struggle with fine-grained office automation (Lv et al., 2026). Workspace-Bench shifts the unit from a single file to a realistic workspace with large-scale file dependencies, requiring agents to retrieve, reason over, and update heterogeneous files (Tang et al., 2026). BankerToolBench evaluates high-stakes professional workflows in which agents produce multi-file investment-banking deliverables—including Excel models, PowerPoint pitch decks, and PDF/Word reports—against expert rubrics (Lau et al., 2026). AIDABench adds end-to-end data-analytics tasks that combine document analysis, visualization, and file generation (Yang et al., 2026). Adjacent work such as OfficeQA Pro probes grounded multi-document reasoning over enterprise corpora, an upstream capability for deliverable provenance (Opsahl-Ong et al., 2026). Together, these benchmarks support the same conclusion as our demand analysis: the key gap is no longer standalone QA, but reliable certification of artifact and workspace states.

Closing the gap requires demand-aware benchmark design. The mismatch between benchmark coverage and deployment demand means that leaderboard rankings may not predict real-world utility: a model that excels at document QA but struggles with formatting and iterative refinement will score well on current benchmarks while underperforming in production. Future benchmark suites should weight task distributions toward observed demand, include formatting and template-compliance tasks alongside content and reasoning tasks, and measure multi-turn refinement as a first-class evaluation target. This demand-side view complements the protocol-level analysis in §4.3: even a perfect evaluation stack cannot improve deployed quality if it is applied to the wrong distribution of tasks.

4.5. Takeaways

Current benchmarks test whether the model can *do* something, but not whether the resulting file is actually good enough to ship. The gap is both in what gets measured and in how it gets measured.

- **Judge the final file, not the steps.** What matters is the committed deliverable $d_0 \rightarrow d_T$, scored on correctness, formatting, provenance, and consistency—not how many tool calls it took (§4.2).
- **Quality coverage is thin.** Layout, styling, cross-sheet consistency, and source provenance are rarely benchmarked, even though users notice them immediately (Table 1).
- **Demand coverage is also thin.** Real users ask for formatting, template-following, and iterative refinement far more than benchmarks reflect (§4.4, Figure 8).
- **No single judge is enough.** Reliable evaluation stacks rule-based checks, rubrics, LLM judges, and agentic verifiers together, then calibrates against human acceptance (§4.3).

5. AI Office Training: From Demonstrations to Rewards

The preceding sections established how agents carry out edits and what criteria make a deliverable acceptable. The remaining question is how to turn those criteria into signals the model can learn from. Early systems learn from curated demonstrations and labeled patterns, but reaching the level of goal-driven office agency requires artifact-level rewards that ground the signal in the deliverable itself rather than in human annotations: formula-execution outcomes, rendered-layout checks, cross-file consistency scores, and organizational acceptance criteria.

Training follows the deliverable target. Given the evaluation criteria of §4, the training problem is not

Paradigm	Supervision	Reward type	Algorithm	Artifacts	Rep. work
Pretraining	Self-superv.	—	—	Sheet · Doc	DocLLM (Wang et al., 2024b)
SFT	Demonstr.	—	—	Multi	SheetCopilot (Li et al., 2023a)
Curated SFT / RS	Verif. filter	Rule · Sim.	RS	Multi	GLM-5 (Team et al., 2026a)
OPD	Own rollouts + teacher	Dense · verif.	distill.	Multi	OPD survey (Song & Zheng, 2026)
RLVR (DeepSeek-AI, 2025)	Verif. checker	Rule · Visual	GRPO (Shao et al., 2024b)/PPO (Schulman et al., 2017)	Sheet · Slide	AeSlides (Pan et al., 2026)
RLEF (Bai et al., 2024a)	Env. outcome	Interact. · Rule	online/async	GUI · Sheet	ComputerRL (Lai et al., 2025)
RLHF (Christiano et al., 2017)/RLAIF (Lee et al., 2023)	Model judge	Visual · Sim.	GRPO	Multi	SlideRL (Ananda Kumar & Arunachalam, 2026)
Multi-agent RL	Task+deleg.	Composite	PARL	Multi	Kimi K2.5 (Team et al., 2026b)
Org. adaptation	Local rubric	Similarity	SFT+adapt.	Multi	Copilot Tuning (Microsoft Learn, 2026b)

Table 3 | Representative office-agent training methods grouped by paradigm, ordered along the imitation → RL → adaptation spectrum (§5); the full per-work classification is in Appendix Table 8.

to teach more office vocabulary but to make the policy $\mathcal{A}$ more consistently improve office artifacts. SFT teaches executable office behavior, and feedback-driven selection and RL connect that behavior to deliverable outcomes scored by $\mathcal{V}(d_T, g)$. Table 1 makes the interface explicit: every deliverable-quality dimension exposes an inspection target, a verifier, and a corresponding training signal. Table 3 groups representative methods along the imitation → RL → adaptation spectrum. Appendix Table 8 gives the per-work details behind this compression, separating artifact type, training stage, supervision source, reward/verifier, and optimization method so cited systems are not treated as interchangeable examples.

Office-specific training aligns lower-level capabilities with final artifact quality. It builds on the capabilities of §3 (layout-aware reading, code, tool use, and GUI grounding) rather than replacing them, so improvements at that layer raise the ceiling (Huang et al., 2024; Team et al., 2026b), and then connects those skills to deliverable-level goals.

The training example is therefore a trace, not a prompt–answer pair. An office training instance records the initial artifact state, the user goal, intermediate observations, committed edits, verifier outputs, and the final artifact state. This trace-level view matters because the same final answer can arise from very different workspaces: a spreadsheet may contain live formulas or pasted constants; a deck may preserve template styles or override them by hand; a report may cite sources through valid references or unsupported prose. Training signals that ignore these distinctions teach surface imitation; signals attached to artifact diffs, rendered previews, execution logs, and provenance checks teach the model which trajectories leave behind usable deliverables. This is the bridge between the operation loop of §3 and the evaluation stack of §4: the data used for SFT, rejection sampling, reward modeling, or RL should expose the same states and checks the deployed verifier will later inspect.

Product systems increasingly expose the same supervision surface. Anthropic's Skills package instructions, scripts, and resources so an agent can apply repeatable procedures for PDFs, spreadsheets, presentations, and documents (Anthropic, 2025d,e); Microsoft frames Work IQ and Copilot Tuning as ways to condition agents on organizational data, memory, examples, and approval patterns (Microsoft 365 Blog, 2025a; Microsoft Learn, 2026b); Google Workspace Studio makes recurring agent workflows shareable and templateable across an organization (Google Workspace, 2025b). These systems are not training pipelines, but they define the data exhaust from which office adaptation can be learned: successful skill invocations, failed validations, user corrections, accepted artifacts, rejected drafts, and recurring local procedures. The product boundary and the training boundary therefore start to overlap: the runtime that executes work also records the traces that can improve future work, provided they are governed, filtered, and converted into safe supervision.

5.1. Learning Office Actions

SFT gives the agent the office action grammar. Its role in AI Office is not to make the model write more fluent explanations, but to teach how a user goal g is converted into executable edits over a persistent workspace. Across the surfaces of §3, a useful demonstration shows how (d_0, g) becomes a sequence of actions $(a_0, \ldots, a_{T-1})$ and intermediate states $(d_1, \ldots, d_T)$: which file is opened, which range is selected, which API is called, which visual state is inspected, and which edit is finally committed. This is why SFT remains the entry point for office agents even when later stages use OPD or RL: without a learned grammar of office actions, the model may know the task conceptually but fail to operate the file.

The demonstration format follows the office surface. For spreadsheets, SFT should teach cell-addressing, formula insertion, range selection, chart creation, and workbook-state preservation; a trajectory might include an API call such as `set_cell(B2, "=SUM(A1:A10)")` plus the resulting workbook state, as in SheetCopilot's atomic-action traces (Li et al., 2023a). For slides, SFT should teach the document-to-deck workflow: extract claims from a source, select a template, populate placeholders, choose charts, preserve theme constraints, and repair visual layout; PPTAgent and AutoPresent instantiate this as structured object-model operations rather than free text alone (Zheng et al., 2025; Ge et al., 2025). For documents, SFT should teach edit operations that preserve reviewability: insert or rewrite paragraphs, add citations, reformat tables, maintain section structure, and keep source links traceable (Raheja et al., 2023; Schick et al., 2022). For GUI-based office work, the same idea appears as screenshot–action demonstrations, where the model learns to map rendered interface states to clicks, menu choices, and keyboard inputs (Cheng et al., 2024; Hong et al., 2024). The common pattern is that SFT teaches *how office work is carried out*, not whether the final deliverable is good enough.

In AI Office, SFT is strongest for conventions and weakest for acceptance. It is effective for recurring procedural conventions: using spreadsheet formulas, invoking a slide template, preserving DOCX structure, navigating a toolbar, or calling a local parser. These behaviors are frequent, reusable, and often easy to demonstrate. Raw demonstrations, however, also inherit human shortcuts and invisible mistakes. A human may paste a value where a live formula is needed, manually nudge a slide until it looks acceptable on one screen, or cite a source without preserving machine-checkable provenance. If such traces are imitated directly, the agent learns plausible office behavior rather than acceptable office deliverables.

Therefore the real SFT axis is raw imitation vs. deliverable-curated imitation. The improvement comes from filtering demonstrations by whether the resulting d_T passes artifact-level checks. At the simplest level, execution-based filtering removes traces where formulas fail, scripts crash, files cannot be reopened, or generated objects violate the application schema. For richer office work, rejection sampling generates multiple candidate trajectories, evaluates each with $\mathcal{V}$ or an agentic verifier, and retains the candidates whose committed artifacts are most usable (Team et al., 2026a). This makes curated SFT an office-specific quality layer: it still teaches action sequences, but it privileges trajectories that leave behind live formulas, readable layouts, traceable claims, and editable files. In the overall AI Office stack, SFT is best treated as the stage that teaches valid operations and local conventions; OPD and RL then address recovery, optimization, and long-horizon acceptance.

5.2. On-Policy Distillation for Office Trajectories

OPD is useful for AI Office because office failures appear on the agent's own path. On-policy distillation (OPD) trains the student on trajectories sampled from the student's current policy, while a stronger teacher or verifier supplies feedback on those student-visited states (Song & Zheng, 2026; Li et al., 2026b). This distinction is especially important in office work. An off-policy demonstration shows a clean path from goal to artifact, but a deployed office agent often enters messy intermediate states: it writes a formula that evaluates but breaks formatting, creates a slide that preserves content but overflows after rendering, or drafts a report whose paragraph is fluent but loses source provenance. These are not merely wrong final answers; they are recoverable workspace states. OPD exposes the model to its own partial mistakes and lets the teacher correct what should happen next, making it a natural bridge between curated SFT and artifact-level RL.

In AI Office, the teacher should judge workspace progress, not only text. For general LLM reasoning, OPD is often described as token-level or sequence-level teacher feedback. For office agents, the useful teacher signal is more operational: given a student-generated trace, the teacher can inspect the workbook, render the deck, open the document, run validators, and then indicate the next better action. In a spreadsheet task, this may mean preserving live formulas instead of pasting computed values; in a slide task, moving from a

text-only critique to a layout repair action after screenshot inspection; in a document task, rewriting a claim only after checking whether the source link and citation state remain valid. Thus the AI Office version of OPD should be *artifact-aware*: the teacher feedback is grounded in d_t, diffs, execution logs, rendered previews, and provenance checks, not just in the linguistic plausibility of the next token.

OPD fills three practical roles in the training stack. First, it is a *recovery trainer*: it teaches the agent how to repair its own invalid intermediate states before the final artifact fails. Second, it is a *capability compressor*: a slower high-capability agent or human-supervised agent can act as the teacher, while a smaller deployable student learns to perform recurring office procedures in enterprise settings where latency and cost matter. Third, it is a *pre-RL stabilizer*: before applying sparse artifact rewards, OPD can provide denser guidance at the exact states the student actually reaches, reducing the gap between supervised traces and online execution. For long office workflows, this matters because small local mistakes accumulate across files: a wrong spreadsheet range can propagate into a chart, then into a slide, then into a narrative conclusion. Dense teacher feedback on student rollouts can catch such drift earlier than final-pass scoring alone.

The office setting also makes OPD harder than ordinary reasoning distillation. The teacher needs to be compatible with the student's action space and tools; otherwise it may recommend operations the student cannot execute reliably. The teacher also needs to add genuinely useful capability instead of merely restating the student's current behavior, a failure mode highlighted in recent OPD analyses (Li et al., 2026b). Office trajectories are also permissioned and stateful. A teacher may be allowed to inspect a confidential workbook inside an enterprise boundary, but the resulting supervision cannot always be stored as raw text or pooled across tenants. OPD for AI Office should therefore distill sanitized signals: repair labels, validator outcomes, preferred action choices, tool-use rationales, or reusable procedural rules. Its value lies in matching the structure of office agency, not in being a fashionable post-training method: long trajectories, recoverable intermediate states, artifact-grounded verification, and deployment pressure to compress expensive supervision into reliable local behavior.

5.3. Turning Runtime Traces into Reusable Capability

The missing middle layer is skill-level learning. Office-agent improvement does not have to flow immediately from user traces into model weights. Between raw trajectories and parameter updates sits a more inspectable artifact: the skill, recipe, checklist, validator, or workflow template that tells a general agent how to act in a recurring office situation. This layer matters because many office failures are procedural, not conceptual. The model may know what a pivot table is, but still save an XLSX after loading formulas with the wrong library flag; it may understand slide design, but forget to render the deck before checking overflow; it may summarize sources correctly, but fail to preserve citation links in the final DOCX. Fixing these errors by fine-tuning alone is slow and opaque, while fixing them in a skill file is fast, auditable, and portable across models.

Runtime traces expose which skills are missing or brittle. A useful office trace contains more than the final success label: it records which files were opened, which parser or API was used, which intermediate checks failed, which repair action succeeded, and which user correction finally made the artifact acceptable. Aggregating such traces produces a failure distribution over procedures. If many spreadsheet runs fail because formulas are overwritten by cached values, the skill needs a stronger preservation rule; if many deck-generation runs pass content checks but fail rendered layout inspection, the skill needs a mandatory screenshot verifier; if document-generation runs repeatedly lose source provenance, the workflow needs citation-state tracking instead of another generic writing demonstration. Trace2Skill formalizes this idea by distilling trajectory-local lessons into transferable skill directories, including spreadsheet workflows, and shows that evolved skills can improve agents without parameter updates and transfer across model scales and out-of-distribution settings (Ni et al., 2026). SkillOS and SkillsBench point to the complementary problem: as skill libraries grow, the agent needs to learn which skills to load, compose, revise, or retire, instead of treating all procedural knowledge as equally useful (Ouyang et al., 2026; Li et al., 2026a).

This creates a three-speed training loop. The fastest loop is *runtime repair*: update a skill, validator, template, or routing rule after observing a concrete failure. The medium-speed loop is *data curation*: convert successful and repaired trajectories into SFT or rejection-sampling data once they pass artifact-level verification. The slowest loop is *parameter training*: use RL, reward modeling, or organizational tuning when a pattern is stable enough to justify changing the model itself. AI Office needs all three. Relying only on skill edits leaves the model dependent on brittle external procedures; relying only on model updates hides operational

policy inside weights; relying only on retrieval or prompting fails when the agent must consistently perform precise file operations. The practical training architecture is therefore hybrid: skills encode current procedural best practice, verifiers turn executions into measurable outcomes, and model training gradually absorbs the behaviors that recur across many tasks.

Traces require governance before becoming training data. Office traces often contain customer files, contracts, financial models, private emails, and organization-specific approval comments. Unlike public web demonstrations, these traces cannot simply be pooled into a generic dataset. Enterprise products increasingly reflect this constraint: Copilot Tuning uses organizational examples and evaluation rubrics while preserving access controls, permissions, and sensitivity labels; Workspace Studio emphasizes admin controls around sharing, webhooks, and data flow; and Agent Skills package procedural knowledge separately from the private files to which it may be applied (Microsoft Learn, 2026b; Google Workspace, 2025b; Anthropic, 2025d). For training, this means the usable signal is often a sanitized abstraction of the trace: a failure type, a validator result, a reusable recipe, a local style rule, or a preference comparison, not the raw artifact itself. The governance boundary is thus not peripheral to training; it determines which signals can be retained, shared, or used to update a model.

5.4. Optimizing Deliverable Outcomes

RL becomes useful when the office artifact itself can answer whether work improved. In AI Office, RL should not be introduced merely because it is the prevailing post-training recipe. It earns its place when an agent can act in a workspace, commit a state d_T, and receive a reward tied to whether that committed artifact is better. Office work is unusually suitable for this because many actions leave inspectable state: spreadsheet formulas execute, documents preserve diffs and citation links, slide decks can be rendered into screenshots, and GUI workflows either reach the intended file state or not. Training can therefore optimize the same artifacts that evaluation later certifies via $\mathcal{V}$, as in ComputerRL and AeSlides (Lai et al., 2025; Pan et al., 2026). The difference from generic RLHF is that the reward is often grounded in the file runtime rather than only in a preference over generated text.

RLVR is the right tool for office dimensions that are verifiable. For spreadsheets, the agent can be rewarded for producing workbooks whose formulas recompute correctly, whose referenced ranges match the task, and whose output cells agree with expected values (Chi et al., 2026; Ma et al., 2024b). This maps directly to office practice: a spreadsheet is not accepted because its explanation sounds right, but because changing an input causes the right downstream cells and charts to update. For slides, verifiable rewards apply to layout constraints such as element overlap, aspect-ratio compliance, whitespace distribution, and text overflow; AeSlides uses such checks as optimization signals for slide generation (Shao et al., 2024b; Pan et al., 2026). The office-specific advantage of RLVR is reproducibility: the same workbook, deck, or document state can be reopened and checked again. Its limit matters just as much: it cannot fully score persuasion, narrative quality, stakeholder fit, or whether a chart is the most appropriate framing of the business question.

RLEF fits office tasks that require interaction with live applications. Some office work cannot be reduced to a single file transform. The agent may need to open a browser-based spreadsheet, navigate a cloud document editor, approve a permission prompt, download a file, or coordinate across email, calendar, and storage. In these cases, the reward comes from environment feedback: did the workflow complete, did the expected file appear, did the application state match the target? ComputerRL, DART, and DigiRL show how online or asynchronous RL can train agents from such environment outcomes (Lai et al., 2025; Li et al., 2025; Bai et al., 2024a). For AI Office, the key characteristic is that environment feedback is often necessary but too coarse. A run may finish the task while leaving a brittle artifact, or fail because of a temporary UI branch, not a capability gap. RLEF is therefore most useful when paired with artifact checks that explain *why* the workspace state is acceptable or not.

RLHF and RLAIF cover the office qualities that are real but hard to formalize. Many deliverable criteria are evaluative, not deterministic: whether an executive deck tells the right story, whether a memo is concise enough for review, whether a report balances evidence and recommendation, or whether a chart communicates the intended comparison. For these dimensions, human preference data or AI feedback can provide reward signals. SlideRL, for instance, combines visual assessment with content-fidelity scoring for presentation quality, while the GLM slide and poster agent describes rule-based and model-judge rewards for improving near-ready artifacts (Ananda Kumar & Arunachalam, 2026; Z.AI, 2026). In AI Office, these judge-based rewards should be treated as proxies for user acceptance, not as ground truth. They are valuable

when deterministic checks stop short of communicative quality, but they can also reward verbose, conventional, or visually polished artifacts that do not actually satisfy the workplace goal.

Agentic verifiers make RL more office-native. The Agent-as-a-Judge of §4.3 reappears in training as an agent-as-a-verifier: it opens the workbook, executes formulas, renders slides, checks document provenance, compares cross-file numbers, and produces a reward, critique, or repair target (Zhuge et al., 2024). This is closer to how office deliverables are reviewed by humans. A reviewer does not score a deck by reading its text tokens; they open the file, inspect visual layout, trace numbers back to the spreadsheet, and ask whether the package can be sent onward. For RL, this means the reward should often be a composed judgment over a deliverable package instead of a scalar score from a single text judge.

RL should be staged after SFT, OPD, and verification infrastructure. SFT teaches the action grammar, OPD teaches recovery on the student's own trajectories, skills and validators patch recurring procedural failures, and RL then optimizes behaviors whose outcomes can be measured reliably. GLM-5 illustrates the system-level pattern: curated SFT, reasoning RL, agentic RL, and cross-stage distillation are combined to improve capability while limiting regression (Team et al., 2026a). The same lesson applies at the deliverable level for AI Office. An initial policy runs in verifiable office environments, produces artifacts, receives rule-based, environment-based, model-judge, or agentic-verifier feedback, and is improved through revised data, OPD, reward modeling, or RL.

The office-specific risk is multi-dimensional regression. Optimizing one artifact dimension can damage another: a slide deck may satisfy layout constraints but misstate the underlying data; a spreadsheet may compute correctly but become unreadable; a document may be concise but lose source provenance; a GUI agent may complete a workflow while leaving the wrong file version committed. This makes regression control central, not optional. Cross-stage distillation, verifier suites, and package-level evaluation are needed because AI Office rewards are not single-objective. The target is not merely a higher reward score, but a committed workspace that remains correct, editable, auditable, and acceptable across the full set of criteria captured by $\mathcal{V}$.

5.5. Team Training and Organizational Adaptation

Adaptation gives the team a shared standard. When multiple agents collaborate on a package-level deliverable (one handling the spreadsheet, another the slides, a third the document), they need to write toward the same quality bar. That bar is not universal: it is set by local examples, comments, templates, and approval habits that vary across organizations. The organizational signal that §4.3 treats as an evaluation rubric (Microsoft Learn, 2026b; Microsoft 365 Blog, 2025b) here becomes a tuning target, conditioning both orchestrator and worker policies on what "acceptable" means in this specific workplace. As knowledge workers shift toward *supervisory office work* (§4), their corrections, rejected drafts, accepted packages, and approval comments become a natural source of adaptation data for teaching an agent local standards.

Multi-agent training distributes the problem. Kimi K2.5 introduces PARL (Parallel Agentic RL), which trains a planning agent and multiple tool-use agents jointly: the planner learns to decompose goals and delegate sub-tasks, while each worker learns to execute its assigned sub-task and report back (Team et al., 2026b). For office work, this maps naturally to package-level deliverables: the orchestrator decomposes "prepare the quarterly review" into spreadsheet analysis, document drafting, and deck assembly, and each specialist agent is trained on its own artifact type while the orchestrator is trained on cross-artifact consistency. The reward signal for the orchestrator depends on the joint output of all workers, however, which makes credit assignment difficult: when the package is inconsistent, is the fault in the plan, in one worker's execution, or in the coordination protocol?

Organizational adaptation requires preserving general capability. Copilot Tuning treats organizational examples, preferred phrasings, formatting conventions, and approval patterns as conditioning signals for fine-tuning (Microsoft Learn, 2026b). The goal is to make the agent produce deliverables that a specific organization would accept without further revision, reducing the review burden. The open challenge is that heavy adaptation to one organization's standards risks overfitting: the agent may become excellent at producing quarterly reports in the style of Company A but lose the ability to generate a different format for Company B. Balancing local adaptation with general capability, learning organizational preferences without regressing on core skills, mirrors the broader regression challenge of multi-stage RL, but at the deployment level instead of inside the training pipeline.

5.6. Takeaways

The core insight is simple: office quality becomes trainable once you ground reward in the actual artifact state rather than in free-form text responses. Three principles recur across the methods we reviewed.

- **Fix fast before retraining.** Patching a skill or validator after a concrete failure (hours) beats retraining on the same pattern (days–weeks). Runtime repair, data curation, and parameter training form a three-speed hierarchy—traverse them in that order (§5.3).
- **Compose rewards or regress.** No single signal covers all quality dimensions; optimizing one in isolation (e.g., correctness) predictably degrades another (e.g., formatting) (§5.4).
- **Org adaptation is its own training problem.** Learning local conventions without eroding general capability is the same regression challenge as multi-stage RL (§5.5).

The open problem: operationalizing these principles without collapsing into shallow imitation, brittle rules, or ungoverned trace collection.

6. Systems, Products, and Deployment Trends

The preceding sections examined how agents operate on office artifacts (§3), how deliverables should be evaluated (§4), and how agents can be trained to improve (§5). This section maps the resulting technical landscape onto deployed systems and products, classifying them along the autonomy spectrum defined in §2.1 and identifying architectural trends that shape the field's trajectory. Its purpose is not to catalogue every office AI product, and no product inclusion implies endorsement, ranking, or controlled capability comparison. It instead asks which deployment signals corroborate the paper's central claim: AI Office is shifting from application-local features toward persistent, governed runtimes that operate across files, tools, permissions, and organizational memory. We therefore treat products as evidence of architectural pressure rather than as endpoints of the taxonomy. Product documentation, telemetry reports, and case studies are treated as deployment or demand signals, not as experimental evidence of technical capability.

6.1. Deployment Signals and Office Demand

Recent deployment reports and public runtime projects suggest that harness-based architectures are becoming a practical substrate for office work and not solely a research idea. Open-source and product runtimes such as Claude Code, Codex CLI, ChatGPT Agent, and reusable skill libraries indicate broad investment in persistent environments that go beyond single-turn chat (Anthropic, 2025b; OpenAI, 2025c,a; Anthropic, 2025d,e). These signals are not controlled evaluations, and their adoption metrics should not be read as capability measurements. Their value for this survey is narrower: they reveal which architectural patterns (workspace state, tool dispatch, reusable procedures, and reviewable edits) are being adopted in systems that increasingly touch office deliverables.

Deployment reports also indicate that these runtimes are no longer used only by software engineers. OpenAI's knowledge-work report describes Codex usage expanding into data analysis, research, reports, PDFs, spreadsheets, contracts, and other knowledge artifacts (OpenAI, 2026b), and a companion analysis documents the broader shift toward agentic usage patterns, including concurrent task delegation and reusable skills (Johnston et al., 2026). By mid-2026 the same runtime had surpassed five million weekly active users, more than 6× its level at the desktop-app launch earlier that year, with knowledge workers, rather than software engineers, accounting for roughly 20% of users and growing more than three times as fast (OpenAI, 2026b). We use this as demand-side context, not as proof that current systems satisfy the deliverable-quality criteria in §4. The relevant implication is architectural: office users appear to need systems that gather scattered context, execute multi-step operations, and return reviewable work products instead of merely answering questions.

Startup and platform evidence points to the same runtime pressure in a more granular way. Y Combinator's recent request for startups emphasizes software that agents can access through machine-readable interfaces such as APIs, MCPs, and CLIs (Y Combinator, 2025a). YC-backed examples cluster around three office-relevant capabilities: bringing AI functions into existing artifacts, as in Melder's Excel add-in (Y Combinator, 2024b); converting messy multimodal office inputs into structured outputs for downstream agents, as in Unsiloed AI (Y

Combinator, 2025c); and coordinating work across SaaS systems and organizational context, as in Capacitive, StackAI, Altrina, and related workflow-automation companies (Y Combinator, 2024a, 2023, 2025b, 2026). These cases should be read as product-market signals for the problem distribution described in Appendix E, not as technical evidence that the products solve AI Office evaluation or training.

6.2. Representative Systems across the Autonomy Spectrum

Figure 2 positioned representative systems along two dimensions: artifact type and autonomy level. We now unpack this classification with the technical vocabulary established in §3–§5, so each system can be located not only by what it does but by how it operates, how it is evaluated, and how it was trained (Table 4 in Appendix A provides the full per-work crosswalk).

Stage I systems. As detailed in the fragmentation discussion of §2.1, this stage consists of many narrow, single-artifact tools (formula generation, conditional-formatting synthesis, grammar correction, layout-aware reading, template synthesis, document-to-slide conversion), one per artifact-and-interface combination; the full per-system listing appears in the crosswalk (Table 4). The classification consequence is that no single Stage I system covers even one artifact type end to end, so assembling a multi-artifact deliverable requires the user to manually chain dozens of specialized features across separate applications.

Stage II systems. On the product side, Microsoft 365 Copilot drafts documents and generates linked charts across Word, Excel, and PowerPoint (Microsoft, 2023); Excel Copilot performs formula generation, data analysis, and chart creation within the spreadsheet surface (Microsoft, 2025b); Google Workspace's Gemini integration performs context-aware drafting across Docs, Sheets, and Slides (Google Workspace, 2025a; Google, 2025); Notion AI chains retrieval with structured editing in a wiki-style workspace (Notion Labs, 2023); and Beautiful.ai and Tome generate complete slide decks from natural-language prompts (Beautiful.ai, 2023; Tome App AI, 2023). On the research side, slide-focused prototypes span the full generation pipeline: PPTAgent extracts reference-based templates and generates decks through a multi-agent pipeline (Zheng et al., 2025); AutoPresent generates slides from free-form user instructions (Ge et al., 2025); SlideTailor personalizes scientific-paper-to-slide generation (Zeng et al., 2026); and PosterGen produces research posters via multi-agent collaboration (Zhang et al., 2025b). For spreadsheets, SheetCopilot orchestrates multi-step operations from natural-language instructions (Li et al., 2023a); Data-Copilot turns a natural-language analysis goal into a workflow over large data resources (Zhang et al., 2023); and SpreadsheetLLM addresses the encoding challenge of large, sparse workbooks (Dong et al., 2024). For documents, STORM extends the copilot pattern to long-form article production by coordinating source discovery, question-driven information seeking, outline construction, and grounded drafting (Shao et al., 2024a). At the product-interface level, this stage still often appears as a set of application-local or task-local entry points: a writing copilot, a spreadsheet assistant, a slide generator, or a document-chat tool. The Stage III pressure is to collapse these capsule-like agents into a unified workspace agent that preserves intent and evidence across file types, so the user asks for the deliverable package rather than choosing the right product surface first.

Stage III systems. Recent foundation models have approached a practical capability range for office delegation: scaling and emergent abilities make heterogeneous artifact understanding feasible (Kaplan et al., 2020; Wei et al., 2022; OpenAI et al., 2023; Team, 2023), and recent advances in reasoning, multimodal perception, and computer use make grounded, multi-step action over real files realistic (Anthropic, 2025a; OpenAI, 2025f,d; Lai et al., 2025). Once base capability reaches this range, the emerging architecture is a *harness*: a runtime that maintains workspace state, loads reusable skills, coordinates procedures for each artifact type, and carries intermediate evidence across actions. Anthropic's computer use work, Claude Code, and reusable Skills (Anthropic, 2024a, 2025b,d,e), OpenAI's agent tools, computer using agent, ChatGPT Agent, and Codex CLI (OpenAI, 2025e,d,a,c), and Microsoft's multi-agent orchestration mechanisms (Microsoft 365 Blog, 2025b; Microsoft Learn, 2026a) all point toward persistent runtimes that combine general models with reusable artifact skills. Several of these runtimes originated in coding or general computer-use settings; we include them here only insofar as they expose the execution pattern AI Office needs: persistent workspace state, callable tools, loadable procedures, and reviewable file edits. Domain-specific agent work in spreadsheets and slides pushes in the same direction, using feedback, rewards, or task-specific procedures to improve artifact-level execution and verification (Chi et al., 2026; Z.AI, 2026; Zeng et al., 2026). By late 2025, major office platforms were making this runtime direction explicit. Microsoft announced Office agents, Work IQ, and Agent 365 governance, later making agentic capabilities in Word, Excel, and PowerPoint generally available (Microsoft 365 Blog, 2025a,

2026); Google Workspace Studio exposed agent creation, templates, Apps Script, webhooks, sharing, and admin controls; and ChatGPT Agent exposed a computer-using runtime with connectors, file creation, and user supervision for consequential actions (Google Workspace, 2025b; OpenAI, 2025a). These product claims are heterogeneous and not directly comparable, so we use them only to support the architectural split used throughout this survey: models provide reasoning, but office delegation depends on a surrounding runtime for identity, permissions, files, tools, memory, and review.

The 2026 deployment wave and the non-Western ecosystem. The runtime direction above is no longer confined to the US platforms surveyed so far, and through 2026 it increasingly took the shape of *delegatable desktop agents* that act directly on a user's local files. Anthropic's Claude Cowork and Zhipu's AutoClaw each package an agent that reads, edits, and creates files in user-specified folders, moving the harness off the chat sidebar and onto the desktop where office work actually happens (Anthropic, 2026a; Zhipu AI, 2026; Qoder, 2025). A second, orthogonal signal is *parallelism as a product feature*: Moonshot's Kimi K2.6 exposes an "agent swarm" that fans a single goal out across up to hundreds of coordinated sub-agents (Moonshot AI, 2026; Team et al., 2026b), a deployment-scale instance of the multi-agent coordination discussed in §6.3. In the Chinese ecosystem in particular, vendors bundle a single super-entry with a toolset of office-relevant skills: Alibaba's Qwen and Quark span chat, document processing, slides, and search (Alibaba Group, 2026a); MiniMax and Manus ship general agents that build slides, sites, and data artifacts (MiniMax, 2026; Manus, 2026); and Alibaba's Wukong packages multi-agent office automation behind enterprise sandboxes, role-based access control, and a modular skill registry (Alibaba Group, 2026b). We cite these heterogeneous products only as architectural evidence, but together they corroborate the shift this survey argues for: from application-local features toward governed, skill-equipped runtimes that operate across a user's files, tools, and accounts. The same period also shows office deliverables feeding multi-stage *creative pipelines*: deck generators such as Gamma and Canva now sit upstream of tools that turn a document or slide set into narrated video. This signals that the artifact boundary this survey draws around documents, spreadsheets, and slides is itself softening toward richer multimodal outputs (Gamma, 2025; Canva, 2025; HeyGen, 2025).

A two-dimensional product landscape. The autonomy spectrum captures how much of a workflow a system will take on, but deployed products also differ in an orthogonal way: how general their entry point is. Crossing autonomy with this generality axis (a single general-purpose entry versus an artifact- or domain-specialized surface) sorts the systems above into four recurring archetypes (Figure 9): general super-agents that pair one entry with long-horizon execution, agentic workbenches that push the same execution depth into specialized desktop or enterprise surfaces, a knowledge and content layer that keeps execution light and human-approved, and creative-pipeline tools that generate a single artifact for downstream media. Each product is placed by reading its public documentation, demos, and deployment reports against two ordinal criteria rather than a measured score. The horizontal *generality* axis is drawn from general-purpose entry points on the left toward artifact-specialized or vertical products on the right; equivalently, reading right-to-left ranks increasing entry-point breadth, from a tool built around a single artifact or medium, through a suite assistant that spans a few artifact types, to one general-purpose entry that accepts open-ended goals across artifacts. The *autonomy* axis (vertical) reuses the spectrum of §2.1: from content assistance that proposes edits for a human to apply, through supervised multi-step editing inside a single surface, to a runtime that executes a long-horizon plan–commit–verify loop over files and tools with minimal supervision. A product's archetype follows from which side of each axis midpoint it falls on, and a system can migrate as it gains autonomy or broadens its entry point, so the coordinates are ordinal and illustrative, not measured. Read as architectural evidence, not a ranking, two patterns stand out: the autonomous-execution region is filling fastest, and products are moving upward across both general-entry and specialized-workbench regions over 2025–2026, both consistent with the shift this survey argues for from application-local features toward governed, cross-file runtimes.

Current systems should be read as partial steps toward Stage III rather than complete solutions. They expose ingredients of planning, workspace state management, and multi-step execution over office artifacts, but reliable office delegation still requires stronger guarantees on provenance, organizational policy, cross-file consistency, and failure recovery. Recent evidence that LLMs can corrupt documents under delegated editing reinforces this point: current systems support useful delegation, not unsupervised trust (Laban et al., 2026).

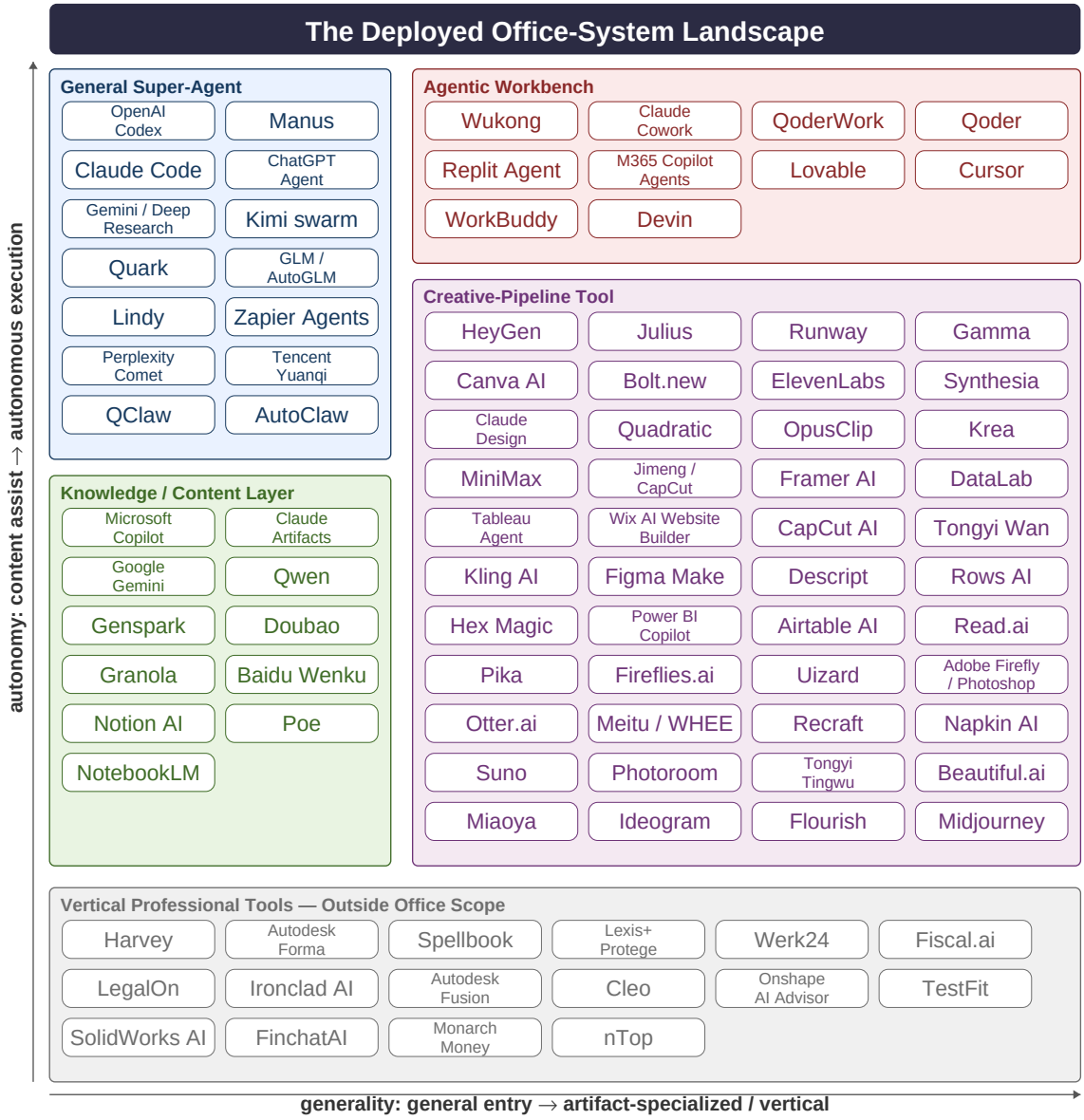


Figure 9 | **The deployed office-system landscape.** A curated product landscape positioned by generality (general entry → artifact-specialized/vertical, horizontal) and autonomy (content assistance → autonomous execution, vertical), reusing the operation surfaces of §3 and the autonomy spectrum of §2.1. Colour denotes archetype: general super-agents, agentic workbenches, the knowledge/content layer, and creative-pipeline tools; grey marks vertical professional tools (CAD, legal, finance) outside our office scope. Cited anchors include ChatGPT Agent (OpenAI, 2025a), Claude Code (Anthropic, 2025b), Codex (OpenAI, 2025c), Manus (Manus, 2026), Kimi (Moonshot AI, 2026), Claude Cowork (Anthropic, 2026a), Qoder (Qoder, 2025), Wukong (Alibaba Group, 2026b), Microsoft 365 Copilot (Microsoft, 2023), Workspace Gemini (Google Workspace, 2025a), Notion AI (Notion Labs, 2023), Gamma (Gamma, 2025), Canva (Canva, 2025), HeyGen (HeyGen, 2025), Runway (Runway, 2025), Devin (Cognition, 2024), and the wave of *OpenClaw*-style desktop agents—Tencent WorkBuddy (Tencent Cloud, 2026) and QClaw (Tencent, 2026) and Zhipu AutoClaw (Z.ai, 2026). Positions are ordinal and curated from public product documentation, not measured; the autonomous-execution region is filling fastest, including both general-entry and specialized-workbench products over 2025–2026. The curated roster of ≈95 systems, grouped by archetype with primary references, is given in Table 5 (Appendix B); it should not be read as a market-size estimate, ranking, or controlled capability comparison.

6.3. Skill Libraries and Runtime Architectures

Stage III systems need reusable skills for documents, spreadsheets, slides, and PDFs, together with verification loops that can catch broken formulas, stale citations, inconsistent numbers, and layout failures before the user sees the package. Skill libraries package procedural knowledge, validators, and repair routines that a general model can reuse across deliverables (Anthropic, 2025d,e; OpenAI, 2025c; Li et al., 2026a; Xu & Yan, 2026). Both Anthropic's and MiniMax's publicly released skill libraries adopt an *unpack–edit–repack* pattern for editing tasks, providing unpack and repack scripts alongside natural-language instructions that teach the agent which XML elements to modify (Anthropic, 2025e; MiniMax, 2026). For complex office platforms, however, a skill is more than a thin wrapper around an API. Its deeper role is closer to a *workflow compiler*: it routes intent to the right artifact procedure, chooses whether a lightweight or full recipe is needed, exposes capability limits early, and preserves the checks needed for safe delivery. This makes intent routing and failure boundaries as important as tool coverage; binding every API is less useful than knowing which actions should be composed, which should be blocked, and which require user approval. Production skill libraries therefore need their own lifecycle discipline. A new or revised skill should be tested against recurring office cases, checked for regressions on neighboring workflows, and repaired from concrete failure traces rather than edited only by intuition. In this sense, skill development begins to resemble CI/CD for office procedures: execute representative tasks, evaluate the resulting artifacts, extract failures, and update the skill without silently degrading other deliverables. Enterprise systems are moving toward delegation structures in which separate workers handle specialized subtasks while an orchestrator maintains the package-level goal (Microsoft 365 Blog, 2025b). Emerging protocols reshape this landscape: MCP standardizes tool exposure so that a single agent can discover and call heterogeneous office APIs without per-integration wiring (Anthropic, 2024b; Li et al., 2026c), while A2A enables inter-agent capability publishing (Google Developers Blog, 2025). The production layer around these skills is also becoming visible. Workspace Studio and Agent 365 emphasize sharing, templates, admin controls, registry, access control, and observability over agent work (Google Workspace, 2025b; Microsoft 365 Blog, 2025a). Such product mechanisms do not establish model quality, but they identify runtime requirements that technical systems must satisfy before office delegation can be trusted: reusable skills, governed access to organizational context, and observability over human-agent work.

6.4. Loop Engineering: Operating the Office Runtime as a Closed Loop

The runtime architecture above describes *what* a Stage III system contains; this subsection describes *how* a practitioner drives it to produce a deliverable. We use the term *loop engineering* for the practice of designing the closed loop that an office runtime executes, rather than authoring a single prompt that an operator issues turn by turn (Osmani, 2025). The shift is from writing instructions to designing the environment, stopping rule, and feedback signals under which an agent iterates until the work is genuinely done. This is the deployment-side expression of the generation-and-verification loop formalized in §2.2: the operator stops being the entity that re-prompts after each step and instead specifies the goal once, as a *recursive objective* over the committed artifact, and lets the harness iterate observe $\Omega(d_t)$, plan, commit d_{t+1}, and verify $\mathcal{V}$ until a stopping condition is met. Conceptually this is the ReAct interleaving of reasoning and action (Yao et al., 2023) wrapped in an explicit completion criterion and budget.

Completion is defined by the verifier, not by self-report. The central design choice in an office loop is the stopping condition. A loop that halts when the model declares itself finished inherits the failure mode practitioners call the *Ralph Wiggum* loop: the agent emits a completion marker while the deliverable is still broken, so the loop exits on a half-finished artifact with no error raised (Huntley, 2025). The deliverable-centric framing supplies the antidote directly: completion must be defined by the verification operator $\mathcal{V}$ of §2.2, not by a free-form claim of success. This makes the loop's stopping rule the same object as the evaluation stack of §4. Its deterministic core is cheap and reliable for explicit constraints: formula re-execution checks that a spreadsheet loop's outputs recompute rather than freeze into pasted constants, OOXML schema validation and diff inspection check structural validity, and render checks catch overflow or element collision before a slide loop halts (Ma et al., 2024b; Li et al., 2023a; Pan et al., 2026). But no single check certifies all six deliverable dimensions of §4.2: a deck whose slides each pass content checks can still be incoherent at deck level (Guo et al., 2024), and a report that answers every factual query can still hallucinate a citation or break a cross-file number (Zou et al., 2025; Wang et al., 2024d). A trustworthy stopping rule is therefore a *composed* verifier: deterministic checks for structural and content validity, rubric-based and LLM-judge scoring for presentational and communicative quality, and an Agent-as-a-Judge that opens the files and inspects rendered and cross-file

state for the dimensions code cannot reach (Zheng et al., 2025; Liu et al., 2023; Zhuge et al., 2024). The caution from §4.3 transfers intact: an LLM-judge stopping condition can itself be gamed, since verbosity, position, and self-consistency biases let a loop "pass" by producing longer or more confident output instead of a better artifact, so the judge must be calibrated against human acceptance and anchored by deterministic checks it cannot talk its way past (Chen et al., 2024). Because the harder dimensions remain subjective (§4.2), a purely closed loop is appropriate only for the verifiable dimensions; the rest keep a human or a calibrated agentic verifier in the stopping rule, not removing review entirely.

Inner and outer loops map onto harness and memory. Two nested loops carry distinct responsibilities. The *inner loop* raises single-deliverable reliability within one task: it is the plan–commit–verify–repair cycle of §3.5, whose reliability depends on whether the agent actually invokes artifact-aware validators instead of asserting success. The *outer loop* raises competence across sessions: failures are distilled into durable artifacts such as revised skills, memory entries, and lessons files, so a later task starts from accumulated procedure instead of rediscovering it. This outer loop is the deployment instance of Reflexion's verbal self-correction (Shinn et al., 2023) and of the skill-evolution discipline in §6.3: the agent forgets between runs, but the repository does not, so organizational procedure persists in skills, templates, and validators, not in any single context window.

Loop components are the runtime components, composed. The mechanisms a loop needs are precisely the runtime ingredients already discussed, assembled into an autonomous cycle: an automation trigger (schedule, watch, or webhook) supplies the heartbeat that starts each iteration; *skills* preload project conventions so the agent need not re-infer them each run (§6.3); connectors expose real tools through MCP and inter-agent capabilities through A2A so the loop can read a ticket, edit the package, and route it for approval rather than work in a sandbox (Anthropic, 2024b; Google Developers Blog, 2025); a generator/critic split assigns drafting and adversarial review to separate sub-agents, reaching quality a single self-judging agent does not (Microsoft 365 Blog, 2025b); isolated workspaces (for example git worktrees) let independent artifacts in a package be produced in parallel without colliding; and a persistent state file, the backbone of the system, records what has been attempted, what passed $\mathcal{V}$, and what remains, so a crashed or scheduled run resumes from disk, not from memory. A minimal office loop is therefore small: a fixed specification, a heartbeat, a state file, and one verifier command; worktree parallelism, generator/critic review, and unattended scheduling are added only once the minimal loop is reliable.

Model-authored orchestration: the loop writes itself. The preceding discussion assumes a human practitioner designs the loop. A recent development shifts this assumption: the model itself can generate the orchestration logic as executable code, which a runtime then executes deterministically. Claude Code's *Dynamic Workflows* feature exemplifies this pattern (Anthropic, 2026b): a user describes a goal in natural language, the model produces a JavaScript script that schedules tens to hundreds of sub-agents across phases, and the runtime executes the script in the background while the session remains responsive. The script, not the model's turn-by-turn judgment, holds the plan, the branching logic, and the intermediate results; the model's context retains only the final output.

This matters for AI Office in three ways. First, the model can compose exactly the loop components listed above, including generator/critic splits, isolated workspaces for parallel artifacts, adversarial cross-checks, and persistent state, into a task-specific orchestration without the practitioner hand-wiring each one. For the quarterly-review scenario of §2.1, a dynamic workflow could fan out agents to analyze spreadsheet data, draft the document narrative, generate slides, and then dispatch independent verifier agents to check that every number cited in the deck matches the source workbook, closing the cross-artifact consistency gap that manual coordination leaves open. Second, because the orchestration is externalized as a reusable script, it can be saved, versioned, and invoked as a command in future sessions, effectively compiling a one-off delegation into a repeatable organizational skill (§6.3). Third, the adversarial patterns the script can encode, such as having independent agents attempt to refute each other's findings before reporting, directly implement the composed verification protocols of §4.3 at orchestration time rather than as a post-hoc evaluation pass.

Model-authored orchestration thus represents the next step in the model–harness co-evolution cycle of §2.4: the model no longer merely executes within a harness designed by a human; it dynamically generates the harness code for a specific task, and the runtime provides the execution guarantees (concurrency caps, token budgets, resumability, permission gates) that make unattended multi-agent work safe. The same governance constraints discussed below, including stopping rules, comprehension-debt limits, and least-privilege credentials,

apply to model-authored loops with equal or greater force, since the orchestration itself is now model-generated and must be inspectable before it runs.

A procedure for constructing an office loop. The preceding components yield a concrete build order, not a generic checklist. First, decide whether the task warrants a loop at all: an office task is a good loop candidate only when it is repeatable, when "done" admits a machine-checkable definition, and when its specification is stable enough that the goal need not be renegotiated each iteration. The deliverable-centric framing makes these requirements precise, since a goal g whose completion cannot be expressed through $\mathcal{V}$ is not yet loop-ready and should stay an interactive copilot session (§2.1). Second, write the stopping rule before the prompt: fix the verifier command(s) that decide completion, drawing the deterministic core from the artifact's native checks (formula recomputation, schema validation, render inspection) and adding rubric, judge, or agentic verifiers for the dimensions those checks miss (§4.3). Third, externalize state and conventions: a state file records progress against $\mathcal{V}$ so a scheduled or crashed run resumes from disk, and a skill file encodes the project's procedures and capability limits so each iteration need not re-infer them (§6.3). Fourth, run the closed loop under an explicit iteration and token budget, repairing from the verifier's concrete failure traces instead of from intuition. This is the same trace-to-skill discipline used for skill evolution and training-data curation in §3.5 and §5. Fifth, scale only after the single-deliverable loop is reliable: split drafting and review into generator/critic sub-agents, isolate parallel artifacts of a package in separate worktrees, and add an unattended trigger. Throughout, evaluation is not a final gate but the loop's inner signal: the same $\mathcal{V}$ that scores the benchmark scores every iteration, so the office benchmarks of §4 (SpreadsheetBench, PPTEval, DocBench, OfficeBench, OdysseyBench) double as templates for the stopping rules a deployed loop should embed (Ma et al., 2024b; Zheng et al., 2025; Zou et al., 2025; Wang et al., 2024d, 2025).

Loop engineering is also a governance surface. An unattended loop that edits real files and calls real connectors is an unsupervised attack and error surface, which makes the loop's guardrails part of its design, not an afterthought. Three risks recur. *Silent failure*, addressed above, is contained by objective stopping conditions and an independent verifier. *Comprehension debt* arises when artifacts are committed faster than any human reviews them, so loop output granularity and human review of merged deliverables must be bounded deliberately. This is an acute concern given evidence that delegated editing can silently corrupt documents (Laban et al., 2026), and one that mirrors the shift toward supervisory engineering reported under agentic code generation, where reviewer bandwidth, not generation throughput, becomes the binding constraint (Vella & Blincoe, 2026; Adams et al., 2026). *Unbounded authority* is the standing cost of automation: least-privilege credentials, on-demand connector scopes, mandatory human confirmation for destructive or externally visible actions (deletion, schema changes, outbound sends), audit logging, and budget and timeout cutoffs are fixed costs of running an office loop, not optional extras. These requirements echo the governance theme of §6.6: execution can be delegated to the loop, but accountability for the committed workspace cannot, and the net benefit of a loop is realized only when time saved in execution exceeds time spent on verification, rework, and oversight (OpenAI, 2026b).

6.5. Deployment Evidence for the Shifting Architecture

The deployment signals above corroborate the shift from application-local logic to model-harness orchestration formalized in §2.4: the vertical unbundling of application logic into a model-driven orchestration layer, and the horizontal softening of boundaries between documents, spreadsheets, and slides. Two product-level observations reinforce that account. First, the *model-harness co-evolution cycle* is visible in shipped systems: capabilities once hand-engineered in the harness (slide-layout heuristics, formula-selection logic) are increasingly absorbed into the model through training, as in learned slide-layout rewards and spreadsheet RL (Z.AI, 2026; Pan et al., 2026; Chi et al., 2026), while the harness moves up to managing multi-artifact consistency, organizational policy, and trust (Microsoft Learn, 2026a). Second, the same architecture turns local work practices into executable assets: recurring report structures, metric definitions, approval routines, and visual standards are encoded as skills, templates, memory, retrieval rules, and validators. That accumulation is productive only if it remains inspectable and governable: as the harness learns how an organization works, ownership, correction, access control, and responsibility become technical design requirements rather than downstream policy concerns. Deployment reports are consistent with this direction, describing demand around knowledge-artifact production, data analysis, and research, but they should be read as contextual evidence for where the runtime is used, not as proof that it satisfies the deliverable guarantees proposed in this paper (OpenAI, 2026b).

6.6. Takeaways

The product landscape is converging toward a single direction: office AI is becoming a runtime—not a better sidebar—that can own procedures end-to-end while remaining auditable.

- **Autonomy is climbing.** Stage I features and Stage II copilots still help, but the strongest product push is toward Stage III systems that hold context, execute multi-step procedures, and self-verify (§6.2).
- **Product features *are* the training/eval interface.** Skills, workflow templates, approval traces, and admin controls are not bolted-on governance; they are exactly how these agents learn local procedures and stay auditable (§6.3).
- **Apps get thinner; the harness gets thicker.** Traditional office apps are not going away, but their role shifts to data models, rendering, and permissions. Intent, memory, policy, and cross-file consistency move to the agentic layer (§6.5).

7. Toward Goal-Driven AI Office

The three preceding sections each traced one facet of the evolution from passive assistance to goal-driven agency. Operation (§3) showed that agents can now invoke code, APIs, and GUI across artifact types; evaluation (§4) showed that no single protocol reliably certifies the result across all quality dimensions; and training (§5) showed that reward signals remain sparse and often misaligned with what users actually accept. Together, these gaps define what remains before dependable goal-driven office agents can be realized. The deliverable-centric framing turns them into concrete open problems organized along two fronts: how to reliably judge and train a single deliverable, and how to scale to packages of artifacts and the systems that produce them.

Judging and training the deliverable. Three obstacles compound here. First, the six evaluation dimensions identified in §4.2 (structural validity, content correctness, presentational quality, usability, provenance, and package consistency) span a spectrum from automatable to deeply subjective; vague office quality ("professional," "audit-friendly") can become a useful training signal only when it is operationalized in constrained, auditable ways rather than collapsed into shallow proxies. AeSlides operationalizes layout aesthetics into verifiable rewards (Pan et al., 2026), but extending this to faithfulness, communicative effectiveness, and organizational fit is open. Second, because rule-based, rubric-based, LLM-judge, and agentic protocols each have failure modes in isolation (§4.3), a practical evaluation stack needs to compose them, but calibrating the composed stack against human acceptance, and auditing it for inherited biases, remains an unsolved systems problem (Zhuge et al., 2024; Chen et al., 2024). Third, even when individual reward signals work, multi-stage training introduces regression: RLVR can optimize verifiable layout constraints while degrading factual accuracy, and RLEF can improve task completion while losing fine-grained quality (§5.4). Cross-stage distillation (Team et al., 2026a) and composite rewards are partial answers, but robustly improving all dimensions of $\mathcal{V}$ simultaneously is unresolved.

Scaling to packages and systems. Real deliverables bundle workbook, deck, document, and email whose numbers and claims must agree, requiring a *cross-artifact consistency* that single-file checks miss and the surveyed public reward setups only partially address (§4.2; OfficeBench-style suites are a start, not a solution Wang et al., 2024d, 2025). Recent benchmarks underscore this challenge: Workspace-Bench requires agents to handle large-scale file dependencies across heterogeneous workspace artifacts (Tang et al., 2026), and BankerToolBench demands consistent multi-file deliverable packages against expert rubrics (Lau et al., 2026), yet package-level consistency and workspace dependencies remain unsolved. Achieving it points to a runtime whose full requirements have not yet been demonstrated by publicly documented general harnesses: an *office-specialized harness* (§3.5) with artifact-aware validators that open files and check rendered state, version-aware memory that tracks how artifacts co-evolve, and adaptation slots for local conventions. Within such a harness, hand-authored, single-artifact *skills* (Anthropic, 2025e; MiniMax, 2026) must be composed across artifacts via MCP and A2A (Anthropic, 2024b; Google Developers Blog, 2025), tested against artifact-level outcomes, and eventually revised or learned from successful trajectories (Ni et al., 2026; Ouyang et al., 2026). Multi-agent training (§5.5) adds a credit-assignment challenge: when a package is inconsistent, distinguishing faults in planning, execution, and coordination requires reward signals richer than those available in the surveyed benchmark set. Learning organization-specific standards (adapting to local formatting, evidence conventions, and approval workflows (Microsoft Learn, 2026b)) without eroding general capability mirrors the regression challenge of multi-stage RL but at deployment level.

Outlook. The challenge thus moves from response correctness to deliverable acceptance, and from single artifacts to orchestrated teams trained against organizational standards, measured less by what models say than by the workspaces they leave behind. Mapped onto the autonomy spectrum of §2.1, these challenges trace the gap between today's Stage II copilots, which propose edits for human review, and the Stage III vision in which the agent is accountable for the entire committed workspace. Closing this gap requires progress on all three dimensions simultaneously: operation surfaces must support richer cross-artifact orchestration, evaluation protocols must assess all six quality dimensions (including package-level consistency), certifying the verifiable dimensions while calibrating subjective ones, and training pipelines must convert organizational acceptance into stable, multi-dimensional reward signals without regressing on general capability. Among these, we view learnable deliverable quality and cross-artifact consistency as promising near-term targets in constrained settings, since artifact-grounded rewards are already being explored for domains such as spreadsheet formulas and slide layouts (Chi et al., 2026; Pan et al., 2026; Lai et al., 2025); extending them to multi-file packages and subjective quality dimensions such as faithfulness, communicative effectiveness, and audit-friendliness is the harder but higher-impact direction. If these problems see sustained progress, and if verification and governance mature alongside generation, AI Office could move from a collection of impressive but siloed demonstrations toward infrastructure for knowledge work, where agents produce deliverables that organizations can verify, revise, and build upon. Early deployment reports are consistent with this transition: they highlight growing use of agentic tools for knowledge-artifact production, data analysis, and research, not only for software development (OpenAI, 2026b). As the same report cautions, however, the largest gains will come only when organizations redesign processes around what the technology has made cheap, echoing the productivity paradox that followed every previous wave of workplace automation. The practical metric is therefore not gross generation speed but net workflow gain: time saved in execution and coordination must exceed the time spent on verification, rework, approval, governance, and risk management. Otherwise, autonomy merely moves labor from production into audit. At the same time, the workflows that make this gain repeatable encode organizational knowledge into skills, memory, templates, and validators. That makes AI Office a governance problem as well as a capability problem: future systems will need to make authorship, accountability, permission boundaries, and memory control inspectable if organizations are to trust the workspaces agents leave behind. Broader survey evidence is consistent with this concern: among roughly 81,000 people, loss of autonomy and agency (21.9%) and inadequate governance (14.7%) rank among the most common concerns about AI (Anthropic, 2026c). Although this survey does not measure AI Office delegation directly, it reinforces the need for inspectable oversight as a precondition for responsible delegation rather than an afterthought. The research agenda outlined above is one technical complement to that organizational redesign.

Limitations

This survey is intentionally centered on office deliverables: documents (including PDFs), spreadsheets, and slide decks. Email, calendar, code, BI dashboards, CRM forms, databases, and design or modeling tools enter mainly when they shape such deliverables, while adjacent work on GUI/OS agents and visually rich document understanding is covered by complementary surveys (Nguyen et al., 2025; Hu et al., 2025b; Ding et al., 2026; Lu et al., 2025). This boundary is pragmatic rather than absolute; the same deliverable-centric lens may extend to broader enterprise artifacts, but those surfaces are not treated systematically here.

The paper also synthesizes a fast-moving literature rather than reporting a new benchmark or deployment study. Accordingly, the crosswalk in Appendix A is curated rather than exhaustive, and our autonomy-stage framing should be read as an organizing lens rather than a complete taxonomy. Claims about office-specialized harnesses, agentic verification, and organizational adaptation are best viewed as research directions for future empirical work, not as settled limitations of current systems.

References

Chris Adams, Arjun Singh Banga, Parveen Bansal, Souvik Bhattacharya, Rujin Cao, Pedro Canahuati, Nate Cook, Brian Ellis, Prabhakar Goyal, Gurinder Grewal, Tianyu He, Matt Labunka, Alex Manners, David Molnar, Ging Cee Ng, Vishal Parekh, Jiefu Pei, Frederic Sagnes, James Saindon, Will Shackleton, Sid Sidhu, Gursharan Singh, Karthik Chengayan Sridhar, Matt Steiner, Pratibha Udmalpet, Sean Xia, Stacey Yan, Audris Mockus, Peter Rigby, and Nachiappan Nagappan. Automating low-risk code review at Meta: RADAR, risk calibration, and review efficiency. *arXiv preprint arXiv:2605.30208*, 2026. URL https://arxiv.org/abs/2605.30208.

Adobe. Adobe Firefly: Generative AI creative studio. https://firefly.adobe.com/, 2023. Generative AI for image, video, and audio creation, integrated with Photoshop and Creative Cloud.

Adobe. Adobe Acrobat AI Assistant. https://www.adobe.com/acrobat/generative-ai-pdf.html, 2024.

Saaket Agashe, Jiuzhou Han, Shuyu Gan, Jiachen Yang, Ang Li, and Xin Eric Wang. Agent s: An open agentic framework that uses computers like a human. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2024.

Airtable. Airtable Omni: AI-native app builder and assistant. https://www.airtable.com/newsroom/introducing-the-ai-native-airtable, 2025. Conversational AI that builds apps, edits data, and runs operations inside Airtable bases.

Alibaba Cloud. Tongyi Wanxiang: AI image and video generation. https://tongyi.aliyun.com/wan/, 2023a. Alibaba Cloud's multimodal generative model for text/image-to-image and video creation.

Alibaba Cloud. Tongyi Tingwu: AI work and study assistant. https://tingwu.aliyun.com/, 2023b. AI audio/video assistant for transcription, speaker separation, summarization, and translation.

Alibaba Group. Quark: AI super assistant application. https://www.quark.cn, 2025. Alibaba's AI super-app integrating search, deep research, writing, and task execution on the Tongyi model.

Alibaba Group. Qwen App and Quark: Consumer AI assistants. https://qwen.ai/; https://www.quark.cn/, 2026a. Official Qwen and Quark websites; Alibaba Group's 2026 announcement describes Qwen App's agentic capabilities.

Alibaba Group. Wukong: An AI-native agentic platform for enterprises. https://www.alibabagroup.com/en-US/document-1971078136456019968, 2026b. Multi-agent office automation with enterprise sandboxes, role-based access control, and a modular skill registry.

Karthik Ragunath Ananda Kumar and Subrahmanyam Arunachalam. Learning to present: Inverse specification rewards for agentic slide generation. *arXiv preprint arXiv:2603.16839*, 2026.

Anthropic. Introducing computer use, a new claude 3.5 sonnet, and claude 3.5 haiku, October 2024a. URL https://www.anthropic.com/news/3-5-models-and-computer-use/. Published: 2024-10-22.

Anthropic. Introducing the model context protocol, November 2024b. URL https://www.anthropic.com/news/model-context-protocol. Published: 2024-11-25.

Anthropic. Introducing Claude 4. https://www.anthropic.com/news/claude-4, May 2025a. Published: 2025-05-22.

Anthropic. Claude Code: An agentic command-line tool. https://www.anthropic.com/claude-code, 2025b. Anthropic product page.

Anthropic. Claude Code on desktop: Desktop application. https://code.claude.com/docs/en/desktop, 2025c. Official Claude Code documentation for the Claude Desktop Code tab.

Anthropic. Equipping agents for the real world with Agent Skills. https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills, 2025d. Anthropic engineering blog.

Anthropic. Agent Skills: Public skill library for Claude. https://github.com/anthropics/skills, 2025e. Open- and source-available skill folders, including the DOCX, PPTX, XLSX, and PDF document skills that power Claude's document capabilities.

Anthropic. Claude Cowork: Agentic AI for knowledge work. https://www.anthropic.com/product/claude-cowork, 2026a. Desktop agent that reads, edits, and creates files in user-specified folders; general availability 2026.

Anthropic. Dynamic workflows: Orchestrate subagents at scale. https://code.claude.com/docs/en/workflows, 2026b. Claude Code documentation, v2.1.154+.

Anthropic. What 81,000 people want from AI, March 2026c. URL https://www.anthropic.com/features/81k-interviews. Survey of 80,508 respondents across 159 countries and 70 languages.

Anthropic. Introducing Claude Design by Anthropic Labs. https://www.anthropic.com/news/claude-design-anthropic-labs, 2026d. AI tool that generates and refines designs, prototypes, slides, and one-pagers.

Autodesk. Forma Site Design: Autodesk AI-powered cloud software for data-driven planning. https://www.autodesk.com/products/forma/overview, 2023. Official Autodesk product page for AI-powered cloud software used in early-stage architecture and planning.

Autodesk. Autodesk Fusion: AI and automation for CAD/CAM. https://www.autodesk.com/products/fusion-360/ai-automation, 2024. AI features in Fusion including the Autodesk Assistant, AutoConstrain, automated drawings, and generative design.

Rahul Suresh Babu and Adarsh Agrawal. Self-healing agentic orchestrators for reliable tool-augmented large language model systems. *arXiv preprint arXiv:2606.01416*, 2026.

Hao Bai, Yifei Zhou, Jiayi Pan, Mert Cemri, Alane Suhr, Sergey Levine, and Aviral Kumar. Digirl: Training in-the-wild device-control agents with autonomous reinforcement learning. *NeurIPS*, 2024a.

Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Ming-Hsuan Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. Qwen2.5-vl technical report. *arXiv (Cornell University)*, 2025.

Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand, 2024b. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.172. URL https://aclanthology.org/2024.acl-long.172/.

Baidu. Baidu Wenku: One-stop AI content creation platform. https://wenku.baidu.com/, 2023. AI-powered document platform that generates slides, mind maps, and reports alongside its document repository.

Lisanne Bainbridge. Ironies of automation. *Automatica*, 19(6):775–779, 1983. doi: 10.1016/0005-1098(83)90046-8. URL https://doi.org/10.1016/0005-1098(83)90046-8.

Beautiful.ai. Beautiful.ai — presentation software with AI. https://www.beautiful.ai/, 2023.

Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, Erik Brynjolfsson, Shyamal Buch, Dallas Card, Rodrigo Castellon, Niladri Chatterji, Annie Chen, Kathleen Creel, Jared Quincy Davis, Dora Demszky, Chris Donahue, Moussa Doumbouya, Esin Durmus, Stefano Ermon, John Etchemendy, Kawin Ethayarajh, Li Fei-Fei, Chelsea Finn, Trevor Gale, Lauren Gillespie, Karan Goel, Noah Goodman, Shelby Grossman, Neel Guha, Tatsunori Hashimoto, Peter Henderson, John Hewitt, Daniel E. Ho, Jenny Hong, Kyle Hsu, Jing Huang, Thomas Icard, Saahil Jain, Dan Jurafsky, Pratyusha Kalluri, Siddharth Karamcheti, Geoff Keeling, Fereshte Khani, Omar Khattab, Pang Wei Koh, Mark Krass, Ranjay Krishna, Rohith Kuditipudi, Ananya Kumar, Faisal Ladhak, Mina Lee, Tony Lee, Jure Leskovec, Isabelle Levent, Xiang Lisa Li, Xuechen Li, Tengyu Ma, Ali Malik,

Christopher D. Manning, Suvir Mirchandani, Eric Mitchell, Zanele Munyikwa, Suraj Nair, Avanika Narayan, Deepak Narayanan, Ben Newman, Allen Nie, Juan Carlos Niebles, Hamed Nilforoshan, Julian Nyarko, Giray Ogut, Laurel Orr, Isabel Papadimitriou, Joon Sung Park, Chris Piech, Eva Portelance, Christopher Potts, Aditi Raghunathan, Rob Reich, Hongyu Ren, Frieda Rong, Yusuf Roohani, Camilo Ruiz, Jack Ryan, Christopher Ré, Dorsa Sadigh, Shiori Sagawa, Keshav Santhanam, Andy Shih, Krishnan Srinivasan, Alex Tamkin, Rohan Taori, Armin W. Thomas, Florian Tramèr, Rose E. Wang, William Wang, Bohan Wu, Jiajun Wu, Yuhuai Wu, Sang Michael Xie, Michihiro Yasunaga, Jiaxuan You, Matei Zaharia, Michael Zhang, Tianyi Zhang, Xikun Zhang, Yuhui Zhang, Lucia Zheng, Kaitlyn Zhou, and Percy Liang. On the opportunities and risks of foundation models. *arXiv (Cornell University)*, 2022.

Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Bucker, Lawrence Jang, and Zack Hui. WindowsAgentArena: Evaluating multi-modal OS agents at scale. *arXiv preprint arXiv:2409.08264*, 2024. URL https://arxiv.org/abs/2409.08264.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020. URL https://arxiv.org/abs/2005.14165.

Erik Brynjolfsson. The productivity paradox of information technology. *Communications of the ACM*, 36(12): 66–77, 1993. URL https://doi.org/10.1145/163298.163309.

Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, Harsha Nori, Hamid Palangi, Marco Tulio Ribeiro, and Yi Zhang. Sparks of artificial general intelligence: Early experiments with GPT-4. *arXiv preprint arXiv:2303.12712*, 2023. URL https://arxiv.org/abs/2303.12712.

J Bughin, M Chui, and J Manyika. Capturing business value with social technologies. Technical report, McKinsey & Company, 2012. URL https://www.mckinsey.com/industries/technology-media-and-telecommunications/our-insights/capturing-business-value-with-social-technologies.

ByteDance. Doubao: ByteDance's AI assistant. https://www.doubao.com/, 2023. Multimodal AI assistant for chat, writing, image generation, voice interaction, and document analysis.

ByteDance. CapCut: AI video editor. https://www.capcut.com/, 2024a. AI-powered video editor offering auto-captions, script-to-video, text-to-speech, and AI generation.

ByteDance. Jimeng AI: AI image and video creation. https://jimeng.jianying.com/, 2024b. ByteDance Jianying/CapCut team's one-stop AI platform for text-to-image and image-to-video generation.

Canva. Magic Design: AI-powered design generation. https://www.canva.com/magic-design/, 2025.

Dongping Chen, Ruoxi Chen, Shilin Zhang, Yinuo Liu, Yaochen Wang, Huichi Zhou, Qihui Zhang, Yao Wan, Pan Zhou, and Lichao Sun. MLLM-as-a-judge: Assessing multimodal LLM-as-a-judge with vision-language benchmark. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. doi: 10.48550/arXiv.2402.04788. URL https://arxiv.org/abs/2402.04788.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew,

Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021a. doi: 10.48550/arXiv.2107.03374. URL https://arxiv.org/abs/2107.03374.

Minghao Chen, Xinyi Hu, Zhou Yu, and Yufei Yin. Autorpa: Efficient gui automation through llm-driven code synthesis from interactions. *arXiv preprint arXiv:2605.21082*, 2026.

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*, 2022.

Xinyun Chen, Petros Maniatis, Rishabh Singh, Charles Sutton, Hanjun Dai, Max Lin, and Denny Zhou. Spreadsheetcoder: Formula prediction from semi-structured context. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pp. 1661–1672, 2021b. URL https://proceedings.mlr.press/v139/chen21m.html.

Yibin Chen, Yifu Yuan, Zeyu Zhang, Yan Zheng, Jinyi Liu, Fei Ni, Jianye Hao, Hangyu Mao, and Fuzheng Zhang. SheetAgent: Towards a generalist agent for spreadsheet reasoning and manipulation via large language models. In *Proceedings of the ACM Web Conference 2025 (WWW '25)*. ACM, 2025. doi: 10.1145/3696410.3714962. URL https://doi.org/10.1145/3696410.3714962.

Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing GUI grounding for advanced visual GUI agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 9313–9332, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.505. URL https://aclanthology.org/2024.acl-long.505/.

Zhoujun Cheng, Tianbao Xie, Peng Shi, Chengzu Li, Rahul Nadkarni, Yushi Hu, Caiming Xiong, Dragomir R. Radev, M. Ostendorf, Luke S. Zettlemoyer, N. A. Smith, and Tao Yu. Binding language models in symbolic languages. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2022.

Banghao Chi, Yining Xie, Mingyuan Wu, Jingcheng Yang, Jize Jiang, Zhaoheng Li, Shengyi Qian, Minjia Zhang, Klara Nahrstedt, Rui Hou, Xiangjun Fan, and Hanchao Yu. Spreadsheet-rl: Advancing large language model agents on realistic spreadsheet tasks via reinforcement learning. *arXiv (Cornell University)*, 2026.

P. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, S. Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Cleo AI. Cleo: AI personal finance assistant. https://web.meetcleo.com/, 2023. Conversational AI assistant for budgeting, spending tracking, savings automation, and bill negotiation.

Cognition. Introducing Devin, the first AI software engineer. https://cognition.com/blog/introducing-devin, 2024. Autonomous software-engineering agent that plans, writes, tests, and ships code.

Cursor. Cursor: The AI code editor. https://cursor.com/, 2025. Anysphere product page.

Dassault Systèmes. SOLIDWORKS 2026: AI-powered 3d design. https://www.3ds.com/newsroom/media-alerts/dassault-systemes-announces-solidworks-2026-ai-powered-design-and-collaboration-generative-economy, 2026. AI-powered SOLIDWORKS release with generative drawing automation and the LEO virtual design assistant.

DataCamp. DataLab: AI-powered data notebook. https://www.datacamp.com/datalab, 2024. AI-enabled cloud data notebook for chatting with data and running Python/R/SQL analysis.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36:28091–28114, 2023.

Descript. Descript: AI video and podcast editor. https://www.descript.com/, 2024. Edits video and audio by editing text, with AI transcription, filler-word removal, and an AI assistant.

Yihao Ding, Siwen Luo, Yue Dai, Yanbei Jiang, Zechuan Li, Qiang Sun, Geoffrey Martin, Wei Liu, and Yifan Peng. A survey on mllm-based visually rich document understanding: Methods, challenges, and emerging trends, 2026. URL https://arxiv.org/abs/2507.09861.

Haoyu Dong, Jianbo Zhao, Yuzhang Tian, Junyu Xiong, Mengyu Zhou, Yun Lin, José Cambronero, Yeye He, Shi Han, and Dongmei Zhang. Encoding spreadsheets for large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 20728–20748, Miami, Florida, USA, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.1154. URL https://aclanthology.org/2024.emnlp-main.1154/.

Haoyu Dong, Pengkun Zhang, Yan Gao, Xuanyu Dong, Yilin Cheng, Mingzhe Lu, Adina Yakefu, and Shuxin Zheng. Finch: Benchmarking finance & accounting across spreadsheet-centric enterprise workflows. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 10768–10794, 2026.

Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. WorkArena: How capable are web agents at solving common knowledge work tasks? In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pp. 11642–11662, 2024. URL https://proceedings.mlr.press/v235/drouin24a.html.

ElevenLabs. ElevenLabs: AI voice generator and voice agents platform. https://elevenlabs.io/, 2023. AI text-to-speech, voice cloning, and conversational voice agents across many languages.

Tyna Eloundou, Sam Manning, Pamela Mishkin, and Daniel Rock. GPTs are GPTs: An early look at the labor market impact potential of large language models. *arXiv preprint arXiv:2303.10130*, 2023. doi: 10.48550/arXiv.2303.10130. URL https://arxiv.org/abs/2303.10130.

Weixi Feng, Wanrong Zhu, Tsu-Jui Fu, Varun Jampani, Arjun R. Akula, Xuehai He, Sugato Basu, Xin Eric Wang, and William Yang Wang. Layoutgpt: Compositional visual planning and generation with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Figma. Figma Make: Prompt-to-app and prototype generation. https://www.figma.com/blog/figma-make-general-availability/, 2025. AI tool that turns prompts and designs into interactive, high-fidelity prototypes and apps.

FinchatAI. FinchatAI: Chatgpt for personal finance. https://www.finchatai.com/, 2023. Personal-finance assistant.

Fireflies.ai. Fireflies.ai: AI meeting notetaker. https://fireflies.ai/, 2023. Auto-joins calls to record, transcribe, and summarize meetings across major conferencing platforms.

Fiscal.ai. Fiscal.ai: AI-powered financial data platform. https://fiscal.ai/, 2025. Conversational AI and APIs for institutional-grade financial data and investment research; formerly FinChat.

Flourish. Flourish: Interactive data visualization. https://flourish.studio/, 2022. Browser-based platform (acquired by Canva) for interactive charts, maps, and data storytelling.

Framer. Framer AI: AI website builder. https://www.framer.com/ai/, 2023. Generates responsive websites from prompts with AI agents for refinement.

Tsu-Jui Fu, William Yang Wang, Daniel McDuff, and Yale Song. DOC2PPT: Automatic presentation slides generation from scientific documents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pp. 634–642, 2022. doi: 10.1609/aaai.v36i1.19943. URL https://ojs.aaai.org/index.php/AAAI/article/view/19943.

Gamma. Gamma — a new medium for presenting ideas, powered by AI. https://gamma.app/, 2023.

Gamma. Gamma: AI-powered presentations. https://gamma.app/, 2025.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning*, 2023. URL https://proceedings.mlr.press/v202/gao23f.html.

Jiaxin Ge, Zora Zhiruo Wang, Xuhui Zhou, Yi-Hao Peng, Sanjay Subramanian, Qinyue Tan, Maarten Sap, Alane Suhr, Daniel Fried, Graham Neubig, et al. Autopresent: Designing structured visuals from scratch. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 2902–2911, 2025.

Genspark. Genspark: All-in-one AI workspace and super agent. https://www.genspark.ai/, 2025. AI super agent that turns prompts into actions using a mixture-of-agents system for research, slides, and content.

Google. Supercharge productivity and creativity with Duet AI in Google Workspace. https://workspace.google.com/blog/product-announcements/duet-ai-in-workspace-now-available, 2023a. Duet AI for Google Workspace was rebranded to Gemini for Google Workspace on February 8, 2024 (Sundar Pichai blog), with the Gemini Enterprise/Business add-on rolled out February 21, 2024.

Google. NotebookLM: AI-powered research and note-taking tool. https://notebooklm.google.com/, 2023b. AI notebook grounded in user-uploaded sources that summarizes, answers questions, and generates audio overviews.

Google. Google Workspace: Help me write in Docs. https://workspace.google.com/solutions/ai/, 2025.

Google DeepMind. Gemini 2.5: Our most intelligent AI model, March 2025. URL https://blog.google/innovation-and-ai/models-and-research/google-deepmind/gemini-model-thinking-updates-march-2025/. Published: 2025-03-25; updated: 2025-03-26.

Google Developers Blog. Announcing the agent2agent protocol (a2a), April 2025. URL https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/. Published: 2025-04-09.

Google Workspace. New ai capabilities in workspace to help every business drive results, April 2025a. URL https://workspace.google.com/blog/product-announcements/new-ai-drives-business-results. Published: 2025-04-10.

Google Workspace. Introducing google workspace studio: Automate everyday work with ai agents, December 2025b. URL https://workspace.google.com/blog/product-announcements/introducing-google-workspace-studio-agents-for-everyday-work. Published: 2025-12-04.

Grammarly. Grammarly: Free AI writing assistance. https://www.grammarly.com/, 2024. Official Grammarly homepage.

Granola. Granola: The AI notepad for back-to-back meetings. https://www.granola.ai/, 2023. AI meeting notepad that transcribes in the background and combines user notes with audio into enhanced notes.

Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pp. 317–330, 2011.

Sumit Gulwani and Mark Marron. Nlyze: Interactive programming by natural language for spreadsheet data analysis and manipulation. In *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*, pp. 803–814, 2014.

Yiduo Guo, Zekai Zhang, Yaobo Liang, Dongyan Zhao, and Nan Duan. Pptc benchmark: Evaluating large language models for powerpoint task completion. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 8682–8701, 2024.

Kunal Handa, Alex Tamkin, Miles McCain, Saffron Huang, Esin Durmus, Sarah Heck, Jared Mueller, Jerry Hong, Stuart Ritchie, Tim Belonax, Kevin K. Troy, Dario Amodei, Jared Kaplan, Jack Clark, and Deep Ganguli. Which economic tasks are performed with AI? evidence from millions of Claude conversations. *arXiv preprint arXiv:2503.04761*, 2025. doi: 10.48550/arXiv.2503.04761. URL https://arxiv.org/abs/2503.04761.

Harvey. Harvey: AI for legal and professional services. https://www.harvey.ai/, 2023. Generative AI platform for law firms and corporate legal teams: research, drafting, and workflow automation.

Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. WebVoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 6864–6890, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.371. URL https://aclanthology.org/2024.acl-long.371/.

Jonathan Herzig, Paweł Krzysztof Nowak, Thomas Müller, Francesco Piccinno, and Julian Martin Eisenschlos. TaPas: Weakly supervised table parsing via pre-training. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 4320–4333. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-main.398. URL https://aclanthology.org/2020.acl-main.398/.

Hex Technologies. Hex Magic: AI for data notebooks. https://hex.tech/blog/magic-ai/, 2023. AI assistant in Hex notebooks that generates SQL, Python, and charts from natural language.

HeyGen. HeyGen: AI video generation from documents and slides. https://www.heygen.com/, 2025. Converts PDF/PPT content into narrated video.

Wenyi Hong, Weihan Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, and Jie Tang. Cogagent: A visual language model for gui agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

Anwen Hu, Haiyang Xu, Jiabo Ye, Ming Yan, Liang Zhang, Bo Zhang, Ji Zhang, Qin Jin, Fei Huang, and Jingren Zhou. mplug-docowl 1.5: Unified structure learning for ocr-free document understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 3096–3120, 2024.

Anwen Hu, Haiyang Xu, Liang Zhang, Jiabo Ye, Ming Yan, Ji Zhang, Qin Jin, Fei Huang, and Jingren Zhou. mplug-docowl2: High-resolution compressing for ocr-free multi-page document understanding. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 5817–5834, 2025a.

Xueyu Hu, Tao Xiong, Biao Yi, Zishu Wei, Ruixuan Xiao, Yurun Chen, Jiasheng Ye, Meiling Tao, Xiangxin Zhou, Ziyu Zhao, Yuhuai Li, Shengze Xu, Shenzhi Wang, Xinchen Xu, Shuofei Qiao, Zhaokai Wang, Kun Kuang, Tieyong Zeng, Liang Wang, Jiwei Li, Yuchen Eleanor Jiang, Wangchunshu Zhou, Guoyin Wang, Keting Yin, Zhou Zhao, Hongxia Yang, Fan Wu, Shengyu Zhang, and Fei Wu. OS agents: A survey on MLLM-based agents for computer, phone and browser use. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 7436–7465, Vienna, Austria, 2025b. Association for Computational Linguistics. URL https://aclanthology.org/2025.acl-long.369/.

Yiming Huang, Jianwen Luo, Yan Yu, Yitong Zhang, Fangyu Lei, Yifan Wei, Shizhu He, Lifu Huang, Xiao Liu, Jun Zhao, and Kang Liu. Da-code: Agent data science code generation benchmark for large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 13487–13521. Association for Computational Linguistics, 2024. URL https://aclanthology.org/2024.emnlp-main.748/.

Yupan Huang, Tengchao Lv, Lei Cui, Yutong Lu, and Furu Wei. LayoutLMv3: Pre-training for document AI with unified text and image masking. In *Proceedings of the 30th ACM International Conference on Multimedia (MM '22)*, pp. 4083–4091, Lisboa, Portugal, 2022. ACM. doi: 10.1145/3503161.3548112. URL https://doi.org/10.1145/3503161.3548112.

Geoffrey Huntley. The ralph Wiggum technique: A deterministically naive agentic loop. https://github.com/ghuntley/how-to-ralph-wiggum, 2025. Practitioner write-up of a single-prompt agentic loop and its silent-failure mode.

Ideogram. Ideogram: AI image generation with text rendering. https://ideogram.ai/, 2023. Text-to-image model noted for accurate in-image typography.

Ironclad. Ironclad AI: Contract lifecycle management and Jurist. https://ironcladapp.com/product/ai-based-contract-management, 2023. AI-native CLM platform with the Jurist assistant for redlining, drafting, and contract analysis.

Abubakarr Jaye, Nigel Boachie Kumankumah, Chidera Biringa, Anjel Shaileshbhai Patel, Sulaiman Vesal, Dayquan Julienne, Charlotte Siska, Manuel Raúl Meléndez Luján, Anthony Twum-Barimah, Mauricio Velazco, et al. Corpgen: Simulating corporate environments with autonomous digital employees in multi-horizon task environments. *arXiv preprint arXiv:2602.14229*, 2026.

Jinhao Jiang, Kun Zhou, Zican Dong, Keming Ye, Xin Zhao, and Ji-Rong Wen. Structgpt: A general framework for large language model to reason over structured data. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

Carlos Jimenez, Kilian Lieret, Karthik Narasimhan, Ofir Press, Alexander Wettig, John Yang, and Shunyu Yao. Swe-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024a.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024b. URL https://openreview.net/forum?id=VTF8yNQM66.

Drew Johnston, David Holtz, Alex Martin Richmond, Christopher Ong, Prasanna Tambe, and Aaron Chatterji. The shift to agentic ai: Evidence from codex. *arXiv (Cornell University)*, 2026.

Julius AI. Julius: AI data analyst. https://julius.ai/, 2024. AI tool that analyzes, visualizes, and forecasts data from spreadsheets and files via natural language.

Kyudan Jung, Hojun Cho, Jooyeol Yun, Soyoung Yang, Jaehyeok Jang, and Jaegul Choo. Talk to your slides: High-efficiency slide editing via language-driven structured data manipulation. *arXiv preprint arXiv:2505.11604*, 2025. URL https://arxiv.org/abs/2505.11604.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

Seungone Kim, Jamin Shin, Yejin Cho, Joel Jang, Shayne Longpre, Hwaran Lee, Sangdoo Yun, Seongjin Shin, Sungdong Kim, James Thorne, and Minjoon Seo. Prometheus: Inducing fine-grained evaluation capability in language models. In *The Twelfth International Conference on Learning Representations*, 2024. doi: 10.48550/arXiv.2310.08491. URL https://arxiv.org/abs/2310.08491.

Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024.

Krea AI. Krea: AI creative suite for images, video, and 3d. https://www.krea.ai/, 2024. Real-time AI generation and editing of images, video, and 3D assets across multiple models.

Kuaishou Technology. Kling AI: Next-generation AI video generation. https://klingai.com/, 2024. Kuaishou's generative model for text-to-video and image-to-video creation.

Srivatsa Kundurthy, Clara Na, Colton Moraine, Anoushka Mohta, Case Winter, George Fang, John Ling, Emma Strubell, and Zach Kirshner. BlueFin: Benchmarking LLM agents on financial spreadsheets. *arXiv preprint arXiv:2605.30907*, 2026.

Philippe Laban, Tobias Schnabel, and Jennifer Neville. Llms corrupt your documents when you delegate. *arXiv (Cornell University)*, 2026.

Hanyu Lai, Xiao Liu, Yanxiao Zhao, Han Xu, Hanchen Zhang, Bohao Jing, Yanyu Ren, Shuntian Yao, Yuxiao Dong, and Jie Tang. Computerrl: Scaling end-to-end online reinforcement learning for computer use agents. *arXiv preprint arXiv:2508.14040*, 2025.

Elaine Lau, Markus Dücker, Ronak Chaudhary, Hui Wen Goh, Rosemary Wei, Vaibhav Kumar, Saed Qunbar, Guram Gogia, Yi Liu, Scott Millslagle, Nasim Borazjanizadeh, Ulyana Tkachenko, Samuel Eshun Danquah, Collin Schweiker, Vijay Karumathil, Asrith Devalaraju, Varsha Sandadi, Haemi Nam, Punit Arani, Ray Epps,

Abdullah Arif, Sahil Bhaiwala, Curtis Northcutt, Skyler Wang, Anish Athalye, Jonas Mueller, and Francisco Guzmán. Bankertoolbench: Evaluating ai agents in end-to-end investment banking workflows. *arXiv (Cornell University)*, 2026.

Harrison Lee, Samrat Phatale, Hassan Mansoor, Thomas Mesnard, Johan Ferret, Kellie Lu, Colton Bishop, Ethan Hall, Victor Carbune, Abhinav Rastogi, and Sushant Prakash. Rlaif vs. rlhf: Scaling reinforcement learning from human feedback with ai feedback. *ICML*, 2023.

LegalOn Technologies. LegalOn: Legal AI for contract review. https://www.legalontech.com/, 2023. AI contract review and redlining driven by attorney-built playbooks for in-house legal teams.

LexisNexis. Lexis+ with Protégé: Legal AI assistant. https://www.lexisnexis.com/en-us/products/lexis-plus-protege.page, 2024. Personalized legal AI assistant for research, drafting, and workflow automation grounded in authoritative sources.

Hongxin Li, Jingran Su, Yuntao Chen, Qing Li, and Zhaoxiang Zhang. SheetCopilot: Bringing software productivity to the next level through large language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023a. URL https://openreview.net/forum?id=tfyr2zRVoK.

Jinyang Li, Binyuan Hui, Ge Qu, Binhua Li, Jiaxi Yang, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin Chen-Chuan Chang, Fei Huang, Reynold Cheng, and Yongbin Li. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023b.

Peng Li, Yeye He, Cong Yan, Yue Wang, and Surajit Chaudhuri. Auto-tables: Synthesizing multi-step transformations to relationalize tables without using examples. *Proceedings of the VLDB Endowment*, 16(11):3391–3403, 2023c. doi: 10.14778/3611479.3611534. URL https://doi.org/10.14778/3611479.3611534.

Peng Li, Yeye He, Dror Yashar, Weiwei Cui, Song Ge, Haidong Zhang, Danielle Rifinski Fainman, Dongmei Zhang, and Surajit Chaudhuri. Table-GPT: Table fine-tuned GPT for diverse table tasks. *Proceedings of the ACM on Management of Data (SIGMOD)*, 2(3):Article 176, 2024. doi: 10.1145/3654979. URL https://doi.org/10.1145/3654979.

Pengxiang Li, Zechen Hu, Zirui Shang, Jingrong Wu, Yang Liu, Hui Liu, Zhi Gao, Chenrui Shi, Bofei Zhang, Zihao Zhang, Xiaochuan Shi, Zedong YU, Yuwei Wu, Xinxiao Wu, Yunde Jia, Liuyu Xiang, Zhaofeng He, and Qing Li. Efficient multi-turn rl for gui agents via decoupled training and adaptive data curation. *arXiv (Cornell University)*, 2025.

Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, Chujun Tao, Binxu Li, Xuandong Zhao, Hejia Geng, Xiaojun Wu, Junwei Zhou, Xiaokun Chen, Hanwen Xing, Yubo Li, Qunhong Zeng, Di Wang, Yuanli Wang, Roey Ben Chaim, Penghao Jiang, Haotian Shen, Luyang Kong, Xinyi Liu, Runhui Wang, Xuanqing Liu, Jiachen Li, Xin Lan, Yueqian Lin, Wengao Ye, Junwei He, Songlin Li, Yue Zhang, Yipeng Gao, Yijiang Li, Ze Ma, Liqiang Jing, Tianyu Wang, Kaixin Li, Yiqi Xue, Haoran Lyu, Yizhuo He, Yuchen Tian, Shutong Wu, Bowei Wang, Yixuan Gao, Bo Chen, Litong Liu, Sikai Cheng, Jiajun Bao, Shuaicheng Tong, Shuwen Xu, Terry Yue Zhuo, Tinghan Ye, Qi Qi, Miao Li, Longtai Liao, Zelin Tan, Chang Shi, Xilin Tang, Srinath Tankasala, Boqin Yuan, Yaoyao Qian, Jianhong Tu, Chenguang Wang, Yizhou Sun, Wei Wang, Aaron Taylor, Ziyue Yang, Changkun Guan, Zhikang Dong, Xinyu Zhang, Steven Dillmann, Han chung Lee, and Dawn Song. Skillsbench: Benchmarking how well agent skills work across diverse tasks, 2026a. URL https://arxiv.org/abs/2602.12670.

Yaxuan Li, Yuxin Zuo, Bingxiang He, Jinqian Zhang, Chaojun Xiao, Cheng Qian, Tianyu Yu, Huan-ang Gao, Wenkai Yang, Zhiyuan Liu, and Ning Ding. Rethinking on-policy distillation of large language models: Phenomenology, mechanism, and recipe. *arXiv preprint arXiv:2604.13016*, 2026b. doi: 10.48550/arXiv.2604.13016. URL https://arxiv.org/abs/2604.13016.

Yuanyang Li, Xue Yang, Longyue Wang, Weihua Luo, and Hongyang Chen. Complexmcp: Evaluation of llm agents in dynamic, interdependent, and large-scale tool sandbox. *arXiv preprint arXiv:2605.10787*, 2026c.

Jiawei Lin, Jiaqi Guo, Shizhao Sun, Zijiang James Yang, Jian-Guang Lou, and Dongmei Zhang. LayoutPrompter: Awaken the design ability of large language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. URL https://openreview.net/forum?id=EF56cv8B3b.

Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Stan Weixian Lei, Lijuan Wang, and Mike Zheng Shou. Showui: One vision-language-action model for gui visual agent. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 19498–19508, 2025.

Lindy AI. Lindy: AI executive assistant and agent builder. https://www.lindy.ai/, 2023. No-code platform for building AI agents that automate email, scheduling, support, and outreach across many tools.

Qian Liu, Bei Chen, Jiaqi Guo, Morteza Ziyadi, Zeqi Lin, and Weizhu Chen. Tapex: Table pre-training via learning a neural sql executor. In *International Conference on Learning Representations (ICLR)*, 2021.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL https://openreview.net/forum?id=zAdUB0aCTQ.

Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-eval: Nlg evaluation using gpt-4 with better human alignment. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pp. 2511–2522, 2023.

Nikolaos Livathinos, Christoph Auer, Maksym Lysak, Ahmed Nassar, Michele Dolfi, Panos Vagenas, Cesar Berrospi Ramis, Matteo Omenetti, Kasper Dinkla, Yusik Kim, Shubham Gupta, Rafael Teixeira de Lima, Valery Weber, Lucas Morin, Ingmar Meijer, Viktor Kuropiatnyk, and Peter W. J. Staar. Docling: An efficient open-source toolkit for ai-driven document conversion. *arXiv (Cornell University)*, 2025.

Lovable. Lovable: AI app and website builder. https://lovable.dev/, 2024. Full-stack AI platform that builds and deploys web apps from natural language.

Weizheng Lu, Jing Zhang, Ju Fan, Zihao Fu, Yueguo Chen, and Xiaoyong Du. Large language model for table processing: a survey. *Frontiers of Computer Science*, 19(2), January 2025. ISSN 2095-2236. doi: 10.1007/s11704-024-40763-6. URL http://dx.doi.org/10.1007/s11704-024-40763-6.

Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. Omniparser for pure vision based gui agent, 2024. URL https://arxiv.org/abs/2408.00203.

Tengchao Lv, Dongdong Zhang, Jiayu Ding, Yilin Jia, Yuzhong Zhao, Yupan Huang, Wenshan Wu, Xiangyang Zhou, Shaohan Huang, Nan Yang, Li Dong, Lei Cui, and Furu Wei. Mind the gap: Can frontier llms pass a standardized office proficiency exam? *arXiv (Cornell University)*, 2026.

Yubo Ma, Yuhang Zang, Liangyu Chen, Meiqi Chen, Yizhu Jiao, Xinze Li, Xinyuan Lu, Ziyu Liu, Yan Ma, Xiaoyi Dong, Pan Zhang, Liangming Pan, Yu-Gang Jiang, Jiaqi Wang, Yixin Cao, and Aixin Sun. MMLongBench-Doc: Benchmarking long-context document understanding with visualizations. In *Advances in Neural Information Processing Systems 37 (NeurIPS) Datasets and Benchmarks Track*, 2024a. URL https://openreview.net/forum?id=loJM1acwzf.

Zeyao Ma, Bohan Zhang, Jing Zhang, Jifan Yu, Xiaokang Zhang, Xiaohan Zhang, Sijia Luo, Xi Wang, and Jie Tang. SpreadsheetBench: Towards challenging real world spreadsheet manipulation. In *Advances in Neural Information Processing Systems 37 (NeurIPS) Datasets and Benchmarks Track*, 2024b. URL https://neurips.cc/virtual/2024/poster/97753.

Zeyao Ma et al. SpreadsheetBench 2: End-to-end business spreadsheet workflows. https://huggingface.co/datasets/KAKA22/SpreadsheetBench-v2, 2026. Dataset release (321 tasks: financial modeling, debugging, and visualization over multi-sheet workbooks); project page https://spreadsheetbench.github.io/; paper forthcoming.

Manus. Manus: A general AI agent. https://manus.im/, 2026. General agent that builds slides, websites, and data artifacts and manages local files.

Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq R. Joty, and Enamul Hoque. Chartqa: A benchmark for question answering about charts with visual and logical reasoning. In *Findings of the Association for Computational Linguistics: ACL 2022*, 2022.

Minesh Mathew, Dimosthenis Karatzas, and C.V. Jawahar. DocVQA: A dataset for VQA on document images. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pp. 2200–2209, 2021. URL https://openaccess.thecvf.com/content/WACV2021/html/Mathew_DocVQA_A_Dataset_for_VQA_on_Document_Images_WACV_2021_paper.html.

Meitu. WHEE: AI image and art creation platform. https://www.whee.com/, 2023. Meitu's AI platform for text-to-image, image-to-image, and AI art on the MiracleVision model.

Miaoya. Miaoya Camera: AI portrait generator. https://www.miaoya.cn/, 2023. Alibaba-backed app that builds a digital avatar from user photos to generate AI portrait styles.

Microsoft. Introducing microsoft 365 copilot — your copilot for work. https://blogs.microsoft.com/blog/2023/03/16/introducing-microsoft-365-copilot-your-copilot-for-work/, 2023. Official announcement, March 16, 2023.

Microsoft. Copilot for Power BI overview. https://learn.microsoft.com/en-us/power-bi/create-reports/copilot-introduction, 2024. Microsoft Learn overview of Copilot capabilities in Power BI.

Microsoft. Microsoft Editor: Checks grammar and more in documents, mail, and the web. https://support.microsoft.com/en-us/word/microsoft-editor-checks-grammar-and-more-in-documents-mail-and-the-web, 2025a.

Microsoft. Copilot in Excel: Your AI assistant for spreadsheets. https://support.microsoft.com/en-us/copilot-excel, 2025b.

Microsoft. PowerPoint Designer: Get design ideas for slides. https://support.microsoft.com/en-us/powerpoint/training/get-design-ideas-for-slides-with-powerpoint-designer, 2025c.

Microsoft 365 Blog. Introducing microsoft copilot actions, new agents, and tools to empower it teams, November 2024. URL https://www.microsoft.com/en-us/microsoft-365/blog/2024/11/19/introducing-copilot-actions-new-agents-and-tools-to-empower-it-teams/. Published: 2024-11-19.

Microsoft 365 Blog. Microsoft ignite 2025: Copilot and agents built to power the frontier firm, November 2025a. URL https://www.microsoft.com/en-us/microsoft-365/blog/2025/11/18/microsoft-ignite-2025-copilot-and-agents-built-to-power-the-frontier-firm/. Published: 2025-11-18.

Microsoft 365 Blog. Introducing microsoft 365 copilot tuning, multi-agent orchestration, and more from microsoft build 2025, May 2025b. URL https://www.microsoft.com/en-us/microsoft-365/blog/2025/05/19/introducing-microsoft-365-copilot-tuning-multi-agent-orchestration-and-more-from-microsoft-build-2025/. Published: 2025-05-19.

Microsoft 365 Blog. Copilot's agentic capabilities in Word, Excel, and PowerPoint are generally available, April 2026. URL https://www.microsoft.com/en-us/microsoft-365/blog/2026/04/22/copilots-agentic-capabilities-in-word-excel-and-powerpoint-are-generally-available/. Published: 2026-04-22.

Microsoft Learn. How the Microsoft 365 Copilot orchestrator chooses actions, January 2026a. URL https://learn.microsoft.com/en-us/microsoft-365-copilot/extensibility/orchestrator. Last updated: 2026-01-23.

Microsoft Learn. Microsoft 365 Copilot Tuning overview (early access preview), March 2026b. URL https://learn.microsoft.com/en-us/microsoft-365/copilot/copilot-tuning-overview. Last updated: 2026-03-10.

Midjourney. Midjourney: AI image generation. https://www.midjourney.com/, 2022. Generative AI service producing images from natural-language prompts via web app and Discord.

MiniMax. MiniMax Skills: Plug-in skill library for AI coding agents. https://github.com/MiniMax-AI/skills, 2026. Open-source skill folders including `minimax-docx`, `minimax-xlsx`, `minimax-pdf`, and `pptx-generator` for office artifact authoring.

Monarch Money. Monarch Money: Personal finance with AI assistant. https://www.monarch.com/, 2024. Budgeting and net-worth app with an AI assistant, AI-powered insights, and weekly financial recaps.

Moonshot AI. Kimi K2.6: Long-horizon agentic model with agent swarm. https://www.kimi.com/, 2026. Open-weight model whose Agent Swarm coordinates up to 300 parallel sub-agents across thousands of steps.

Napkin AI. Napkin AI: Visual AI for business storytelling. https://www.napkin.ai/, 2024. Transforms text into diagrams, charts, and infographics for visual storytelling.

Dang Nguyen, Jian Chen, Yu Wang, Gang Wu, Namyong Park, Zhengmian Hu, Hanjia Lyu, Junda Wu, Ryan Aponte, Yu Xia, Xintong Li, Jing Shi, Hongjie Chen, Viet Dac Lai, Zhouhang Xie, Sungchul Kim, Ruiyi Zhang, Tong Yu, Mehrab Tanjim, Nesreen K. Ahmed, Puneet Mathur, Seunghyun Yoon, Lina Yao, Branislav Kveton, Jihyung Kil, Thien Huu Nguyen, Trung Bui, Tianyi Zhou, Ryan A. Rossi, and Franck Dernoncourt. Gui agents: A survey, 2025. URL https://arxiv.org/abs/2412.13501.

Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjun Jiang. Trace2skill: Distill trajectory-local lessons into transferable agent skills. *arXiv (Cornell University)*, 2026.

Harsha Nori, Yin Tat Lee, Sheng Zhang, Dean Carignan, Richard Edgar, Nicolo Fusi, Nicholas King, Jonathan Larson, Yuanzhi Li, Weishung Liu, Renqian Luo, Scott Mayer McKinney, Robert Osazuwa Ness, Hoifung Poon, Tao Qin, Naoto Usuyama, Chris White, and Eric Horvitz. Can generalist foundation models outcompete special-purpose tuning? Case study in medicine. *arXiv preprint arXiv:2311.16452*, 2023. URL https://arxiv.org/abs/2311.16452.

Notion Labs. Notion: The AI workspace that works for you. https://www.notion.com/, 2023. Official Notion product homepage.

nTop. nTop: Computational and generative engineering design. https://www.ntop.com/, 2023. Implicit-modeling computational design software for generative, simulation-driven engineering (formerly nTopology).

Michael Ofengenden, Yunze Man, Ziqi Pang, and Yu-Xiong Wang. PPTArena: A benchmark for agentic PowerPoint editing. *arXiv preprint arXiv:2512.03042*, 2025. URL https://arxiv.org/abs/2512.03042.

OpenAI. Introducing chatgpt agent: Bridging research and action, July 2025a. URL https://openai.com/index/introducing-chatgpt-agent/. Published: 2025-07-17.

OpenAI. ChatGPT desktop. https://openai.com/chatgpt/desktop/, 2025b. OpenAI desktop application.

OpenAI. OpenAI Codex CLI: A coding agent for the terminal. https://github.com/openai/codex, 2025c. GitHub repository.

OpenAI. Computer-using agent, January 2025d. URL https://openai.com/index/computer-using-agent/. Published: 2025-01-23.

OpenAI. New tools for building agents, March 2025e. URL https://openai.com/index/new-tools-for-building-agents/. Published: 2025-03-11.

OpenAI. Introducing OpenAI o3 and o4-mini, April 2025f. URL https://openai.com/index/introducing-o3-and-o4-mini/. Published: 2025-04-16.

OpenAI. Function calling, 2026a. URL https://developers.openai.com/api/docs/guides/function-calling. Accessed: 2026-05-20.

OpenAI. The next era of knowledge work: How Codex is helping people navigate the complexity of modern work, June 2026b. URL https://cdn.openai.com/pdf/the-next-era-of-knowledge-work.pdf. Published: 2026-06-02.

Rai OpenAI, Rai Josh Achiam, Rai Steven Adler, Rai Sandhini Agarwal, Rai Lama Ahmad, Rai Ilge Akkaya, Rai Florencia Leoni Aleman, Rai Diogo Almeida, Rai Janko Altenschmidt, Rai Sam Altman, Rai Shyamal Anadkat, Rai Red Avila, Rai Igor Babuschkin, Rai Suchir Balaji, Rai Valerie Balcom, Rai Paul Baltescu, Rai Haiming Bao, Rai Mohammad Bavarian, Rai Jeff Belgum, Rai Irwan Bello, Rai Jake Berdine, Rai Gabriel Bernadett-Shapiro, Rai Christopher Berner, Rai Lenny Bogdonoff, Rai Oleg Boiko, Rai Madelaine Boyd, Rai Anna-Luisa Brakman, Rai Greg Brockman, Rai Tim Brooks, Rai Miles Brundage, Rai Kevin Button, Rai Trevor Cai, Rai Rosie Campbell, Rai Andrew Cann, Rai Brittany Carey, Rai Chelsea Carlson, Rai Rory Carmichael, Rai Brooke Chan, Rai Che Chang, Rai Fotis Chantzis, Rai Derek Chen, Rai Sully Chen, Rai Ruby Chen, Rai Jason Chen, Rai Mark Chen, Rai Ben Chess, Rai Chester Cho, Rai Casey Chu, Rai Hyung Won Chung, Rai Dave Cummings, Rai Jeremiah Currier, Rai Yunxing Dai, Rai Cory Decareaux, Rai Thomas Degry, Rai Noah Deutsch, Rai Damien Deville, Rai Arka Dhar, Rai David Dohan, Rai Steve Dowling, Rai Sheila Dunning, Rai Adrien Ecoffet, Rai Atty Eleti, Rai Tyna Eloundou, Rai David Farhi, Rai Liam Fedus, Rai Niko Felix, Rai Simón Posada Fishman, Rai Juston Forte, Rai Isabella Fulford, Rai Leo Gao, Rai Elie Georges, Rai Christian Gibson, Rai Vik Goel, Rai Tarun Gogineni, Rai Gabriel Goh, Rai Rapha Gontijo-Lopes, Rai Jonathan Gordon, Rai Morgan Grafstein, Rai Scott Gray, Rai Ryan Greene, Rai Joshua Gross, Rai Shixiang Shane Gu, Rai Yufei Guo, Rai Chris Hallacy, Rai Jesse Han, Rai Jeff Harris, Rai Yuchen He, Rai Mike Heaton, Rai Johannes Heidecke, Rai Chris Hesse, Rai Alan Hickey, Rai Wade Hickey, Rai Peter Hoeschele, Rai Brandon Houghton, Rai Kenny Hsu, Rai Shengli Hu, Rai Xin Hu, Rai Joost Huizinga, Rai Shantanu Jain, Rai Shawn Jain, Rai Joanne Jang, Rai Angela Jiang, Rai Roger Jiang, Rai Haozhun Jin, Rai Denny Jin, Rai Shino Jomoto, Rai Billie Jonn, Rai Heewoo Jun, Rai Tomer Kaftan, Rai Łukasz Kaiser, Rai Ali Kamali, Rai Ingmar Kanitscheider, Rai Nitish Shirish Keskar, Rai Tabarak Khan, Rai Logan Kilpatrick, Rai Jong Wook Kim, Rai Christina Kim, Rai Yongjik Kim, Rai Jan Hendrik Kirchner, Rai Jamie Kiros, Rai Matt Knight, Rai Daniel Kokotajlo, Rai Łukasz Kondraciuk, Rai Andrew Kondrich, Rai Aris Konstantinidis, Rai Kyle Kosic, Rai Gretchen Krueger, Rai Vishal Kuo, Rai Michael Lampe, Rai Ikai Lan, Rai Teddy Lee, Rai Jan Leike, Rai Jade Leung, Rai Daniel Levy, Rai Chak Ming Li, Rai Rachel Lim, Rai Molly Lin, Rai Stephanie Lin, Rai Mateusz Litwin, Rai Theresa Lopez, Rai Ryan Lowe, Rai Patricia Lue, Rai Anna Makanju, Rai Kim Malfacini, Rai Sam Manning, Rai Todor Markov, Rai Yaniv Markovski, Rai Bianca Martin, Rai Katie Mayer, Rai Andrew Mayne, Rai Bob McGrew, Rai Scott Mayer McKinney, Rai Christine McLeavey, Rai Paul McMillan, Rai Jake McNeil, Rai David Medina, Rai Aalok Mehta, Rai Jacob Menick, Rai Luke Metz, Rai Andrey Mishchenko, Rai Pamela Mishkin, Rai Vinnie Monaco, Rai Evan Morikawa, Rai Daniel Mossing, Rai Tong Mu, Rai Mira Murati, Rai Oleg Murk, Rai David Mély, Rai Ashvin Nair, Rai Reiichiro Nakano, Rai Rajeev Nayak, Rai Arvind Neelakantan, Rai Richard Ngo, Rai Hyeonwoo Noh, Rai Long Ouyang, Rai Cullen O'Keefe, Rai Jakub Pachocki, Rai Alex Paino, Rai Joe Palermo, Rai Ashley Pantuliano, Rai Giambattista Parascandolo, Rai Joel Parish, Rai Emy Parparita, Rai Alex Passos, Rai Mikhail Pavlov, Rai Andrew Peng, Rai Adam Perelman, Rai Filipe de Avila Belbute Peres, Rai Michael Petrov, Rai Henrique Ponde de Oliveira Pinto, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. Gpt-4 technical report. *arXiv (Cornell University)*, 2023.

OpenCode Contributors. OpenCode: The open-source coding agent. https://opencode.ai/, 2025. Project site; source at https://github.com/anomalyco/opencode.

Krista Opsahl-Ong, Arnav Singhvi, Jasmine Collins, Ivan Zhou, Cindy Wang, Ashutosh Baheti, Owen Oertell, Jacob Portes, Sam Havens, Erich Elsen, Michael Bendersky, Matei Zaharia, and Xing Chen. Officeqa pro: An enterprise benchmark for end-to-end grounded reasoning. *arXiv (Cornell University)*, 2026.

OpusClip. OpusClip: AI video clipping and editing tool. https://www.opus.pro/, 2024. AI that turns long videos into short, captioned clips optimized for social platforms.

Addy Osmani. Loop engineering: Designing agentic loops instead of prompts. https://addyosmani.com/blog/loop-engineering/, 2025. Engineering blog.

Otter.ai. Otter.ai: AI meeting transcription and notes. https://otter.ai/, 2023. Real-time meeting transcription, speaker ID, and AI summaries for Zoom, Google Meet, and Teams.

Siru Ouyang, Jun Yan, Yanfei Chen, Rujun Han, Zifeng Wang, Bhavana Dalvi Mishra, Rui Meng, Chun-Liang Li, Yizhu Jiao, Kaiwen Zha, Maohao Shen, Vishy Tirumalashetty, George Lee, Jiawei Han, Tomas Pfister, and Chen-Yu Lee. Skillos: Learning skill curation for self-evolving agents. *arXiv (Cornell University)*, 2026.

Yiming Pan, Chengwei Hu, Xuancheng Huang, Can Huang, Mingming Zhao, Yuean Bi, Xiaohan Zhang, Aohan Zeng, and Linmei Hu. AeSlides: Incentivizing aesthetic layout in LLM-based slide generation via verifiable rewards. *arXiv preprint arXiv:2604.22840*, 2026. doi: 10.48550/arXiv.2604.22840. URL https://arxiv.org/abs/2604.22840.

Raja Parasuraman and Victor Riley. Humans and automation: Use, misuse, disuse, abuse. *Human Factors*, 39 (2):230–253, 1997.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis, 2023. URL https://arxiv.org/abs/2305.15334.

Justin Payan, Swaroop Mishra, Mukul Singh, Carina Negreanu, Christian Poelitz, Chitta Baral, Subhro Roy, Rasika Chakravarthy, Benjamin Van Durme, and Elnaz Nouri. InstructExcel: A benchmark for natural language instruction in excel. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 4026–4043, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.265. URL https://aclanthology.org/2023.findings-emnlp.265/.

Perplexity AI. Comet: a personal AI browser and assistant. https://www.perplexity.ai/comet, 2025. AI-native browser that searches the web, summarizes pages, and autonomously completes browser tasks.

Fatemeh Pesaran Zadeh, Juyeon Kim, Jin-Hwa Kim, and Gunhee Kim. Text2Chart31: Instruction tuning for chart generation with automatic feedback. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 11459–11480, Miami, Florida, USA, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.640. URL https://aclanthology.org/2024.emnlp-main.640/.

Photoroom. Photoroom: AI-powered photo editor and listing studio for product photography. https://www.photoroom.com/, 2023. AI photo editor for background removal, product shots, and batch image editing.

Pika Labs. Pika: AI video generation. https://pika.art/, 2023. Generates and edits video from text, images, and footage with effects.

PptxGenJS. PptxGenJS documentation, 2026. URL https://gitbrent.github.io/PptxGenJS/. Accessed: 2026-05-20.

PTC. Onshape AI Advisor: Generative AI design assistant. https://www.ptc.com/en/news/2025/ptc-announces-onshape-ai-advisor-and-onshape-government, 2025. Generative AI assistant embedded in Onshape providing real-time CAD and PDM guidance.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL https://openreview.net/forum?id=dHng2O0Jjr.

Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, Wanjun Zhong, Kuanye Li, Jiale Yang, Yu Miao, Woyu Lin, Longxiang Liu, Xu Jiang, Qianli Ma, Jingyu Li, Xiaojun Xiao, Kai Cai, Chuang Li, Yaowei Zheng, Chaolin Jin, Chen Li, Xiao Zhou, Minchao Wang, Haoli Chen, Zhaojian Li, Haihua Yang, Haifeng Liu, Feng Lin, Tao Peng, Xin Liu, and Guang Shi. UI-TARS: Pioneering automated GUI interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025. URL https://arxiv.org/abs/2501.12326.

Qoder. Qoder: An autonomous development desktop AI coding assistant. https://qoder.com/, 2025. Product page.

Quadratic. Quadratic: The AI spreadsheet. https://www.quadratichq.com/, 2025. AI-native spreadsheet supporting formulas, Python, SQL, and JavaScript with live data connections.

Quora. Poe: Unified access to AI chatbots. https://poe.com/, 2023. Platform giving users unified access to multiple AI chatbots and models in a single interface.

Vipul Raheja, Dhruv Kumar, Ryan Koo, and Dongyeop Kang. CoEdIT: Text editing by task-specific instruction tuning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 5274–5291, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.350. URL https://aclanthology.org/2023.findings-emnlp.350/.

Vipul Raheja, Dimitris Alikaniotis, Vivek Kulkarni, Bashar Alhafni, and Dhruv Kumar. mEdIT: Multilingual text editing via instruction tuning. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024. URL https://arxiv.org/abs/2402.16472.

Read AI. Read AI: AI meeting assistant and notetaker. https://www.read.ai/, 2023. Real-time transcription, summaries, and action items across Zoom, Teams, and Google Meet.

Recraft. Recraft: AI design platform for vector and raster art. https://www.recraft.ai/, 2024. AI design platform generating raster and vector images, mockups, and custom reusable styles.

Houxing Ren, Mingjie Zhan, Zimu Lu, Ke Wang, Yunqiao Yang, Haotian Hou, and Hongsheng Li. Towards robust real-world spreadsheet understanding with multi-agent multi-format reasoning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1906–1933, 2026.

Replit. Replit Agent: AI software development agent. https://replit.com/ai, 2024. Autonomous AI agent that builds, tests, and deploys full applications from natural-language prompts.

Rows. Rows AI: AI-powered spreadsheet and data analyst. https://rows.com/ai, 2024. Spreadsheet with a built-in AI analyst to get, transform, and analyze data without formulas.

Runway. Introducing Runway Gen-4, 2025. URL https://runwayml.com/research/introducing-runway-gen-4. Controllable video generation with consistent characters, scenes, and objects.

Jon Saad-Falcon, Joe Barrow, Alexa F. Siu, Ani Nenkova, Ryan A. Rossi, and Franck Dernoncourt. PDFTriage: Question answering over long, structured documents. *arXiv preprint arXiv:2309.08872*, 2023. URL https://arxiv.org/abs/2309.08872.

Timo Schick, Jane Dwivedi-Yu, Zhengbao Jiang, Fabio Petroni, Patrick Lewis, Gautier Izacard, Qingfei You, Christoforos Nalmpantis, Edouard Grave, and Sebastian Riedel. PEER: A collaborative language model. *arXiv preprint arXiv:2208.11663*, 2022. URL https://arxiv.org/abs/2208.11663. Also accepted at ICLR 2023.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. URL https://openreview.net/forum?id=Yacmpz84TH.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Yijia Shao, Yucheng Jiang, Theodore A. Kanell, Peter Xu, Omar Khattab, and Monica S. Lam. Assisting in writing Wikipedia-like articles from scratch with large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024a. URL https://arxiv.org/abs/2402.14207.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv (Cornell University)*, 2024b.

Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. URL https://openreview.net/forum?id=yHdTscY6Ci.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. URL https://openreview.net/forum?id=vAElhFcKW6.

Lei Shu, Liangchen Luo, Jayakumar Hoskere, Yun Zhu, Yinxiao Liu, Simon Tong, Jindong Chen, and Lei Meng. RewriteLM: An instruction-tuned large language model for text rewriting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024. URL https://arxiv.org/abs/2305.15685.

Mukul Singh, José Cambronero, Sumit Gulwani, Vu Le, Carina Negreanu, Elnaz Nouri, Mohammad Raza, and Gust Verbruggen. FormaT5: Abstention and examples for conditional table formatting with natural language. *Proceedings of the VLDB Endowment*, 17(3):497–510, 2023. doi: 10.14778/3632093.3632111. URL https://doi.org/10.14778/3632093.3632111.

Robert M. Solow. We'd better watch out. *The New York Times Book Review*, pp. 36, July 1987. Review of S. S. Cohen and J. Zysman, Manufacturing Matters: The Myth of the Post-Industrial Economy.

Linxin Song, Yutong Dai, Viraj Prabhu, Jieyu Zhang, Taiwei Shi, Li Li, Junnan Li, Silvio Savarese, Zeyuan Chen, Jieyu Zhao, Ran Xu, and Caiming Xiong. Coact-1: Computer-using multi-agent system with coding actions. *ArXiv.org*, 2025.

Mingyang Song and Mao Zheng. A survey of on-policy distillation for large language models. *arXiv preprint arXiv:2604.00626*, 2026. doi: 10.48550/arXiv.2604.00626. URL https://arxiv.org/abs/2604.00626.

Spellbook. Spellbook: AI contract review and drafting. https://spellbook.com/, 2023. AI assistant that drafts and reviews contracts inside Microsoft Word for transactional lawyers.

StackBlitz. Bolt.new: AI app builder. https://bolt.new/, 2024. Browser-based AI builder that generates and deploys full-stack web and mobile apps from a prompt.

Aofeng Su, Aowen Wang, Chao Ye, Chen Zhou, Ga Zhang, Gang Chen, Guangcheng Zhu, Haobo Wang, Haokai Xu, Hao Chen, Haoze Li, Haoxuan Lan, Jiaming Tian, Jing Yuan, Junbo Zhao, Junlin Zhou, Kaizhe Shou, Liangyu Zha, Lin Long, Liyao Li, Pengzuo Wu, Qi Zhang, Qingyi Huang, Saisai Yang, Tao Zhang, Wentao Ye, Wufang Zhu, Xiaomeng Hu, Xijun Gu, Xinjie Sun, Xiang Li, Yuhang Yang, and Zhiqing Xiao. TableGPT2: A large multimodal model with tabular data integration. *arXiv preprint arXiv:2411.02059*, 2024. URL https://arxiv.org/abs/2411.02059.

Edward Sun, Yufang Hou, Dakuo Wang, Yunfeng Zhang, and Nancy X. R. Wang. D2S: Document-to-slide generation via query-based text summarization. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pp. 1405–1418. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.naacl-main.111. URL https://aclanthology.org/2021.naacl-main.111/.

Suno. Suno: AI music generator. https://suno.com/, 2024. Generates complete original songs with vocals, lyrics, and production from a text prompt.

Synthesia. Synthesia: AI video platform with avatars. https://www.synthesia.io/, 2024. Generates AI videos from text using realistic AI avatars and multilingual voiceovers.

Tableau. Tableau Agent: Trusted AI built into the Tableau experience. https://www.tableau.com/products/tableau-agent, 2024. Official Tableau product page for the generative AI assistant used across data preparation, exploration, visualization, and consumption.

Zirui Tang, Xuanhe Zhou, Yumou Liu, Linchun Li, Weizheng Wang, Hongzhang Huang, Jun Zhou, Jiachen Song, Shaoli Yu, Jinqi Wang, Zihang Zhou, Hongyi Zhou, Yuting Lv, Jinyang Li, Jiashuo Liu, Ruoyu Chen, Chunwei Liu, and GuoLiang Li. Workspace-bench 1.0: Benchmarking ai agents on workspace tasks with large-scale file dependencies. *arXiv (Cornell University)*, 2026.

5 Team, Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chengxing Xie, Cunxiang Wang, Gengzheng Pan, Hao Zeng, Haoke Zhang, Haoran Wang, Huilong Chen, Jiajie Zhang, Jian Jiao, Jiaqi Guo, Jingsen Wang, Jingzhao Du, Jinzhu Wu, Kedong Wang, Lei Li, Lin Fan, Lucen Zhong, Mingdao Liu, Mingming Zhao, Pengfan Du, Qian Dong, Rui Lu, Shuang-Li, Shulin Cao, Song Liu, Ting Jiang, Xiaodong Chen, Xiaohan Zhang, Xuancheng Huang, Xuezhen Dong, Yabo Xu, Yao Wei, Yifan An, Yilin Niu, Yitong Zhu, Yuanhao Wen, Yukuo Cen, Yushi Bai, Zhongpei Qiao, Zihan Wang, Zikang Wang, Zilin Zhu, Ziqiang Liu, Zixuan Li, Bojie Wang, Bosi Wen, Can Huang, Changpeng Cai, Chao Yu, Chenghua Huang, Chengwei Hu, Chenhui Zhang, Chenzheng Zhu, Congfeng Yin, Daoyan Lin, Dayong Yang, Di Wang, Ding Ai, Erle Zhu, Fangzhou Yi, Feiyu Chen, Guohong Wen, Hailong Sun, Haisha Zhao, Haiyi Hu, Hanchen Zhang, Hanrui Liu, Hanyu Zhang, Hao Peng, Hao Tai, Haobo Zhang, He Liu, Hongwei Wang, Hongxi Yan, Hongyu Ge, Huanpeng Chu, Jia'ni Zhao, Jiachen Wang, Jiajing Zhao, Jiamin Ren, Jiapeng Wang, Jiaxin Zhang, Jiayi Gui, Jiayue Zhao, Jijie Li, Jing An, Jing Li, Jingwei Yuan, Jinhua Du, Jinxin Liu, Junkai Zhi, Junwen Duan, Kaiyue Zhou, Kangjian Wei, Ke Wang, Keyun Luo, Laiqiang Zhang, Leigang Sha, Liang Xu, Lindong Wu, Lintao Ding, Lu Chen, Minghao Li, Nianyi Lin, Pan Ta, Qiang Zou, Rongjun Song, Ruiqi Yang, Shangqing Tu, Shangtong Yang, Shaoxiang Wu, Shengyan Zhang, Shijie Li, Shuang Li, Shuyi Fan, Wei Qin, Wei Tian, Weining Zhang, Wenbo Yu, Wenjie Liang, Xiang Kuang, Xiangmeng Cheng, Xiangyang Li, Xiaoquan Yan, Xiaowei Hu, Xiaoying Ling, Xing Fan, Xingye Xia, Xinyuan Zhang, Xinze Zhang, Xirui Pan, Xunkai Zhang, Yandong Wu, Yanfu Li, Yidong Wang, Yifan Zhu, Yijun Tan, Yilin Zhou, Yiming Pan, Ying Zhang, Yinpei Su, Yong Yan, Yonglin Tan, Yuean Bi, Yuhan Shen, Yuhao Yang, Yujiang Li, Yunan Liu, Yunqing Wang, Yuntao Li, Yurong Wu, Yutao Zhang, Yuxi Duan, Yuxuan Zhang, Zezhen Liu, Zhengtao Jiang, Zhenhe Yan, Zheyu Zhang, Zhixiang Wei, Zhuo Chen, Zhuoer Feng, Zijun Yao, Ziwei Chai, Ziyuan Wang, Zuzhou Zhang, Bin Xu, Minlie Huang, Hongning Wang, Juanzi Li, and Yuxiao Dong. Glm-5: from vibe coding to agentic engineering. *arXiv (Cornell University)*, 2026a.

Gemini Team. Gemini: A family of highly capable multimodal models. *arXiv (Cornell University)*, 2023.

Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, S. H. Cai, Yuan Cao, Y. Charles, H. S. Che, Cheng Chen, Guanduo Chen, Huarong Chen, Jia Chen, Jiahao Chen, Jianlong Chen, Jun Chen, Kefan Chen, Liang Chen, Ruijue Chen, Xinhao Chen, Yanru Chen, Yanxu Chen, Yicun Chen, Yimin Chen, Yingjiang Chen, Yuankun Chen, Yujie Chen, Yutian Chen, Zhirong Chen, Ziwei Chen, Dazhi Cheng, Minghan Chu, Jialei Cui, Jiaqi Deng, Muxi Diao, Hao Ding, Mengfan Dong, Mengnan Dong, Yuxin Dong, Yuhao Dong, Angang Du, Chenzhuang Du, Dikang Du, Lingxiao Du, Yulun Du, Yu Fan, Shengjun Fang, Qiulin Feng, Yichen Feng, Garimugai Fu, Kelin Fu, Hongcheng Gao, Tong Gao, Yuyao Ge, Shangyi Geng, Chengyang Gong, Xiaochen Gong, Zhuoma Gongque, Qizheng Gu, Xinran Gu, Yicheng Gu, Longyu Guan, Yuanying Guo, Xiaoru Hao, Weiran He, Wenyang He, Yunjia He, Chao Hong, Hao Hu, Jiaxi Hu, Yangyang Hu, Zhenxing Hu, Ke Huang, Ruiyuan Huang, Weixiao Huang, Zhiqi Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yu Jing, Guokun Lai, Aidi Li, C. Li, Fang Li, Guanghe Li, Guanyu Li, Haitao Li, Haoyang Li, Jia Li, Jingwei Li, Junxiong Li, Lincan Li, Mo Li, Weihong Li, Wentao Li, Xinhang Li, Xinhao Li, Yang Li, Yanhao Li, Yiwei Li, Yuxiao Li, Zhaowei Li, Zheming Li, Weilong Liao, Jiawei Lin, Xiaohan Lin, Zhishan Lin, Zichao Lin, Cheng Liu, Chenyu Liu, Hongzhang Liu, Liang Liu, Shaowei Liu, Shudong Liu, Shuran Liu, Tianwei Liu, Tianyu Liu, Weizhou Liu, Xiangyan Liu, Yangyang Liu, Yanming Liu, Yibo Liu, Yuanxin Liu, Yue Liu, Zhengying Liu, Zhongnuo Liu, Enzhe Lu, Haoyu Lu, Zhiyuan Lu, Junyu Luo, Tongxu Luo, Yashuo Luo, Long Ma, Yingwei Ma, Shaoguang Mao, Yuan Mei, Xin Men, Fanqing Meng, Zhiyong Meng, Yibo Miao, Minqing Ni, Kun Ouyang, Siyuan Pan, Bo Pang, Yuchao Qian, Ruoyu Qin, Zeyu Qin, Jiezhong Qiu, Bowen Qu, Zeyu Shang, Youbo Shao, Tianxiao Shen, Zhennan Shen, Juanfeng Shi, Lidong Shi, Shengyuan Shi, Feifan Song, Pengwei Song, Tianhui Song, Xiaoxi Song, Hongjin Su, Jianlin Su, Zhaochen Su, Lin Sui, Jinsong Sun, Junyao Sun, Tongyu Sun, Flood Sung, Yunpeng Tai, Chuning Tang, Heyi Tang, Xiaojuan Tang, Zhengyang Tang, Jiawen Tao, Shiyuan Teng, Chaoran Tian, Pengfei Tian, Ao Wang, Bowen Wang, Chensi Wang, Chuang Wang, Congcong Wang, Dingkun Wang, Dinglu Wang, Dongliang Wang, Feng Wang, Hailong Wang, Haiming Wang, Hengzhi Wang, Huaqing Wang, Hui

Wang, Jiahao Wang, Jinhong Wang, Jiuzheng Wang, Kaixin Wang, Linian Wang, Qibin Wang, Shengjie Wang, Shuyi Wang, Si Wang, Wei Wang, Xiaochen Wang, Xinyuan Wang, Yao Wang, Yejie Wang, Yipu Wang, Yiqin Wang, Yucheng Wang, Yuzhi Wang, Zhaoji Wang, Zhaowei Wang, Zhengtao Wang, Zhexu Wang, Zihan Wang, Zizhe Wang, Chu Wei, Ming Wei, Chuan Wen, Zichen Wen, Chengjie Wu, Haoning Wu, Junyan Wu, Rucong Wu, Wenhao Wu, Yuefeng Wu, Yuhao Wu, Yuxin Wu, Zijian Wu, Chenjun Xiao, Jin Xie, Xiaotong Xie, Yuchong Xie, Yifei Xin, Bowei Xing, Boyu Xu, Jianfan Xu, Jing Xu, Jinjing Xu, L. H. Xu, Lin Xu, Suting Xu, Weixin Xu, Xinbo Xu, Xinran Xu, Yangchuan Xu, Yichang Xu, Yuemeng Xu, Zelai Xu, Ziyao Xu, Junjie Yan, Yuzi Yan, Guangyao Yang, Hao Yang, Junwei Yang, Kai Yang, Ningyuan Yang, Ruihan Yang, Xiaofei Yang, Xinlong Yang, Ying Yang, Zhen Yang, Zhilin Yang, Zonghan Yang, Haotian Yao, Dan Ye, Wenjie Ye, Zhuorui Ye, Bohong Yin, Chengzhen Yu, Longhui Yu, Tao Yu, Tianxiang Yu, Enming Yuan, Mengjie Yuan, Xiaokun Yuan, Yang Yue, Weihao Zeng, Dunyuan Zha, Haobing Zhan, Dehao Zhang, Hao Zhang, Jin Zhang, Puqi Zhang, Qiao Zhang, Rui Zhang, Xiaobin Zhang, Y. Zhang, Yadong Zhang, Yangkun Zhang, Yichi Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yushun Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Chenguang Zhao, Feifan Zhao, Jinxiang Zhao, Shuai Zhao, Xiangyu Zhao, Yikai Zhao, Zijia Zhao, Huabin Zheng, Ruihan Zheng, Shaojie Zheng, Tengyang Zheng, Junfeng Zhong, Longguang Zhong, Weiming Zhong, M. Zhou, Runjie Zhou, Xinyu Zhou, Zaida Zhou, Jinguo Zhu, Liya Zhu, Xinhao Zhu, Yuxuan Zhu, Zhen Zhu, Jingze Zhuang, Weiyu Zhuang, Ying Zou, and Xinxing Zu. Kimi k2.5: Visual agentic intelligence. *arXiv (Cornell University)*, 2026b.

Tencent. Tencent Yuanqi: AI agent creation and distribution platform. https://yuanqi.tencent.com/, 2024. Zero-code platform on the Hunyuan model for building and distributing AI agents across WeChat, QQ, and Tencent Cloud.

Tencent. QClaw: One-click local OpenClaw deployment with messaging-app control. https://qclawsg.qq.com/, 2026. Personal local AI agent with one-click OpenClaw deployment and WeChat/QQ remote task control.

Tencent Cloud. WorkBuddy: An AI-native desktop agent for office work. https://www.workbuddy.ai/, 2026. Multi-agent desktop office agent producing reports, decks, and spreadsheets with enterprise tool integration.

TestFit. TestFit: AI real estate feasibility platform. https://www.testfit.io/, 2023. Generative-design platform that produces and optimizes building site plans and feasibility analyses.

Rubèn Tito, Dimosthenis Karatzas, and Ernest Valveny. Hierarchical multimodal transformers for multipage DocVQA. *Pattern Recognition*, 144:109834, 2023. doi: 10.1016/j.patcog.2023.109834. URL https://doi.org/10.1016/j.patcog.2023.109834.

Tome App AI. Tome App AI: AI presentation maker. https://tomeapp.ai/, 2023.

Uizard. Uizard: AI-powered UI design. https://uizard.io/, 2023. AI UI design tool that generates wireframes, mockups, and prototypes from prompts and sketches.

Annie Vella and Kelly Blincoe. The impact of AI coding assistants on software engineering: A longitudinal study. *arXiv preprint arXiv:2605.23135*, 2026. URL https://arxiv.org/abs/2605.23135.

Bin Wang, Chao Xu, Xiaomeng Zhao, Linke Ouyang, Fan Wu, Zhiyuan Zhao, Rui Xu, Kaiwen Liu, Yuan Qu, Fukai Shang, Bo Zhang, Liqun Wei, Zhihao Sui, Wei Li, Botian Shi, Yu Qiao, Dahua Lin, and Conghui He. Mineru: An open-source solution for precise document content extraction. *arXiv (Cornell University)*, 2024a.

Dongsheng Wang, Natraj Raman, Mathieu Sibue, Zhiqiang Ma, Petr Babkin, Simerjot Kaur, Yulong Pei, Armineh Nourbakhsh, and Xiaomo Liu. DocLLM: A layout-aware generative language model for multimodal document understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8529–8548, Bangkok, Thailand, 2024b. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.463. URL https://aclanthology.org/2024.acl-long.463/.

Weixuan Wang, Dongge Han, Daniel Madrigal Diaz, Jin Xu, Victor Rühle, and Saravan Rajmohan. Odysseybench: Evaluating llm agents on long-horizon complex office application workflows. *arXiv preprint arXiv:2508.09124*, 2025. doi: 10.48550/arXiv.2508.09124. URL https://arxiv.org/abs/2508.09124.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *Proceedings of the 41st International Conference on Machine Learning*, 2024c. URL https://proceedings.mlr.press/v235/wang24h.html.

Zilong Wang, Yuedong Cui, Li Zhong, Zimin Zhang, Da Yin, Bill Yuchen Lin, and Jingbo Shang. Officebench: Benchmarking language agents across multiple applications for office automation. *arXiv preprint arXiv:2407.19056*, 2024d. doi: 10.48550/arXiv.2407.19056. URL https://arxiv.org/abs/2407.19056.

Zilong Wang, Hao Zhang, Chun-Liang Li, Julian Martin Eisenschlos, Vincent Perot, Zifeng Wang, Lesly Miculicich, Yasuhisa Fujii, Jingbo Shang, Chen-Yu Lee, and Tomas Pfister. Chain-of-table: Evolving tables in the reasoning chain for table understanding. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024e. URL https://openreview.net/forum?id=4L0xnS4GQM.

Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022.

Haohan Weng, Danqing Huang, Yu Qiao, Zheng Hu, Chin-Yew Lin, Tong Zhang, and C. L. Philip Chen. Desigen: A pipeline for controllable design template generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. URL https://arxiv.org/abs/2403.09093.

Werk24. Werk24: AI feature extraction for technical drawings. https://werk24.io/, 2023. API that parses technical drawings to extract PMI, GD&T, tolerances, and threads as structured JSON.

Wix. Wix AI website builder. https://www.wix.com/ai-website-builder, 2024. Builds complete websites with business tools from a natural-language description via an AI agent.

Xianjie Wu, Jian Yang, Linzheng Chai, Ge Zhang, Jiaheng Liu, Xinrun Du, Di Liang, Daixin Shu, Xianfu Cheng, Tianzhen Sun, Guanglin Niu, Tongliang Li, and Zhoujun Li. TableBench: A comprehensive and complex benchmark for table question answering. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 25497–25506, 2025. doi: 10.1609/aaai.v39i24.34739. URL https://ojs.aaai.org/index.php/AAAI/article/view/34739.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems 37 (NeurIPS) Datasets and Benchmarks Track*, 2024. URL https://openreview.net/forum?id=tN61DTr4Ed.

Frank F. Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Z. Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, Mingyang Yang, Hao Yang Lu, Amaad Martin, Zhe Su, Leander Maksimov, Yujie Du, Qiuyu Liu, Yuekang Wang, Yueqi Tang, Robert Wang, Shuyan Yao, Ye Liu, Steven Y. Ou, Robert L. Russell, Frank Tang, Russ Salakhutdinov, Diyi Yang, Shunyu Yao, Ofir Press, and Graham Neubig. TheAgentCompany: Benchmarking LLM agents on consequential real world tasks, 2024. URL https://arxiv.org/abs/2412.14161.

Renjun Xu and Yang Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward, 2026. URL https://arxiv.org/abs/2602.12430.

Yang Xu, Yiheng Xu, Tengchao Lv, Lei Cui, Furu Wei, Guoxin Wang, Yijuan Lu, Dinei Florencio, Cha Zhang, Wanxiang Che, Min Zhang, and Lidong Zhou. LayoutLMv2: Multi-modal pre-training for visually-rich document understanding. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 2579–2591. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.acl-long.201. URL https://aclanthology.org/2021.acl-long.201/.

Y Combinator. StackAI: The enterprise AI transformation platform, 2023. URL https://www.ycombinator.com/companies/stackai. Y Combinator company profile, Winter 2023.

Y Combinator. Capacitive: AI agent building for knowledge workers, 2024a. URL https://www.ycombinator.com/launches/NdC-capacitive-ai-agent-building-for-knowledge-workers. Launch YC profile.

Y Combinator. Melder: AI formulas and document support for Microsoft Excel, 2024b. URL https://www.ycombinator.com/companies/melder. Y Combinator company profile, Fall 2024.

Y Combinator. Requests for startups: Make something agents want, 2025a. URL https://www.ycombinator.com/rfs. Y Combinator request for startups on agent-first software.

Y Combinator. Altrina: The SOP automation platform, 2025b. URL https://www.ycombinator.com/companies/altrina. Y Combinator company profile.

Y Combinator. Unsiloed AI: Api for parsing multimodal unstructured data, 2025c. URL https://www.ycombinator.com/companies/unsiloed-ai. Y Combinator company profile and launch page.

Y Combinator. Workflow automation startups funded by Y Combinator, 2026. URL https://www.ycombinator.com/companies/industry/workflow-automation. Startup directory snapshot accessed June 2026.

Yibo Yang, Fei Lei, Yixuan Sun, Yantao Zeng, Chengguang Lv, Jiancao Hong, Jiaojiao Tian, Tianyu Qiu, Xin Wang, Yanbing Chen, Yanjie Li, Zheng Pan, Xiaochen Zhou, Guanzhou Chen, Haoran Lv, Yuning Xu, Yue Ou, Haodong Liu, Shiqi He, Anya Jia, Yulei Xin, Huan Wu, Liang Liu, Jiaye Ge, Jianxin Dong, Dahua Lin, and Wenxiu Sun. Aidabench: Ai data analytics benchmark. *arXiv (Cornell University)*, 2026.

Yuhao Yang, Zhen Yang, Zi-Yi Dou, Anh Nguyen, Keen You, Omar Attia, Andrew Szot, Michael Feng, Ram Ramrakhya, Alexander Toshev, Chao Huang, Yinfei Yang, and Zhe Gan. Ultracua: A foundation model for computer use agents with hybrid action. *arXiv (Cornell University)*, 2025.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

Jiabo Ye, Anwen Hu, Haiyang Xu, Qinghao Ye, Ming Yan, Yuhao Dan, Chenlin Zhao, Guohai Xu, Chenliang Li, Junfeng Tian, Qian Qi, Ji Zhang, and Fei Huang. mPLUG-DocOwl: Modularized multimodal large language model for document understanding. *arXiv preprint arXiv:2307.02499*, 2023a. URL https://arxiv.org/abs/2307.02499.

Yunhu Ye, Binyuan Hui, Min Yang, Binhua Li, Fei Huang, and Yongbin Li. Large language models are versatile decomposers: Decomposing evidence and questions for table-based reasoning. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, pp. 174–184. ACM, 2023b. doi: 10.1145/3539618.3591708. URL https://doi.org/10.1145/3539618.3591708.

Thomson Yen, Julian Poeltl, Harshith Srinivas Gear, Yilin Meng, Joshua Fan, Adam Shen, Yili Liu, Ali Bauyrzhan, Siri Du, Haoyang Liu, Daniel Guetta, and Hongseok Namkoong. MBABench: Evaluating LLM agents on end-to-end spreadsheet tasks in finance. *arXiv preprint arXiv:2605.22664*, 2026. URL https://arxiv.org/abs/2605.22664.

Ori Yoran, Samuel Joseph Amouyal, Chaitanya Malaviya, Ben Bogin, Ofir Press, and Jonathan Berant. AssistantBench: Can web agents solve realistic and time-consuming tasks? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 8938–8968, Miami, Florida, USA, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.505. URL https://aclanthology.org/2024.emnlp-main.505/.

Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3911–3921, Brussels, Belgium, 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1425. URL https://aclanthology.org/D18-1425/.

Z.AI. GLM Slide/Poster Agent (beta) overview. https://docs.z.ai/guides/agents/slide, 2026. Accessed: 2026-05-21.

Z.ai. Z.ai: Inspiring AGI to benefit humanity. https://www.zhipuai.cn/en, 2026. Official Zhipu AI/Z.ai homepage used as a stable source for this product-family reference.

Zapier. Zapier Agents: Autonomous AI teammates. https://zapier.com/agents, 2025. AI agents that act as goal-oriented teammates connecting to thousands of apps to build and run workflows.

Wenzheng Zeng, Mingyu Ouyang, Langyuan Cui, and Hwee Tou Ng. SlideTailor: Personalized presentation slide generation for scientific papers. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 34584–34592, 2026. doi: 10.1609/aaai.v40i41.40758. URL https://arxiv.org/abs/2512.20292.

Chaoyun Zhang, Liqun Li, Shilin He, Xu Zhang, Bo Qiao, Si Qin, Minghua Ma, Yu Kang, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. Ufo: A ui-focused agent for windows os interaction, 2024a. URL https://arxiv.org/abs/2402.07939.

Chaoyun Zhang, He Huang, Chiming Ni, Jian Mu, Si Qin, Shilin He, Lu Wang, Fangkai Yang, Pu Zhao, Chao Du, Liqun Li, Yu Kang, Zhao Jiang, Suzhen Zheng, Rujia Wang, Jiaxu Qian, Minghua Ma, Jian-Guang Lou, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. Ufo2: The desktop agentos. *arXiv (Cornell University)*, 2025a.

Le Zhang, Yixiong Xiao, Xinjiang Lu, Jingjia Cao, Yusai Zhao, Jingbo Zhou, Lang An, Zikan Feng, Wanxiang Sha, Yu Shi, Congxi Xiao, Jian Xiong, Yankai Zhang, Hua Wu, and Haifeng Wang. Omegause: Building a general-purpose gui agent for autonomous task execution. *arXiv preprint arXiv:2601.20380*, 2026. URL https://arxiv.org/abs/2601.20380.

Tianshu Zhang, Xiang Yue, Yifei Li, and Huan Sun. TableLlama: Towards open large generalist models for tables. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 6024–6044. Association for Computational Linguistics, 2024b. doi: 10.18653/v1/2024.naacl-long.335. URL https://aclanthology.org/2024.naacl-long.335/.

Wenqi Zhang, Yongliang Shen, Zeqi Tan, Guiyang Hou, Weiming Lu, and Yueting Zhuang. Data-copilot: Bridging billions of data and humans with autonomous workflow. *arXiv preprint arXiv:2306.07209*, 2023. URL https://arxiv.org/abs/2306.07209.

Zhilin Zhang, Xiang Zhang, Jiaqi Wei, Yiwei Xu, and Chenyu You. PosterGen: Aesthetic-aware paper-to-poster generation via multi-agent LLMs. *arXiv preprint arXiv:2508.17188*, 2025b. URL https://arxiv.org/abs/2508.17188. Accepted to CVPR 2026 Findings.

Wei Zhao, Zhitao Hou, Siyuan Wu, Yan Gao, Haoyu Dong, Yao Wan, Hongyu Zhang, Yulei Sui, and Haidong Zhang. Nl2formula: Generating spreadsheet formulas from natural language queries. In *Findings of the Association for Computational Linguistics: EACL 2024*, pp. 2377–2388. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-eacl.158. URL https://aclanthology.org/2024.findings-eacl.158/.

Hao Zheng, Xinyan Guan, Hao Kong, Wenkai Zhang, Jia Zheng, Weixiang Zhou, Hongyu Lin, Yaojie Lu, Xianpei Han, and Le Sun. PPTAgent: Generating and evaluating presentations beyond text-to-slides. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 14413–14429, Suzhou, China, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.emnlp-main.728. URL https://aclanthology.org/2025.emnlp-main.728/. Introduces both the PPTAgent system and the PPTEval evaluation framework.

Hao Zheng, Guozhao Mo, Xinru Yan, Qianhao Yuan, Wenkai Zhang, Xuanang Chen, Yaojie Lu, Hongyu Lin, Xianpei Han, and Le Sun. Deeppresenter: Environment-grounded reflection for agentic presentation generation. *arXiv (Cornell University)*, 2026.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems 36*, 2023. doi: 10.48550/arXiv.2306.05685. URL https://arxiv.org/abs/2306.05685.

Zhipu AI. AutoGLM and AutoClaw: Browser and desktop agents on the GLM foundation. https://www.zhipuai.cn/, 2026. AutoClaw packages a local agent client with a built-in office-automation skill library.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2023.

Ruiyan Zhu, Xi Cheng, Ke Liu, Brian Zhu, Daniel Jin, Neeraj Parihar, Zhoutian Xu, and Oliver Gao. Sheetmind: An end-to-end llm-powered multi-agent framework for spreadsheet automation. *arXiv (Cornell University)*, 2025.

Mingchen Zhuge, Changsheng Zhao, Dylan Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, Yangyang Shi, Vikas Chandra, and Jürgen Schmidhuber. Agent-as-a-judge: Evaluate agents with agents. *arXiv preprint arXiv:2410.10934*, 2024. doi: 10.48550/arXiv.2410.10934. URL https://arxiv.org/abs/2410.10934.

Anni Zou, Wenhao Yu, Hongming Zhang, Kaixin Ma, Deng Cai, Zhuosheng Zhang, Hai Zhao, and Dong Yu. Docbench: A benchmark for evaluating llm-based document reading systems. In *Proceedings of the 4th International Workshop on Knowledge-Augmented Methods for Natural Language Processing*, pp. 359–373, 2025.

A. Crosswalk of Representative Work

Table 4 lists representative office-agent work along the four axes used in the main text (artifact, operation surface in §3, evaluation target in §4, and training signal in §5), plus a *source* column separating peer-reviewed publications, arXiv preprints, and product or protocol documentation. The table is curated to be representative rather than exhaustive; it is the visible artifact of the annotation procedure described in §1. Rows should therefore be read as role annotations within the survey framework, not as claims of equal evidentiary strength.

Entries are organized by artifact family; the three middle columns align with the dimensions of §3–§5 (operation surface, evaluation target, training signal), so each row locates a system within the framework. For product and protocol rows, these columns summarize publicly documented or reported capabilities rather than independently evaluated performance.

Table 4 | Crosswalk of representative office-agent work along the axes used in this survey: artifact class, operation surface (§3), evaluation target (§4), training signal (§5), and source type. "Source" separates peer-reviewed publications, arXiv preprints, and product / protocol documentation; the last is treated as motivating evidence, not as load-bearing technical claims. Product rows summarize publicly reported capabilities rather than controlled evaluations.

Work	Artifact	Op. surface	Evaluation target	Training signal	Source
Multi-application office workflows					
OfficeBench (Wang et al., 2024d)	Multi	GUI + API	Workflow success, package state	None (benchmark)	arXiv
OdysseyBench (Wang et al., 2025)	Multi	GUI + API	Long-horizon workflow success	None (benchmark)	arXiv
TheAgentCompany (Xu et al., 2024)	Multi (know. work)	GUI + API	Task completion in enterprise env.	None (benchmark)	arXiv
Microsoft 365 Copilot (Microsoft, 2023)	Multi	API + GUI	N/A (product)	N/A	Product
Microsoft Multi-Agent (Microsoft 365 Blog, 2025b)	Multi	API + harness	Cross-artifact delegation	N/A	Product
Spreadsheets					
SheetCopilot (Li et al., 2023a)	Spreadsheet	API (atomic)	Execution + state match	SFT (demonstr.)	NeurIPS
SheetAgent (Chen et al., 2025)	Spreadsheet	Code + API	Execution + reasoning trace	None (system)	WWW
SpreadsheetBench (Ma et al., 2024b)	Spreadsheet	Code + API	Real-world manipulation	None (benchmark)	NeurIPS D&B
SpreadsheetBench 2 (Ma et al., 2026)	Spreadsheet	Code + API	End-to-end workflow (cell match + chart rubric)	None (benchmark)	Dataset
SpreadsheetLLM (Dong et al., 2024)	Spreadsheet	Representation	Understanding	SFT	arXiv
NL2Formula (Zhao et al., 2024)	Spreadsheet	Code (formula)	Formula correctness	SFT	arXiv
SpreadsheetCoder (Chen et al., 2021b)	Spreadsheet	Code (formula)	Formula generation	SFT	ACL
InstructExcel (Payan et al., 2023)	Spreadsheet	Code + API	Instruction-following edits	SFT	EMNLP
Flash Fill (Gulwani, 2011)	Spreadsheet	Program synth.	Pattern-based autofill	Symbolic	POPL
Spreadsheet-RL (Chi et al., 2026)	Spreadsheet	Code	Manipulation correctness	RL (online)	arXiv
Tables and structured data (adjacent)					
Spider (Yu et al., 2018)	Table / SQL	Code (SQL)	Query correctness	None (benchmark)	EMNLP
BIRD (Li et al., 2023b)	Table / SQL	Code (SQL)	Execution accuracy	None (benchmark)	NeurIPS
Chain-of-Table (Wang et al., 2024e)	Table	Code-interleaved	Reasoning accuracy	None (method)	ICLR
TableLLaMA (Zhang et al., 2024b)	Table	Code + reading	Table understanding	SFT	arXiv
TAPAS (Herzig et al., 2020)	Table	Reading	Table QA	Pretrain + SFT	ACL
TableGPT (Li et al., 2024)	Table	Code + API	Table manipulation	SFT	arXiv
TableBench (Wu et al., 2025)	Table	Code	Comprehensive table reasoning	None (benchmark)	ICLR
Slides / Presentations					
PPTAgent / PPTEval (Zheng et al., 2025)	Slides	Code + API	Multi-aspect deliverable quality	SFT (demonstr.)	EMNLP
AutoPresent (Ge et al., 2025)	Slides	Code	Structured slide design	SFT	CVPR
Talk-to-Slides (Jung et al., 2025)	Slides	API (edit ops)	Edit correctness	None (system)	arXiv
AeSlides (Pan et al., 2026)	Slides	Code	Aesthetic layout (verifiable)	RL (GRPO)	arXiv
PPTC (Guo et al., 2024)	Slides	API (PPT ops)	Multi-turn task completion	None (benchmark)	ACL Find.

Work	Artifact	Op. surface	Evaluation target	Training signal	Source
PosterGen (Zhang et al., 2025b)	Posters	Code	Layout + content quality	SFT	arXiv
SlideTailor (Zeng et al., 2026)	Slides	API	Edit & reuse fidelity	None (system)	arXiv
GLM Slide Agent (Z.AI, 2026; Team et al., 2026a)	Slides	Code + harness	Near-ready artifact quality	SFT + RL + RS	Tech rep.
SlideRL (Ananda Kumar & Arunachalam, 2026)	Slides	Code	Presentation quality	RL (GRPO)	arXiv
Doc2PPT (Fu et al., 2022)	Slides	Code	Doc-to-slide generation	SFT	ECCV
D2S (Sun et al., 2021)	Slides	Code	Doc-to-slide conversion	SFT	AAAI
Text2Chart3.1 (Pesaran Zadeh et al., 2024)	Charts	Code	Chart generation from text	SFT	ACL
Documents / PDFs					
DocBench (Zou et al., 2025)	Document / PDF	Reading	User-centric reading questions	None (benchmark)	arXiv
MMLongBench-Doc (Ma et al., 2024a)	PDF (long)	Reading	Multi-page understanding	None (benchmark)	NeurIPS D&B
DocLLM (Wang et al., 2024b)	Document	Reading	Layout-aware understanding	Pretraining	arXiv
mPLUG-DocOwl2 (Hu et al., 2025a)	Document	Reading	High-res / multi-page underst.	SFT	arXiv
CoEdIT (Raheja et al., 2023)	Document	Diffable edits	Edit instruction following	SFT	EMNLP
PEER (Schick et al., 2022)	Document	Diffable edits	Collaborative editing	SFT	arXiv
LayoutLMv2 (Xu et al., 2021)	Document	Reading	Layout-aware pretraining	Pretraining	ACL
LayoutLMv3 (Huang et al., 2022)	Document	Reading	Unified text + layout + image	Pretrain + SFT	ACM MM
DocVQA (Mathew et al., 2021)	Document	Reading	Extractive document QA	None (benchmark)	WACV
MP-DocVQA (Tito et al., 2023)	PDF (multi-page)	Reading	Multi-page document QA	None (benchmark)	ICDAR
PDFTriage (Saad-Falcon et al., 2023)	PDF	Reading + routing	Question routing over structure	SFT	arXiv
GUI / OS / Computer-use (adjacent)					
OSWorld (Xie et al., 2024)	OS (incl. office)	GUI	Execution-based interface tasks	None (benchmark)	NeurIPS D&B
WorkArena (Drouin et al., 2024)	Enterprise web	GUI	Knowledge-work tasks	None (benchmark)	ICML
WindowsAgentArena (Bonatti et al., 2024)	Windows OS	GUI	OS-level tasks	None (benchmark)	arXiv
WebArena (Zhou et al., 2023)	Web	GUI	Web-based task completion	None (benchmark)	ICLR
Mind2Web (Deng et al., 2023)	Web	GUI	Web agent generalization	SFT	NeurIPS
UI-TARS (Qin et al., 2025)	GUI (general)	GUI (vision)	GUI grounding / action	SFT + RL	arXiv
ComputerRL (Lai et al., 2025)	Computer use	GUI + API	Online RL on computer tasks	RL (online)	arXiv
DART (Li et al., 2025)	GUI (general)	GUI	OSWorld-style benchmarks	RL (async)	arXiv
SeeClick (Cheng et al., 2024)	GUI (screenshot)	GUI	Action grounding	SFT	ACL
CogAgent (Hong et al., 2024)	GUI (general)	GUI (vision)	GUI understanding + grounding	SFT	CVPR
Evaluation methods					
G-Eval (Liu et al., 2023)	Any	LLM judge	NLG evaluation via CoT	None (method)	ICLR
LLM-as-Judge (Zheng et al., 2023)	Any	LLM judge	Pairwise / scoring with LLMs	None (method)	NeurIPS
Prometheus (Kim et al., 2024)	Any	LLM judge	Fine-grained rubric evaluation	SFT	ICLR
Agent-as-a-Judge (Zhuge et al., 2024)	Multi	Agentic judge	Process + artifact inspection	None (method)	arXiv
MLLM-as-Judge (Chen et al., 2024)	Visual	MLLM judge	Multimodal judging + bias	None (analysis)	arXiv
Harness, skills, protocols (engineering / product)					
Claude Code (Anthropic, 2025b)	Code + multi	Harness	Adjacent runtime pattern	N/A	Product
Codex CLI (OpenAI, 2025c)	Code + multi	Harness	Adjacent runtime pattern	N/A	Product
ChatGPT Agent (OpenAI, 2025a)	Multi	Harness + GUI	Office deliverable runtime	N/A	Product
Anthropic Skills (Anthropic, 2025d,e)	DOCX, PPTX, XLSX, PDF	Skill loader	N/A (capability layer)	N/A	Product
MCP (Anthropic, 2024b)	Any (tools)	Protocol	N/A	N/A	Protocol
A2A (Google Developers Blog, 2025)	Multi-agent	Protocol	N/A	N/A	Protocol
Google Workspace (Google Workspace, 2025a; Google, 2025)	Multi	API + GUI	N/A (product)	N/A	Product
Notion AI (Notion Labs, 2023)	Document + multi	API	N/A (product)	N/A	Product
Copilot Tuning (Microsoft Learn, 2026b)	Multi	Adaptation	Local rubrics / examples	SFT + adapt.	Product

Work	Artifact	Op. surface	Evaluation target	Training signal	Source
System-scale training					
GLM-5 (Team et al., 2026a)	Multi	Code + harness	Multi-stage benchmark	SFT→RL	Tech rep.
Kimi K2.5 / PARL (Team et al., 2026b)	Multi	Code + harness	Multi-agent task delegation	Multi-agent RL	Tech rep.
DaCode (Huang et al., 2024)	Code/data (adj.)	Code	Executable data tasks	SFT	arXiv

B. Deployed Office Systems in the Product Landscape

Table 5 enumerates the curated systems plotted in Figure 9, grouped by the four archetypes (plus the grey vertical-tool band) used there. Each entry gives a primary reference—an official product page or launch announcement (or, for research prototypes, the cited paper). As in the crosswalk (Table 4), product documentation is treated as motivating evidence rather than as a load-bearing technical claim. Product names are used for identification only; their inclusion does not imply endorsement, ranking, market coverage, or controlled capability comparison. A dash (—) marks a system for which we could not identify a stable primary source.

Table 5 | Curated deployed office systems shown in Figure 9 (≈95 products), grouped by archetype, each with a primary reference (95 of 95 sourced). The roster supports the architectural discussion in §6; it is not a market-size estimate, ranking, or controlled capability comparison.

Product	Primary reference
General super-agents	
OpenAI Codex	OpenAI (2025c)
Manus	Manus (2026)
Claude Code	Anthropic (2025b)
ChatGPT Agent	OpenAI (2025a)
Gemini / Deep Research	Team (2023)
Kimi swarm	Moonshot AI (2026)
Quark	Alibaba Group (2025)
GLM / AutoGLM	Zhipu AI (2026)
Lindy	Lindy AI (2023)
Zapier Agents	Zapier (2025)
Perplexity Comet	Perplexity AI (2025)
Tencent Yuanqi	Tencent (2024)
QClaw	Tencent (2026)
AutoClaw	Z.ai (2026)
Agentic workbenches	
Wukong	Alibaba Group (2026b)
Claude Cowork	Anthropic (2026a)
QoderWork	Qoder (2025)
Qoder	Qoder (2025)
Replit Agent	Replit (2024)
M365 Copilot Agents	Microsoft 365 Blog (2025b)
Lovable	Lovable (2024)
Cursor	Cursor (2025)
WorkBuddy	Tencent Cloud (2026)
Devin	Cognition (2024)
Knowledge / content layer	
Microsoft Copilot	Microsoft (2023)
Claude Artifacts	Anthropic (2025c)
Google Gemini	Google Workspace (2025a)
Qwen	Alibaba Group (2026a)
Genspark	Genspark (2025)
Doubao	ByteDance (2023)
Granola	Granola (2023)
Baidu Wenku	Baidu (2023)
Notion AI	Notion Labs (2023)
Poe	Quora (2023)
NotebookLM	Google (2023b)
Creative-pipeline tools	
HeyGen	HeyGen (2025)

Product	Primary reference
Julius	Julius AI (2024)
Runway	Runway (2025)
Gamma	Gamma (2025)
Canva AI	Canva (2025)
Bolt.new	StackBlitz (2024)
ElevenLabs	ElevenLabs (2023)
Synthesia	Synthesia (2024)
Claude Design	Anthropic (2026d)
Quadratic	Quadratic (2025)
OpusClip	OpusClip (2024)
Krea	Krea AI (2024)
MiniMax	MiniMax (2026)
Jimeng / CapCut	ByteDance (2024b)
Framer AI	Framer (2023)
DataLab	DataCamp (2024)
Tableau Agent	Tableau (2024)
Wix AI Website Builder	Wix (2024)
CapCut AI	ByteDance (2024a)
Tongyi Wan	Alibaba Cloud (2023a)
Kling AI	Kuaishou Technology (2024)
Figma Make	Figma (2025)
Descript	Descript (2024)
Rows AI	Rows (2024)
Hex Magic	Hex Technologies (2023)
Power BI Copilot	Microsoft (2024)
Airtable AI	Airtable (2025)
Read.ai	Read AI (2023)
Pika	Pika Labs (2023)
Fireflies.ai	Fireflies.ai (2023)
Uizard	Uizard (2023)
Adobe Firefly / Photoshop	Adobe (2023)
Otter.ai	Otter.ai (2023)
Meitu / WHEE	Meitu (2023)
Recraft	Recraft (2024)
Napkin AI	Napkin AI (2024)
Suno	Suno (2024)
Photoroom	Photoroom (2023)
Tongyi Tingwu	Alibaba Cloud (2023b)
Beautiful.ai	Beautiful.ai (2023)
Miaoya	Miaoya (2023)
Ideogram	Ideogram (2023)
Flourish	Flourish (2022)
Midjourney	Midjourney (2022)
Vertical professional tools (outside office scope)	
Harvey	Harvey (2023)
Autodesk Forma	Autodesk (2023)
Spellbook	Spellbook (2023)
Lexis+ Protege	LexisNexis (2024)
Werk24	Werk24 (2023)
Fiscal.ai	Fiscal.ai (2025)
LegalOn	LegalOn Technologies (2023)
Ironclad AI	Ironclad (2023)
Autodesk Fusion	Autodesk (2024)
Cleo	Cleo AI (2023)
Onshape AI Advisor	PTC (2025)
TestFit	TestFit (2023)
SolidWorks AI	Dassault Systèmes (2026)
FinchatAI	FinchatAI (2023)
Monarch Money	Monarch Money (2024)
nTop	nTop (2023)

C. Representative Work by Operation Surface

Table 6 expands the operation taxonomy used in §3. Rather than treating citations as interchangeable examples, it states what each cluster contributes to AI Office: the action surface it opens, the artifact constraint it makes tractable, or the runtime capability it motivates. The table is intentionally organized by capability family, so a single work may also appear in the master crosswalk when it contributes to evaluation or training.

Table 6 | Representative work grouped by operation surface and office artifact class. Each row describes how the cited line of work is used in the survey's argument, not merely where it was published.

Scope	Role in the operation taxonomy
Code	
Code generation and code-as-action	These works establish code as both a reasoning medium and an executable action language: models can synthesize programs, call external runtimes, inspect failures, and repair outputs before committing artifact changes (Chen et al., 2021a; Gao et al., 2023; Chen et al., 2022; Wang et al., 2024c; Jimenez et al., 2024b,a).
Spreadsheets, formulas, and workbooks	Spreadsheet systems show why office artifacts are not plain text: the agent must recover table structure, generate formulas or scripts, preserve workbook state, and verify results through executable cells and transformations (Li et al., 2023a; Chen et al., 2025, 2021b; Payan et al., 2023; Zhao et al., 2024; Singh et al., 2023; Dong et al., 2024; Ma et al., 2024b; Wu et al., 2025).
Tables, SQL, and code-interleaved reasoning	Table and SQL work contributes structured reasoning patterns for selecting schema elements, executing queries, and combining natural-language reasoning with symbolic operations; these capabilities transfer directly to spreadsheet and report-generation workflows (Yu et al., 2018; Wang et al., 2024e; Li et al., 2023b; Herzig et al., 2020; Liu et al., 2021; Zhang et al., 2024b; Li et al., 2024; Su et al., 2024; Jiang et al., 2023; Cheng et al., 2022; Ye et al., 2023b; Li et al., 2023c).
Slides, posters, and structured object models	Slide and poster systems treat presentation files as object graphs rather than image canvases, making layout, reuse, and editing possible through structured operations over placeholders, text boxes, charts, and visual hierarchy (Zheng et al., 2025; Ge et al., 2025; Jung et al., 2025; Zeng et al., 2026; Zhang et al., 2025b; Fu et al., 2022; Sun et al., 2021; PptxGenJS, 2026).
Charts and layout specifications	Chart and layout-generation work supplies the intermediate representations needed when an office deliverable must communicate quantitative claims visually, rather than merely compute correct numbers (Pesaran Zadeh et al., 2024; Feng et al., 2023; Lin et al., 2023).
Documents and diffable edits	Document-editing systems motivate treating text artifacts as persistent states with typed edits, revision history, and local constraints, rather than as one-shot generated passages (Raheja et al., 2023; Schick et al., 2022).
API / tool use	
Tool-use foundations and typed interfaces	Tool-use work defines the contract between a language model and external capabilities: typed function signatures, structured returns, routing among tools, and protocol-level governance that office agents inherit when calling file, calendar, email, or enterprise APIs (Schick et al., 2023; Qin et al., 2024; OpenAI, 2026a; Anthropic, 2024b; Patil et al., 2023; Shen et al., 2023; OpenAI, 2025e).
Office connectors and governed workflows	Enterprise connectors show the platform side of office agency: the agent must retrieve approved data, respect permissions, trigger workflows, and operate inside administrative boundaries rather than only manipulate local files (Microsoft, 2023; Google Workspace, 2025a; Microsoft 365 Blog, 2024; Google, 2023a).
Vertical authoring assistants	Authoring assistants demonstrate product-level verbs for writing, rewriting, designing, summarizing, and exporting deliverables; they supply examples of bounded office capabilities that broader agents can orchestrate (Notion Labs, 2023; Z.AI, 2026; Gamma, 2023; Tome App AI, 2023; Beautiful.ai, 2023; Adobe, 2024; Grammarly, 2024).
Inter-agent capability publishing	Agent-to-agent protocols motivate a future office runtime where specialist agents publish reusable capabilities and negotiate task boundaries across document, sheet, slide, and workflow services (Google Developers Blog, 2025).
GUI / OS / computer-use	
Web, desktop, and enterprise-agent benchmarks	GUI benchmarks expose the outer control layer of office work: the agent acts through windows, menus, forms, and enterprise applications, and success is measured by the resulting state of the environment (Xie et al., 2024; Drouin et al., 2024; Deng et al., 2023; Zhou et al., 2023; Koh et al., 2024; He et al., 2024; Yoran et al., 2024; Bonatti et al., 2024).
Vision-grounded GUI agents	Vision-grounded agents contribute screenshot-to-action grounding, element localization, and action prediction, which are necessary when an office task depends on rendered UI state or lacks complete API exposure (Cheng et al., 2024; Qin et al., 2025; Hong et al., 2024; Lin et al., 2025; Lu et al., 2024; Li et al., 2025; Zhang et al., 2026).
Production computer-use systems and surveys	Production systems and surveys mark computer use as an emerging general interface layer; in this survey, they are adjacent evidence for why office agents need rendered-state verification and OS-level recovery, not a replacement for artifact-specific methods (Zhang et al., 2024a; Anthropic, 2024a; Agashe et al., 2024; OpenAI, 2025d; Hu et al., 2025b; Nguyen et al., 2025; Lu et al., 2025).
Harness / skills	
General harnesses and skill loading	General harnesses provide the planning, tool dispatch, workspace memory, and validation loops that turn single calls into long-horizon workflows; skill loading makes these capabilities portable and selectively available (Anthropic, 2025b; OpenAI, 2025c; Anthropic, 2025d).
Office skill libraries and portability	Office skill libraries demonstrate how artifact-specific know-how can be packaged as instructions, scripts, and assets for DOCX, PPTX, XLSX, and PDF tasks, allowing the same agent runtime to compose file operations across formats (Anthropic, 2025e; MiniMax, 2026; Cursor, 2025; OpenCode Contributors, 2025).
Model–harness co-evolution and office evaluation	These systems and benchmarks show that improved base models do not eliminate the runtime: stronger models still need harnesses for validation, versioning, multi-step recovery, and package-level evaluation (Team et al., 2026b,a; Lai et al., 2025; OpenAI, 2025a; Wang et al., 2024d, 2025).
Cross-format and multi-artifact workflows	Multi-agent workflow products motivate the package-level setting emphasized in this survey, where a final answer may require a consistent workbook, slide deck, document, and message rather than a single edited file (Microsoft 365 Blog, 2025b).

Scope	Role in the operation taxonomy
Office-specialized runtimes and artifact-aware validation	Office-specialized runtimes combine general harness patterns with artifact validators: they open files, execute formulas, render decks, inspect layouts, and use organization-specific templates or rubrics as part of the loop (OpenAI, 2025a; Microsoft 365 Blog, 2025a; Google Workspace, 2025b; Chen et al., 2025; Zheng et al., 2025; Pan et al., 2026; Microsoft Learn, 2026b).

D. Representative Work by Evaluation Target

Table 7 expands the evaluation taxonomy used in §4. The upper section lists *benchmarks* that define evaluation instances (artifact, goal, and acceptance criteria); the lower section lists *evaluation protocols* that define how a judge scores the resulting deliverable. A single work may appear in both sections when it contributes both a task suite and a scoring method (e.g., PPTAgent supplies PPTEval). The descriptions state how each work informs this survey's evaluation lens; they should not be read as claiming that every work fully implements deliverable-level certification.

Table 7 | Representative work grouped by evaluation role. Each row describes how the cited work informs the evaluation argument of §4, not merely where it was published; some descriptions are survey interpretations of relevance rather than claims that the cited work fully solves office-deliverable evaluation.

Scope	Role in the evaluation taxonomy
Benchmarks: multi-application and long-horizon	
OfficeBench (Wang et al., 2024d)	Evaluates agents that coordinate spreadsheet analysis, document drafting, and email dispatch within a single task; success requires package-level consistency across files, not just per-artifact correctness.
OdysseyBench (Wang et al., 2025)	Extends multi-application evaluation to long-horizon workflows, testing whether agents maintain cross-artifact coherence across many steps and tool invocations.
TheAgentCompany (Xu et al., 2024)	Embeds office tasks in a simulated enterprise environment with organizational context, where the unit of evaluation is long-horizon knowledge work, not isolated tool calls.
AgentBench (Liu et al., 2024)	A multi-domain agent benchmark that includes office-adjacent tasks (database, web); it motivates workspace-transition evaluation by scoring cumulative environment state rather than single-turn responses.
Benchmarks: spreadsheets and tables	
SpreadsheetBench (Ma et al., 2024b)	Supplies real-world spreadsheet manipulation tasks where $\mathcal{V}$ reduces to formula evaluation and output-cell comparison, making it the most directly checkable artifact type.
SpreadsheetBench 2 (Ma et al., 2026)	Extends spreadsheet evaluation from isolated manipulation to end-to-end business workflows (financial modeling, debugging, visualization) over multi-sheet workbooks; $\mathcal{V}$ combines exact cell-level matching (all required cells changed, all others untouched) with rubric-based vision-language judging of generated charts, probing package-level workspace transitions rather than a single output value.
Spider (Yu et al., 2018)	A cross-database SQL benchmark that established execution-based evaluation for structured-data reasoning; its evaluation paradigm (comparing query outputs, not query strings) transfers directly to spreadsheet formula verification.
BIRD (Li et al., 2023b)	Advances SQL evaluation with large, dirty real-world databases; highlights the gap between syntactically valid queries and semantically correct results on messy data, paralleling spreadsheet robustness concerns.
TableBench (Wu et al., 2025)	A comprehensive table reasoning benchmark covering fact verification, data analysis, and chart generation; it tests whether models can reason over structured data across multiple reasoning steps.
ChartQA (Masry et al., 2022)	A question-answering benchmark over charts requiring visual and logical reasoning; representative of the QA/comprehension cluster whose benchmark coverage we contrast with online demand in §4.4.
Benchmarks: slides and presentations	
PPTAgent / PPTEval (Zheng et al., 2025)	Contributes both a generation system and a multi-aspect evaluation framework (PPTEval) that scores content, design, and coherence at element, slide, and deck levels, operationalizing $\mathcal{V}$ for presentation artifacts.
PPTC (Guo et al., 2024)	Exposes the gap between single-turn and session-level slide editing success, demonstrating that per-action correctness does not guarantee multi-turn task completion.
AeSlides (Pan et al., 2026)	Translates low-level layout aesthetics (aspect ratio, whitespace, collision, balance) into verifiable evaluation criteria, showing that some subjective slide quality dimensions can be operationalized as deterministic checks.
AutoPresent (Ge et al., 2025)	Evaluates structured slide design from research materials, combining content fidelity with layout quality metrics across the full deck.
Benchmarks: documents and PDFs	
DocBench (Zou et al., 2025)	User-centric document reading benchmark with questions requiring integration of text, table, and figure evidence; highlights provenance and citation validity as evaluation dimensions.
MMLongBench-Doc (Ma et al., 2024a)	Tests multi-page, multimodal document understanding at lengths that stress context windows, demonstrating that $\Omega(d_t)$ quality degrades with document length.
DocVQA (Mathew et al., 2021)	Foundational extractive document QA benchmark; establishes the evaluation paradigm for reading comprehension over visually rich documents, a prerequisite for verifying document edits.
MP-DocVQA (Tito et al., 2023)	Extends DocVQA to multi-page settings where answers span across pages, testing whether models can maintain coherent understanding across document boundaries.
PDFTriage (Saad-Falcon et al., 2023)	Evaluates question routing over PDF structure, testing whether models can identify which structural elements (text, table, figure) contain relevant evidence for provenance verification.
LongBench (Bai et al., 2024b)	Long-context evaluation benchmark spanning multiple tasks; provides evidence for how context length affects document understanding quality across different reading strategies.
Benchmarks: GUI and computer-use (adjacent)	
OSWorld (Xie et al., 2024)	The broadest desktop-agent benchmark, including office applications; it scores interface-level task success but not the internal validity of the resulting office files, illustrating the boundary between GUI evaluation and deliverable evaluation.
WorkArena (Drouin et al., 2024)	Enterprise web benchmark on ServiceNow; tests knowledge-work tasks within organizational systems, bridging GUI evaluation and office workflow evaluation.
WebArena (Zhou et al., 2023)	Realistic web-based task benchmark that established environment-state evaluation for web agents; its evaluation paradigm (checking page state, not action sequences) informs workspace-transition scoring.

Scope	Role in the evaluation taxonomy
Evaluation protocols and methods	
G-Eval (Liu et al., 2023)	Proposes using LLM chain-of-thought for NLG evaluation with fine-grained criteria, establishing the paradigm of rubric-conditioned LLM judges that office evaluation adapts for deliverable scoring.
LLM-as-a-Judge (Zheng et al., 2023)	Systematically validates LLM-based pairwise and scoring evaluation against human preferences, providing the methodological foundation for scalable office deliverable assessment.
Prometheus (Kim et al., 2024)	A fine-grained evaluation model trained on rubric-based scoring, demonstrating that evaluation criteria can be made explicit and reproducible, which is a requirement for artifact-level quality judgments.
Agent-as-a-Judge (Zhuge et al., 2024)	Extends evaluation from response scoring to process-and-artifact inspection: the judge opens files, executes code, and traces intermediate states, mirroring how human reviewers verify office deliverables.
MLLM-as-a-Judge (Chen et al., 2024)	Analyzes biases in multimodal LLM judges (position, verbosity, scoring instability), providing calibration evidence essential for rendered-output evaluation of slides and documents.
Copilot Tuning (Microsoft Learn, 2026b)	Treats organizational examples, comments, and approval habits as evaluation rubrics, operationalizing the observation that the same artifact is judged differently across organizations.

E. Online Office Query Intent Analysis

This appendix provides the demand-side evidence behind the benchmark–deployment distribution gap discussed in §4.4. We analyze three task families from online AI Office query logs: spreadsheet-oriented Excel tasks, Doc/PDF document tasks, and presentation-oriented PPT tasks. The available exports contain 5,661 Excel conversations, 3,063 Doc/PDF conversations, and 3,084 PPT conversations. After removing rows with invalid or missing labels, the analysis uses 11,155 successfully labeled conversations: 5,656 Excel, 3,054 Doc/PDF, and 2,445 PPT. All percentages in the following figures are computed within each task family rather than over the union of all tasks. The exports do not include a reliable collection window, so we report these distributions as observed online-query evidence rather than as a complete estimate of all AI Office demand. The query-log statistics are aggregate and de-identified. We do not analyze raw user prompts, uploaded files, or user-level behavior in this paper. Because the exports reflect a particular product surface, user population, classifier, and time window, the numbers should be read as descriptive evidence for demand-aware benchmark design, not as a market-wide estimate of AI Office usage. The raw conversations are not released because they may contain private user content; the appendix therefore reports only aggregate counts, normalized label definitions, and within-family percentages. This limits external reproducibility of the raw classification step, but makes the aggregation logic auditable: every headline percentage in §4.4 is derived from the denominators above and the normalized labels described below.

E.1. Data Processing and Label Normalization

The analysis starts from existing classification outputs rather than from raw unstructured chat logs. For Excel and Doc/PDF, each conversation is assigned a dominant dialogue intent, an operation-semantics label, a business-scene label, task-span and ambiguity labels, and a verification-risk label. For PPT, the export contains presentation-specific labels: operation intent, PPT action, input modality, output expectation, content domain, knowledge dependency, complexity, interaction mode, and attachment metadata. Because these classifiers were designed for different software families, we do not force them into a single shared ontology for every chart. Instead, each task family is visualized with its native labels where they are most informative, while cross-family concepts such as user intent and business context are normalized into English display labels. The goal is descriptive rather than causal: the figures characterize what online users ask for, not whether a model can satisfy those requests. We apply four normalization rules before computing the cross-family summary. First, rows with null, malformed, or out-of-vocabulary primary labels are removed from the denominator, which accounts for the gap between exported rows and successfully labeled conversations. Second, labels are mapped only when their semantics are comparable across families: for example, Excel direct execution, Doc/PDF generation/revision/conversion, and PPT deck creation are kept separate in the family dashboards but grouped into broader task categories only in Table 2. Third, weighted online shares are computed by conversation count across the three valid family denominators, not by averaging family percentages equally. Fourth, verification-risk labels are reported only for Excel and Doc/PDF because the PPT export uses a different label schema. These choices make the analysis conservative: it preserves native family structure where possible and uses cross-family aggregation only for the benchmark-gap argument.

Mapping to cross-family task categories. The six task categories in Table 2 are an explicit, coarse aggregation of the family-native labels, computed by conversation count over the three valid denominators. *Direct execution* aggregates operation-level edits that transform an existing artifact in place: Excel direct-operation requests (table structuring, data cleaning/transformation, formula writing) plus the operation-level edit, conversion, and restructuring requests in Doc/PDF. *Artifact generation* aggregates create-from-scratch or create-from-source requests: PPT deck generation, Doc/PDF deliverable writing, and Excel workbook construction. *Iterative refinement* aggregates multi-turn revise/polish intents (Doc/PDF iterative refinement and the multi-turn share of PPT sessions). *Formatting / templates* aggregates style, formatting, and template-compliance requests (Doc/PDF format/style and PPT formatting/template). *QA / understanding* aggregates standalone question-answering or comprehension requests that do not modify an artifact. *Cross-artifact workflow* has no dedicated online label because the exports are classified per single-artifact family; it is therefore reported as "—" on the demand side. Because these categories cut across three classifiers built for different software families, the resulting shares are approximate aggregates rather than exact partitions; the headline percentages should be read at the level of "dominant vs. marginal" rather than as precise point estimates, and the residual mass not falling cleanly into one category is omitted from the table rather than forced into it. The benchmark

counts in the same table are likewise coarse: they count surveyed benchmarks per category and are not weighted by benchmark size, task difficulty, or evaluation maturity, so they indicate relative attention across categories rather than a measure of evaluation capacity.

Relation to Figure 8. Table 2 and Figure 8 use the same demand-side evidence but at different resolutions. The table reports the canonical cross-family aggregation used for the benchmark-gap argument: six coarse task categories whose shares are computed by conversation count across Excel, Doc/PDF, and PPT. Figure 8 instead breaks this aggregation into artifact-family sub-tasks so that the reader can see where the mismatch appears within spreadsheets, documents, and presentations. These sub-task labels are therefore not a second ontology and should not be read as a one-to-one partition of the table rows. Their percentages inherit the artifact-family label spaces used for each dashboard and should be interpreted directionally, not as an exact partition that sums to the table rows. Benchmark counts in the figure are mapped to the closest artifact-specific sub-task labels, whereas Table 2 maps surveyed benchmark counts to the broader task categories. For example, the coarse category *Direct execution* covers several Figure 8 spreadsheet and editing points such as table structuring, data transformation, formula/calculation, content editing, and format conversion; *Artifact generation* covers deck creation, document writing, and related content-generation points; *Iterative refinement* covers revision/editing and session-level refinement; *Formatting / templates* covers styling, design, and template-compliance points; and *QA / understanding* covers the QA and comprehension points. *Cross-artifact workflow* remains unavailable as a demand-side label in Table 2, even though cross-artifact benchmarks are discussed in the main text. Accordingly, the table should be used for the headline cross-family percentages, while Figure 8 should be read as a diagnostic visualization of which artifact-specific sub-tasks drive the same imbalance.

E.2. Task-Family Profiles

Excel requests are dominated by concrete execution. The most common user intent is direct operation execution (80.4%), and the operation labels show that users most often ask the assistant to manage table structure (52.7%), clean or transform data (30.0%), and produce usable spreadsheet artifacts. This demand is not limited to one domain: workplace and industry (26.1%), education (23.7%), administration and government (19.7%), general office (15.8%), and finance (6.5%) all appear prominently. The verification-risk distribution is also important: although many spreadsheet requests look operational, 30.5% carry high or critical verification risk because errors in financial, research, administrative, or operational data can propagate into downstream decisions.

Doc/PDF requests show a broader mixture of generation, revision, formatting, and conversion. Compared with Excel, document queries contain more iterative refinement (19.5%) and deliverable-oriented writing (15.8%), because users frequently ask the assistant to draft, revise, polish, convert, or restructure documents. The largest contexts are education and academic (43.8%) and general office work (28.3%), followed by administration (12.4%) and industry use (6.6%). Verification risk remains non-trivial: 25.6% of Doc/PDF requests are high or critical risk, even when the surface action is "write," "revise," or "convert," because correctness may depend on source fidelity, domain knowledge, institutional style, or legal and financial sensitivity.

PPT requests are the most deliverable-oriented of the three families. The dominant pattern is not a small edit or a standalone answer, but a request to create a presentation file (82.7% deliverable generation), either from a plain-text prompt (61.1%) or from uploaded source material (single file 17.0%, text with image 12.3%, multi-file 9.6%). This is consistent with the role of presentations as synthesis and delivery artifacts: users often provide source content, ask the assistant to structure it into a deck, and then iterate on style, outline, or content. The complexity distribution shows that 69.3% are medium complexity and 10.2% are complex tasks, while 42.9% require multi-turn iterative interaction.

E.3. Summary of Observed Demand

Across the three software families, online users more often ask for bounded artifact work than for open-ended reasoning in isolation. For Excel, 80.4% of successful classifications are direct-execution requests, and 82.7% of operations fall into structure management or data transformation. For Doc/PDF, 88.2% of requests are concentrated in generation, revision, formatting, or conversion. For PPT, 82.7% of requests are presentation-generation intents, and 83.0% of action labels are either creation from scratch or creation from source files. These distributions suggest a practical interpretation of AI Office demand: users often seek systems that can take

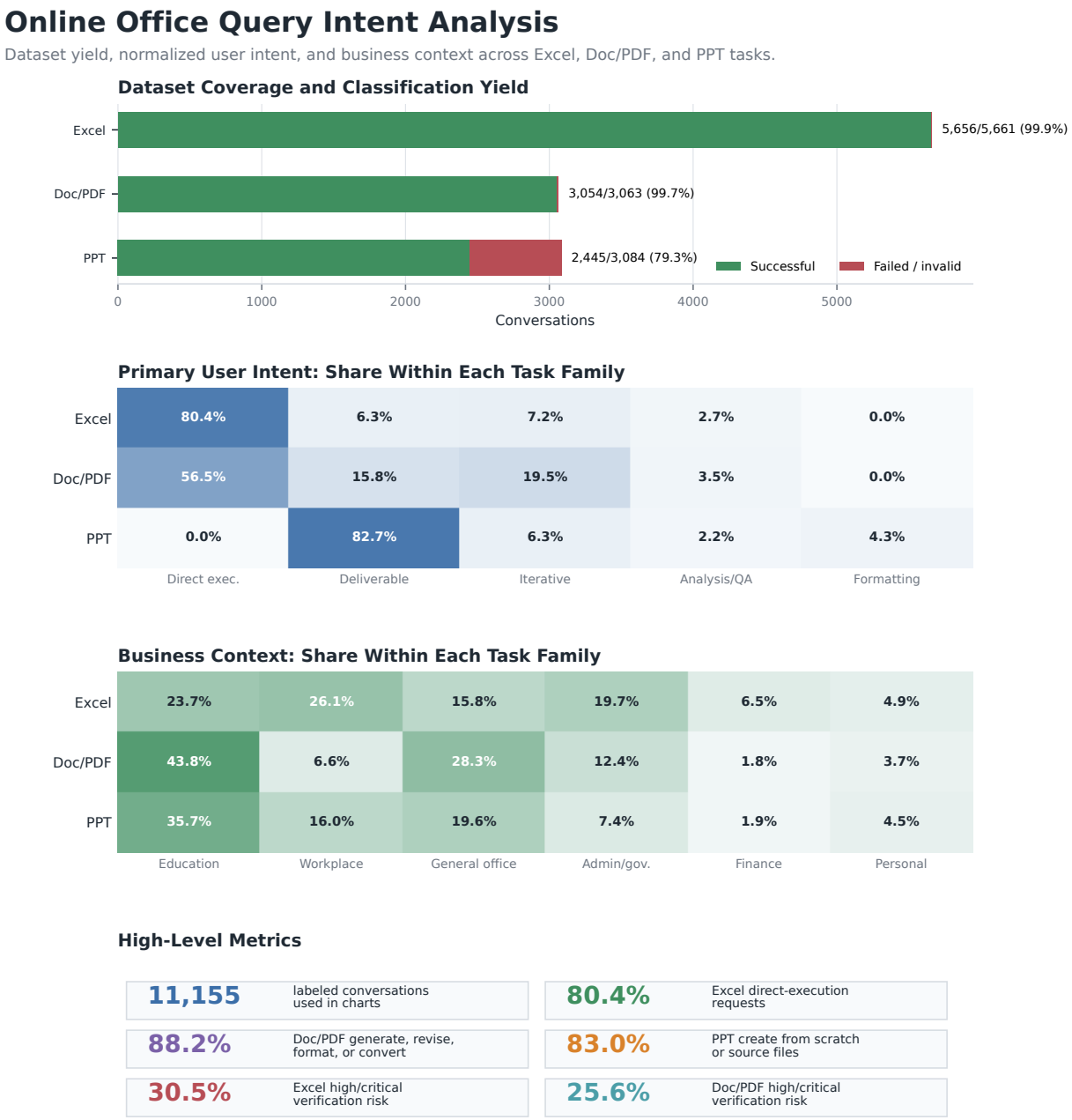


Figure 10 | **Overview of the analyzed online Office query exports.** The dashboard summarizes dataset yield, normalized user-intent distributions, business-context distributions, and the main numerical takeaways. Heatmap cells report within-family shares, and the bottom metric cards define each headline number.

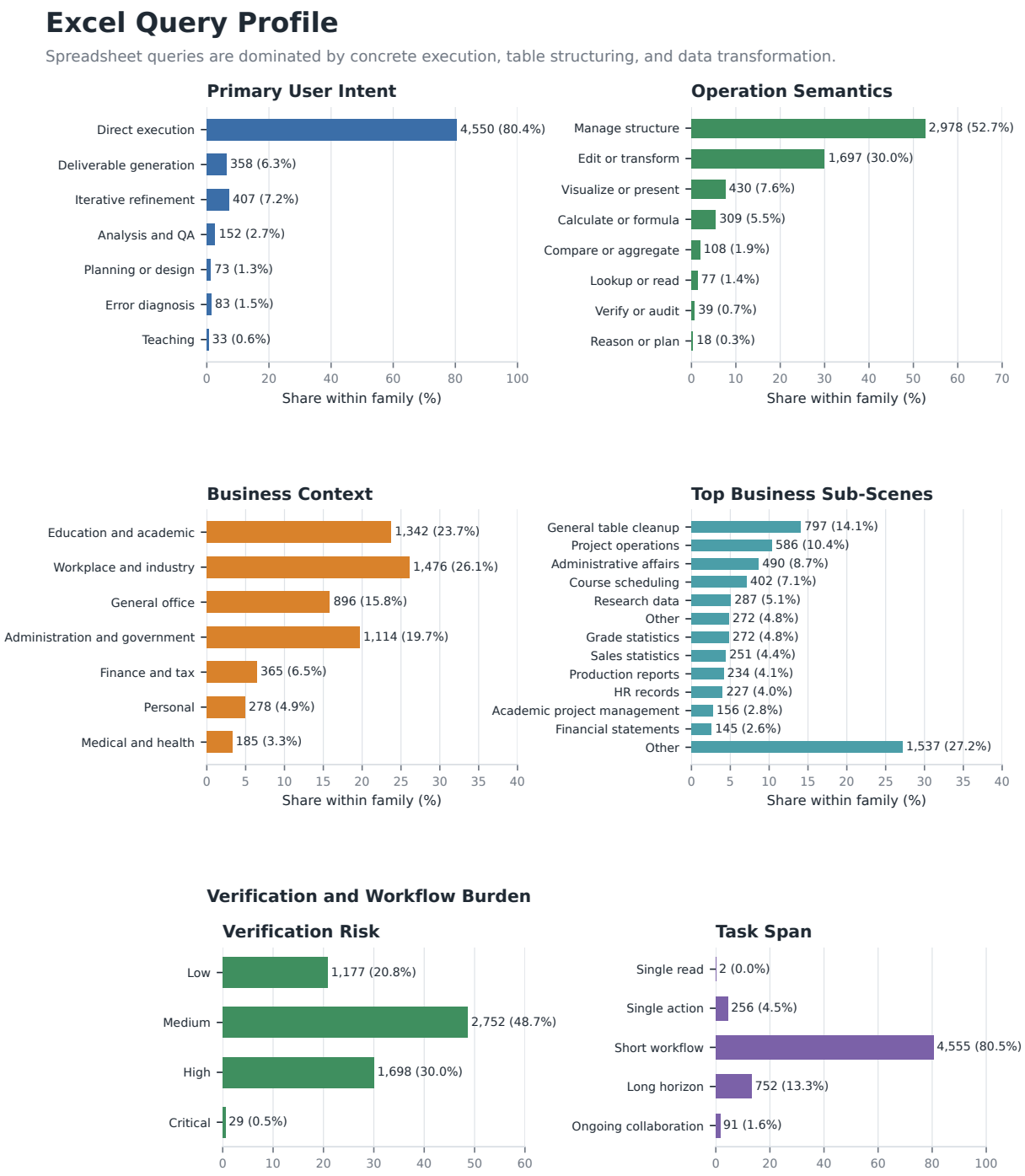


Figure 11 | **Excel query profile.** The dashboard combines primary user intent, operation semantics, business context, top sub-scenes, verification risk, and task span.

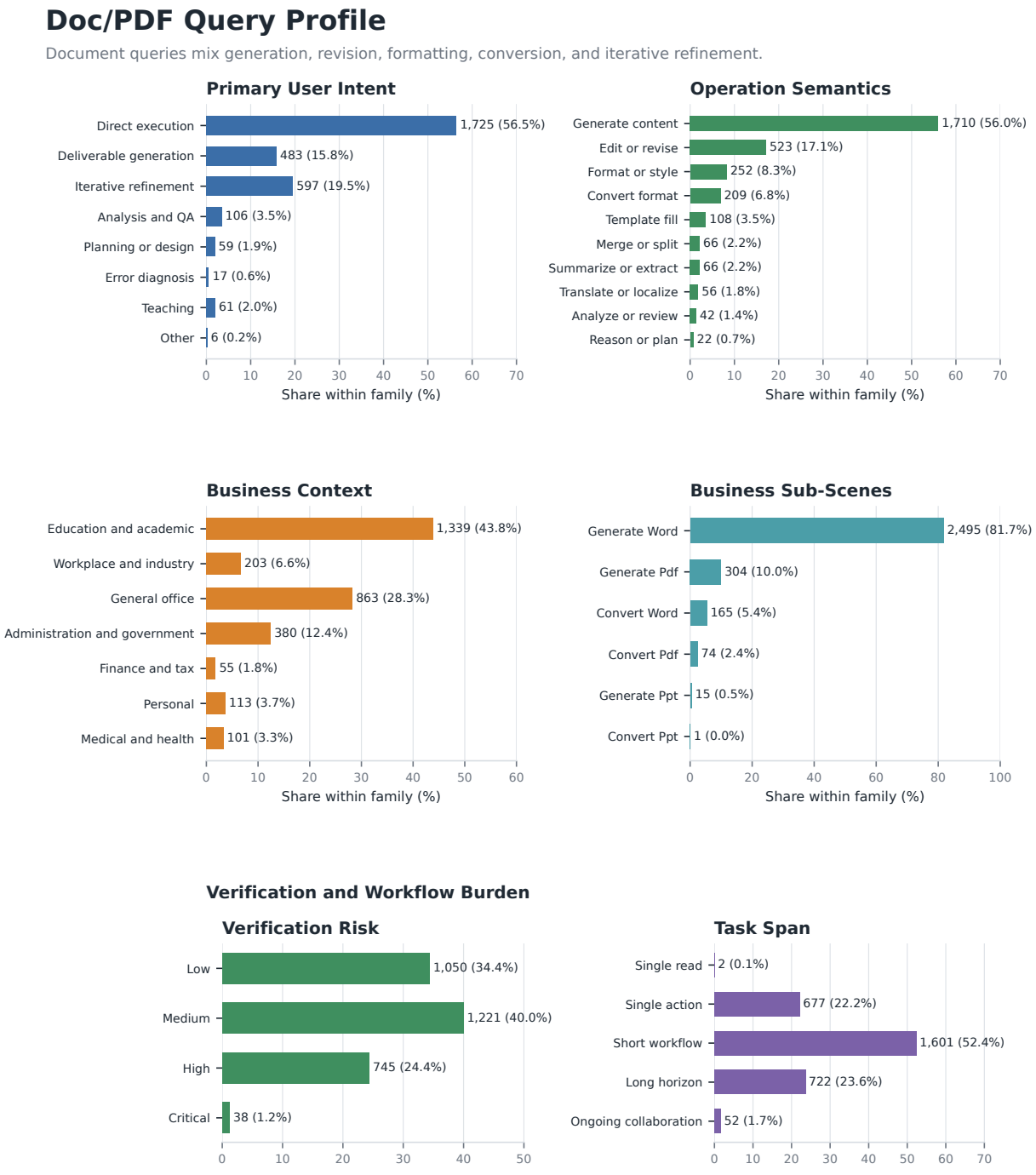


Figure 12 | **Doc/PDF query profile.** The dashboard combines primary user intent, operation semantics, business context, business sub-scenes, verification risk, and task span.

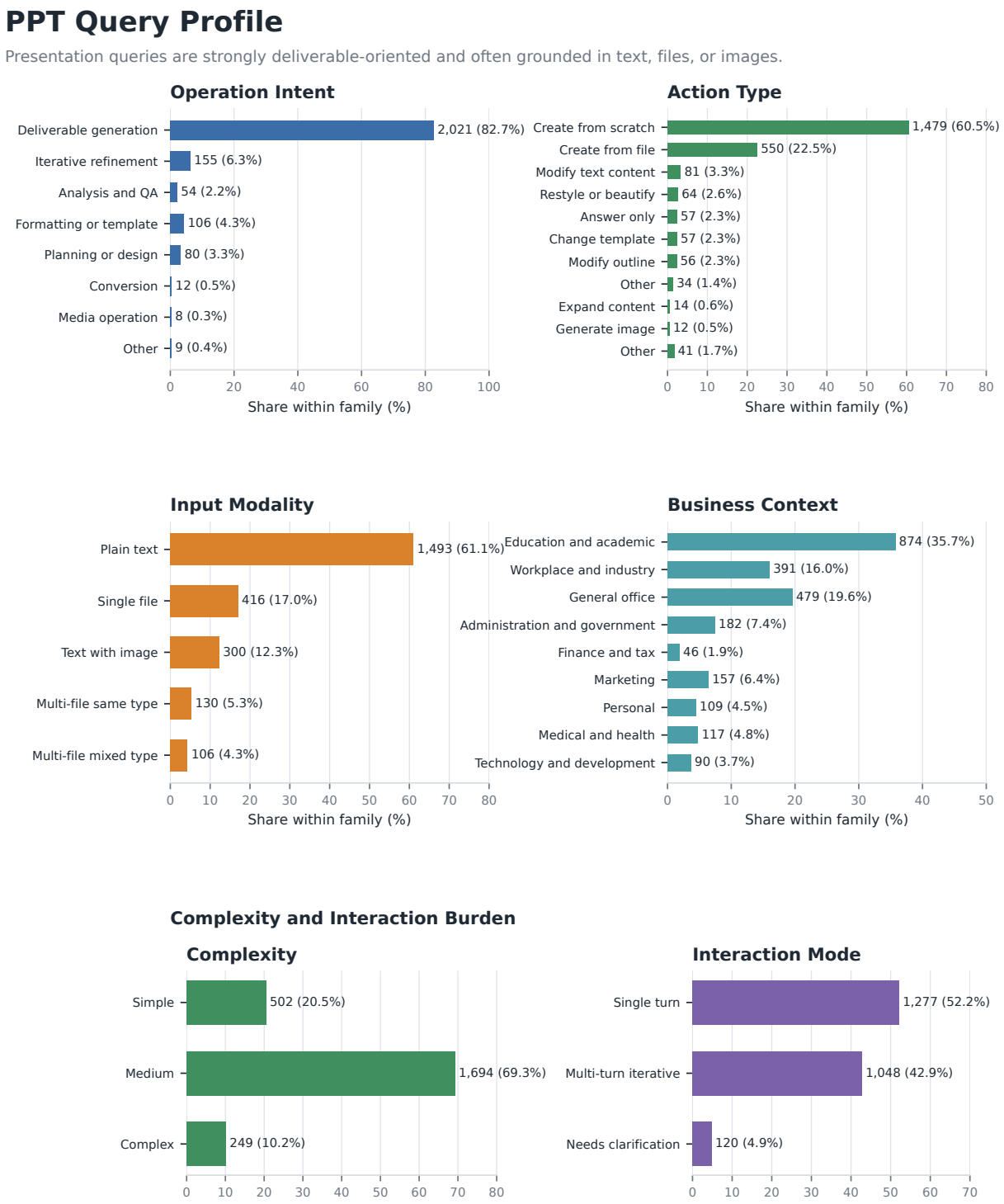


Figure 13 | **PPT query profile.** The dashboard combines operation intent, action type, input modality, business context, complexity, and interaction mode.

messy instructions and source materials, perform concrete artifact operations, and return editable deliverables. At the same time, the verification-risk labels for Excel and Doc/PDF show that many apparently routine requests still require careful checking. In the analyzed successful classifications, 30.5% of Excel requests and 25.6% of Doc/PDF requests carry high or critical verification risk. The product and evaluation challenge is therefore not only to generate plausible office artifacts, but to reduce the user's cost of checking whether those artifacts are correct, faithful, and appropriate for the target work context.

F. Representative Work by Training Paradigm

Table 8 unpacks the compressed training overview in §5. The important distinction is not only whether a method uses SFT, OPD, or RL, but what office role the signal serves: demonstrations teach valid operations and local conventions, OPD teaches recovery on the agent's own trajectories, verifiable checkers reward executable or rendered artifact states, and adaptation methods align outputs with local organizational standards. Rows labeled GUI, Web, Computer use, or Code/data are adjacent training substrates rather than office deliverable methods by themselves; they are included only where their supervision signal transfers to AI Office through GUI grounding, environment feedback, executable checks, or multi-step tool use. The reward/verifier column records the kind of signal relevant to the survey's framework; it does not imply that every listed method explicitly trains on that reward.

Table 8 | Per-work view of office-agent training, with columns finer than the master crosswalk (Table 4): training *stage*, *supervision source*, the reward/verifier *type* used in the training taxonomy (Rule-based, Visual, Interactive, Similarity), and the optimization *algorithm*. The reward/verifier type records the signal role relevant to AI Office and does not imply that every listed method explicitly optimizes that reward. "—" denotes no explicit reward (imitation/pretraining) or an unreported field.

Work	Artifact	Stage	Supervision source	Reward type	Algorithm
Imitation (SFT / pretraining)					
SpreadsheetLLM (Dong et al., 2024)	Spreadsheet	Pretrain / SFT	Self-supervised structure	—	—
SheetCopilot (Li et al., 2023a)	Spreadsheet	SFT	Demonstration	Rule-based	—
NL2Formula (Zhao et al., 2024)	Spreadsheet	SFT	Demonstration	Rule-based	—
SpreadsheetCoder (Chen et al., 2021b)	Spreadsheet	SFT	Demonstration	Rule-based	—
InstructExcel (Payan et al., 2023)	Spreadsheet	SFT	Demonstration	Rule-based	—
PPTAgent (Zheng et al., 2025)	Slides	SFT	Demonstration	Visual	—
AutoPresent (Ge et al., 2025)	Slides	SFT	Demonstration	Visual / Rule	—
PosterGen (Zhang et al., 2025b)	Posters	SFT	Demonstration	Visual	—
Doc2PPT (Fu et al., 2022)	Slides	SFT	Demonstration	Visual	—
D2S (Sun et al., 2021)	Slides	SFT	Demonstration	Visual	—
DocLLM (Wang et al., 2024b)	Document	Pretrain	Self-supervised layout	—	—
LayoutLMv2 (Xu et al., 2021)	Document	Pretrain	Self-supervised layout	—	—
LayoutLMv3 (Huang et al., 2022)	Document	Pretrain + SFT	Self-supervised layout	—	—
mPLUG-DocOwl2 (Hu et al., 2025a)	Document	SFT	Demonstration	—	—
CoEdIT (Raheja et al., 2023)	Document	SFT	Demonstration	Similarity	—
PEER (Schick et al., 2022)	Document	SFT	Demonstration	Similarity	—
PDFTriage (Saad-Falcon et al., 2023)	PDF	SFT	Demonstration	Rule-based	—
SeeClick (Cheng et al., 2024)	GUI (adj.)	SFT	Demonstration	Interactive	grounding
CogAgent (Hong et al., 2024)	GUI (adj.)	SFT	Demonstration	Interactive	grounding
Mind2Web (Deng et al., 2023)	Web (adj.)	SFT	Demonstration	Interactive	tool navigation
Reinforcement learning					
Spreadsheet-RL (Chi et al., 2026)	Spreadsheet	RL	Verifiable checker	Rule-based	online RL
AeSlides (Pan et al., 2026)	Slides	RL	Verifiable checker	Rule / Visual	GRPO
SlideRL (Ananda Kumar & Arunachalam, 2026)	Slides	RL	Verif. + model judge	Visual / Sim.	GRPO
ComputerRL (Lai et al., 2025)	Computer (adj.)	RL	Environment pass/fail	Interactive	online RL
DART (Li et al., 2025)	GUI (adj.)	RL	Environment pass/fail	Interactive	async RL
UI-TARS (Qin et al., 2025)	GUI (adj.)	SFT + RL	Demonstration + env.	Interactive	SFT+RL
GLM Slide Agent (Z.AI, 2026)	Slides	SFT+RL+RS	Verifiable + curated	Rule / Visual	RL + RS
System-scale, multi-agent, and adaptation					

Work	Artifact	Stage	Supervision source	Reward type	Algorithm
GLM-5 (Team et al., 2026a)	Multi	Multi-stage RL	Verifiable + agentic	Rule + agentic	SFT→RL
On-policy distillation (Song & Zheng, 2026; Li et al., 2026b)	Multi	Post-training	Teacher on student rollouts	Dense / verifier	OPD
Kimi K2.5 / PARL (Team et al., 2026b)	Multi	Multi-agent RL	Task + delegation	Composite	PARL
DaCode (Huang et al., 2024)	Code/data (adj.)	SFT + RL	Execution pass/fail	Rule-based	SFT + RL
DigiRL (Bai et al., 2024a)	App (adj.)	RL	Environment outcome	Interactive	online RL
Agent-as-a-Judge (Zhuge et al., 2024)	Multi	Verifier	Agentic inspection	Visual / Sim.	—
Copilot Tuning (Microsoft Learn, 2026b)	Multi	Adaptation	Local rubrics / examples	Similarity	SFT + adapt.